

Нормальные формы

Лекция

Две основные проблемы проектирования

Логическая проблема проектирования

Каким образом отобразить объекты предметной области в абстрактные объекты модели данных, чтобы:

- это отображение не противоречило семантике предметной области?
- Это отображение было лучшим/эффективным/удобным и т. д.)?

Физическая проблема проектирования

Как обеспечить эффективность выполнения запросов к базе данных?

Нормализация отношений

Проблема проектирования реляционной базы данных состоит в:

- обоснованном принятии решений о том, из каких отношений должна состоять база данных
- какие атрибуты должны быть у этих отношений.

Сначала предметная область представляется в виде одного или нескольких отношений, а далее осуществляется процесс нормализации схем отношений.

Основные свойства нормальных форм:

- Каждая следующая нормальная форма обладает свойствами лучшими, чем предыдущая.
- При переходе к следующей нормальной форме свойства предыдущих нормальных свойств сохраняются.
- Каждой нормальной форме соответствует некоторый определенный набор ограничений.
- Отношение находится в некоторой нормальной форме, если удовлетворяет свойственному ей набору ограничений.

Первая нормальная форма (1НФ или 1NF)

Определение:

Отношение находится в 1НФ, если все его атрибуты являются простыми, все используемые домены должны содержать только скалярные значения.

Простыми словами:

- Не должно быть повторений строк в таблице
- Не должно быть данных, логически соединенных в одну ячейку

Первая нормальная форма (1НФ или 1NF)

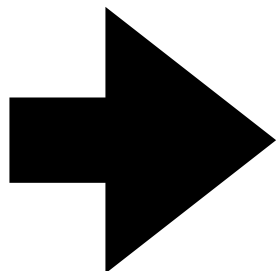
Пример

Предположим, что мы храним данные дилерского центра.

В отношении присутствуют следующие ФЗ:

{Фирма} -> {Модели}

Фирма	Модели
BMW	M5, X5M, M1
Nissan	GT-R



Фирма	Модели
BMW	M5
BMW	X5M
BMW	M1
Nissan	GT-R

Вторая нормальная форма (2НФ или 2NF)

Определение:

Отношение находится во 2НФ, если:

- Отношение находится в 1НФ
- Каждый не ключевой атрибут неприводимо зависит от Первичного Ключа(ПК).

Неприводимость означает, что в составе потенциального ключа отсутствует меньшее подмножество атрибутов, от которого можно также вывести данную функциональную зависимость.

Простыми словами:

- Не должно быть повторений строк в таблице
- Не должно быть данных, логически соединенных в одну ячейку
- Любой атрибут, который не является ключевым, должен зависеть от всех атрибутов ключа. Не от его части

Вторая нормальная форма (2НФ или 2NF)

Пример

Предположим, что мы храним данные дилерского центра.

Какие ключ присутствуют в отношении?

Какие ФЗ присутствуют в отношении?

Модель	Фирма	Цена	Скидка
A5	BMW	5500000	5%
A3	BMW	6000000	5%
A1	BMW	2500000	5%
A5	Nissan	5000000	10%

Вторая нормальная форма (2НФ или 2NF)

Пример

Предположим, что мы храним данные дилерского центра.

Ключ: {Модель, Фирма}

В отношении присутствуют следующие ФЗ:

- {Модель, Фирма} -> {Цена}
- {Модель, Фирма} -> {Скидка}
- {Фирма} -> {Скидка}

Модель	Фирма	Цена	Скидка
A5	BMW	55000000	5%
A3	BMW	60000000	5%
A1	BMW	25000000	5%
A5	Nissan	50000000	10%

Вторая нормальная форма (2НФ или 2NF)

Пример

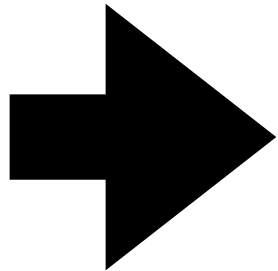
Предположим, что мы храним данные дилерского центра.

Ключ: {Модель, Фирма}

В отношении присутствуют следующие ФЗ:

- {Модель, Фирма} -> {Цена}
- {Модель, Фирма} -> {Скидка}
- {Фирма} -> {Скидка}

Модель	Фирма	Цена	Скидка
A5	BMW	55000000	5%
A3	BMW	60000000	5%
A1	BMW	25000000	5%
A5	Nissan	50000000	10%



Модель	Фирма	Цена
A5	BMW	55000000
A3	BMW	60000000
A1	BMW	25000000
A5	Nissan	50000000

Фирма	Скидка
BMW	5%
Nissan	10%

Третья нормальная форма (3НФ или 3NF)

Определение:

Отношение находится в 3НФ, когда:

- Находится во 2НФ
- Каждый не ключевой атрибут не транзитивно зависит от первичного ключа.

Нетранзитивная зависимость — это функциональная зависимость, которая не является транзитивной.

Простыми словами:

- Не должно быть повторений строк в таблице
- Не должно быть данных, логически соединенных в одну ячейку
- Любой атрибут, который не является ключевым, должен зависеть от всех атрибутов ключа. Не от его части
- Все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы, должны быть вынесены в отдельные таблицы

Третья нормальная форма (3НФ или 3NF)

Пример

Предположим, что мы храним данные дилерского центра.

Какие ключ присутствуют в отношении?

Какие ФЗ присутствуют в отношении?

Модель	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12

Третья нормальная форма (3НФ или 3NF)

Пример

Предположим, что мы храним данные дилерского центра.

Ключ: {Модель}

В отношении присутствуют следующие ФЗ:

- {Модель} -> {Магазин}
- {Модель} -> {Телефон}
- {Магазин} -> {Телефон}

Модель	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12

Третья нормальная форма (3НФ или 3NF)

Пример

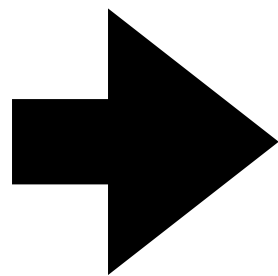
Предположим, что мы храним данные дилерского центра.

Ключ: {Модель}

В отношении присутствуют следующие ФЗ:

- {Модель} -> {Магазин}
- {Модель} -> {Телефон}
- {Магазин} -> {Телефон}

Модель	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12



Модель	Магазин
BMW	Риал-авто
Audi	Риал-авто
Nissan	Некст-Авто

Магазин	Телефон
Риал-авто	87-33-98
Некст-Авто	94-54-12

Третья нормальная форма (3НФ или 3NF)

Определение:

Отношение находится в НФБК, когда:

- Находится в 2НФ
- Каждая нетривиальная и неприводимая слева функциональная зависимость обладает потенциальным ключом в качестве детерминанта.

Для отношений, имеющих один потенциальный ключ (первичный), НФБК является 3НФ.

Простыми словами:

- Не должно быть повторений строк в таблице
- Не должно быть данных, логически соединенных в одну ячейку
- Любой атрибут, который не является ключевым, должен зависеть от всех атрибутов ключа. Не от его части
- Все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы, должны быть вынесены в отдельные таблицы

Нормальная форма Бойса-Кодда (НФБК)

Пример

Предположим, что мы храним данные сессии.

Какие ключ присутствуют в отношении?

Какие ФЗ присутствуют в отношении?

Курс	Группа	Экзкменатор	Телефон
1	М3239	Корнеев Г. А.	111-11-11
4	М3439	Корнеев Г. А.	111-11-11
1	М3238	Ведерников Н. В.	222-22-22
2	М3439	Киракозов А. Х.	333-33-33
3	М3239	Корнеев Г. А.	111-11-11
5	М3439	Корнеев Г. А.	111-11-11

Нормальная форма Бойса-Кодда (НФБК)

Пример

Предположим, что мы храним данные сессии.

Ключи:

- {Курс, Группа}
- {Курс, Экзаменатор}
- {Курс, Телефон}

В отношении присутствуют следующие ФЗ:

- {Курс, Группа} -> {Экзаменатор}
- {Курс, Экзаменатор} -> {Группа}
- {Экзаменатор} -> {Телефон}
- {Телефон} -> {Экзаменатор}

Курс	Группа	Экзкменатор	Телефон
1	М3239	Корнеев Г. А.	111-11-11
4	М3439	Корнеев Г. А.	111-11-11
1	М3238	Ведерников Н. В.	222-22-22
2	М3439	Киракозов А. Х.	333-33-33
1	М3239	Корнеев Г. А.	111-11-11
4	М3439	Корнеев Г. А.	111-11-11

Нормальная форма Бойса-Кодда (НФБК)

Пример

Если преподаватель ведет разные предметы у разных групп, такая структура отношения позволяет задать преподавателю разные телефоны, что нарушает целостность БД.

Почему так получилось?

- В данном отношении есть несколько ключей
- Каждый атрибут является частью хотя бы одного ключа (даже {Курс, Телефон} – это ключ)
- Первые три НФ не накладывают никакие ограничения на ключевые атрибуты

Курс	Группа	Экзкменатор
1	М3239	Корнеев Г. А.
4	М3439	Корнеев Г. А.
1	М3238	Ведерников Н. В.
2	М3439	Киракозов А. Х.
1	М3239	Корнеев Г. А.
4	М3439	Корнеев Г. А.

Экзкменатор	Телефон
Корнеев Г. А.	111-11-11
Ведерников Н. В.	222-22-22
Киракозов А. Х.	333-33-33

Четвертая нормальная форма (4НФ или 4NF)

Определение:

Отношение находится в 4НФ, когда:

- Находится в НФБК
- Все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от ее потенциальных ключей.

Простыми словами:

- Не должно быть повторений строк в таблице
- Не должно быть данных, логически соединенных в одну ячейку
- Любой атрибут, который не является ключевым, должен зависеть от всех атрибутов ключа. Не от его части
- Все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы, должны быть вынесены в отдельные таблицы
- Нет многозначных зависимостей

Многозначная зависимость

Пусть есть отношение $R(A, B, C)$

Определение:

В отношении R существует многозначная зависимость $A \twoheadrightarrow B$ в том и только в том случае, если множество значений B , соответствующее паре значений A и C , зависит только от A и не зависит от C .

Пример:

Дисциплина	Книга	Лектор
Мат. анализ	Кудрявцев	Иванов А.
Мат. анализ	Фихтенгольц	Петров Б.
Мат. анализ	Кудрявцев	Петров Б.
Мат. анализ	Фихтенгольц	Иванов А.
Мат. анализ	Кудрявцев	Смирнов В.
Мат. анализ	Фихтенгольц	Смирнов В.
ВМ	Кудрявцев	Иванов А.
ВМ	Кудрявцев	Петров Б.

Многозначные зависимости:

$\{Дисциплина\} \twoheadrightarrow \{Книга\}, \{Дисциплина\} \twoheadrightarrow \{Лектор\}$
 $\{Дисциплина\} \twoheadrightarrow \{Книга\} \mid \{Лектор\}$

Четвертая нормальная форма (4НФ или 4NF)

Предположим, что мы храним данные ресторанов доставки пиццы.

Какие ключ присутствуют в отношении?

Какие ФЗ присутствуют в отношении?

Ресторан	Вид пиццы	Район
A1 Пицца	Толстая корка	Springfield
A1 Пицца	Толстая корка	Shelbyville
A1 Пицца	Толстая корка	Столица
A1 Пицца	Фаршированная корочка	Springfield
A1 Пицца	Фаршированная корочка	Shelbyville
A1 Пицца	Фаршированная корочка	Столица
Элитная пицца	Толстая корка	Столица
Элитная пицца	Фаршированная корочка	Столица
Пицца Винченцо	Толстая корка	Springfield
Пицца Винченцо	Толстая корка	Shelbyville

Четвертая нормальная форма (4НФ или 4NF)

Предположим, что мы храним данные ресторанов доставки пиццы.

Ключи:

- {Ресторан, Вид пиццы, Район доставки}

В отношении присутствуют следующие ФЗ:

- {Ресторан} -> {Вид пиццы}
- {Ресторан} -> {Район доставки}

Ресторан	Вид пиццы	Район
A1 Пицца	Толстая корка	Springfield
A1 Пицца	Толстая корка	Shelbyville
A1 Пицца	Толстая корка	Столица
A1 Пицца	Фаршированная корочка	Springfield
A1 Пицца	Фаршированная корочка	Shelbyville
A1 Пицца	Фаршированная корочка	Столица
Элитная пицца	Толстая корка	Столица
Элитная пицца	Фаршированная корочка	Столица
Пицца Винченцо	Толстая корка	Springfield
Пицца Винченцо	Толстая корка	Shelbyville

Четвертая нормальная форма (4НФ или 4NF)

При добавлении нового вида пиццы придется внести по одному новому кортежу для каждого района доставки.

Возможна логическая аномалия, при которой определенному виду пиццы будут соответствовать лишь некоторые районы доставки из обслуживаемых рестораном районов.

Почему так получилось?

- Все атрибуты входят в состав ключа отношения. В отношении нет не ключевых атрибутов

Ресторан	Район доставки
А1 Пицца	Springfield
А1 Пицца	Shelbyville
А1 Пицца	Столица
Элитная пицца	Столица
Пицца Винченцо	Springfield
Пицца Винченцо	Shelbyville

Ресторан	Вид пиццы
А1 Пицца	Толстая корка
А1 Пицца	Фаршированная корочка
Элитная пицца	Толстая корка
Элитная пицца	Фаршированная корочка
Пицца Винченцо	Толстая корка

Четвертая нормальная форма (4НФ или 4NF)

А что если в таблице будет не ключевой атрибут?

({Ресторан, Вид пиццы, Район доставки}) -> {Район доставки}

Ресторан	Вид пиццы	Район	Цена
A1 Пицца	Толстая корка	Springfield	100
A1 Пицца	Толстая корка	Shelbyville	150
A1 Пицца	Толстая корка	Столица	120
A1 Пицца	Фаршированная корочка	Springfield	100
A1 Пицца	Фаршированная корочка	Shelbyville	150
A1 Пицца	Фаршированная корочка	Столица	120
Элитная пицца	Толстая корка	Столица	200
Элитная пицца	Фаршированная корочка	Столица	250
Пицца Винченцо	Толстая корка	Springfield	300
Пицца Винченцо	Толстая корка	Shelbyville	300

Пятая нормальная форма (5НФ или 5NF)

Определение:

Отношение находится в 5НФ (проекционно-соединительная нормальная форма), когда:

- Находится в 4НФ
- Каждая нетривиальная зависимость определяется потенциальным ключом (ключами) этого отношения.

Продавец	Фирма	Товар
Иванов	Рога и Копыта	Пылесос
Иванов	Рога и Копыта	Хлебница
Петров	Безенчук&Ко	Сучкорез
Петров	Безенчук&Ко	Пылесос
Петров	Безенчук&Ко	Хлебница
Петров	Безенчук&Ко	Зонт
Сидоров	Безенчук&Ко	Пылесос
Сидоров	Безенчук&Ко	Телескоп
Сидоров	Рога и Копыта	Пылесос
Сидоров	Рога и Копыта	Лампа
Сидоров	Геркулес	Вешалка

Пятая нормальная форма (5НФ или 5NF)

Ключ:

{Продавец, Фирма, Товар}

ФЗ:

- {Продавец} -> {Фирма}
- {Фирма} -> {Товар}
- {Продавец} -> {Товар}

Продавец	Фирма
Иванов	Рога и Копыта
Петров	Безенчук&Ко
Сидоров	Безенчук&Ко
Сидоров	Рога и Копыта
Сидоров	Геркулес

Фирма	Товар
Рога и Копыта	Пылесос
Рога и Копыта	Хлебница
Рога и Копыта	Лампа
Безенчук&Ко	Сучкорез
Безенчук&Ко	Пылесос
Безенчук&Ко	Хлебница
Безенчук&Ко	Зонт
Безенчук&Ко	Телескоп
Геркулес	Вешалка

Продавец	Товар
Иванов	Пылесос
Иванов	Хлебница
Петров	Сучкорез
Петров	Пылесос
Петров	Хлебница
Петров	Зонт
Сидоров	Телескоп
Сидоров	Пылесос
Сидоров	Лампа
Сидоров	Вешалка

Шестая нормальная форма (6НФ или 6NF)

Определение:

Переменная отношения находится в шестой нормальной форме тогда и только тогда, когда она удовлетворяет всем нетривиальным зависимостям соединения.

Простыми словами:

Переменная находится в 6НФ тогда и только тогда, когда она неприводима, то есть не может быть подвергнута дальнейшей декомпозиции без потерь.

Таб. №	Время	Должность	Домашний адрес
6575	01-01-2000: 10-02-2003	слесарь	ул.Ленина,10
6575	11-02-2003: 15-06-2006	слесарь	ул.Советская, 22
6575	16-06-2006: 05-03-2009	бригадир	ул.Советская, 22

Шестая нормальная форма (6НФ или 6NF)

Идея «декомпозиции до конца» выдвигалась еще до начала исследований в области хронологических данных. Именно для хронологических баз данных максимально возможная декомпозиция позволяет бороться с избыточностью и упрощает поддержание целостности базы данных.

В теории проектирования хранилищ данных шестая нормальная форма называется «**Якорная модель**».

Таб. №	Время	Должность
6575	01-01-2000:10-02-2003	слесарь
6575	16-06-2006:05-03-2009	бригадир

Таб. №	Время	Домашний адрес
6575	01-01-2000:10-02-2003	ул.Ленина,10
6575	11-02-2003:15-06-2006	ул.Советская, 22

Пример приведения отношения к нормальным формам

Задача:

Дано отношение $R = \{A, B, C, D, E, F, G, H, I, J\}$ со следующим набором функциональных зависимостей $F = \{$

$\{A, B\} \rightarrow \{C\},$

$\{A\} \rightarrow \{D, E\},$

$\{B\} \rightarrow \{F\},$

$\{F\} \rightarrow \{G, H\},$

$\{D\} \rightarrow \{I, J\}$

$\}.$

Какие ключи есть в отношении R ? Декомпозируйте R в 2NF, а затем в 3NF.

Пример приведения отношения к нормальным формам

Найдем ключ отношения:

Для того чтобы найти ключ отношения необходимо вычислить замыкание атрибутов. Если замыкание атрибутов равно множеству всех атрибутов отношения, то множество атрибутов для которого рассчитывали замыкание - ключ.

Атрибуты которые есть только в левой части (точно должны быть в ключе): A, B

- $\{A\}^+ = \{A, D, E, I, J\}$
- $\{B\}^+ = \{B, F, G, H\}$
- $\{A, B\}^+ = \{A, B, C, D, E, F, G, H, I, J\} - R$

Набор $\{A, B\}$ - минимальный $\rightarrow$ это потенциальный ключ

Пример приведения отношения к нормальным формам

Приведем к 2NF:

Для приведения к 2NF, удалим атрибуты которые функционально зависят только от части ключа (A или B) в отношении R.

Декомпозируем на отдельные отношения R1 и R2 вместе с частью ключа. Остальные функциональные зависимости сохраняем в отношении R3:

- $R1 = \{A, D, E, I, J\}$, ключ $\{A\}$, ФЗ = $\{\{A\} \rightarrow \{D, E\}, \{D\} \rightarrow \{I, J\}\}$
- $R2 = \{B, F, G, H\}$, ключ $\{B\}$, ФЗ = $\{\{B\} \rightarrow \{F\}, \{F\} \rightarrow \{G, H\}\}$
- $R3 = \{A, B, C\}$, ключ $\{A, B\}$, ФЗ = $\{\{A, B\} \rightarrow \{C\}\}$

Пример приведения отношения к нормальным формам

Приведем к 3NF:

Устраняем транзитивные зависимости в отношениях R1, R2 и R3.

Отношение R1. Есть транзитивные зависимости $\{A\} \rightarrow \{D\} \rightarrow \{I, J\}$, разбиваем отношение по зависимостям:

- R11 = {D, I, J}, ключ {D}, ФЗ = $\{\{D\} \rightarrow \{I, J\}\}$
- R12 = {A, D, E} ключ {A}, ФЗ = $\{\{A\} \rightarrow \{D, E\}\}$

Отношение R2. Есть транзитивные зависимости $\{B\} \rightarrow \{F\}, \{F\} \rightarrow \{G, H\}$, разбиваем отношение по зависимостям:

- R21 = {F, G, H}, ключ {F}, ФЗ = $\{\{F\} \rightarrow \{G, H\}\}$
- R22 = {B, F} ключ {B}, ФЗ = $\{\{B\} \rightarrow \{F\}\}$

Отношение R3. Одна ФЗ, транзитивных ФЗ нет.

Пример приведения отношения к нормальным формам

Задача:

Дано отношение $R = \{A, B, C, D, E, F, G, H, I\}$
со следующим набором ФЗ

$F = \{$

$\{A, B\} \rightarrow \{C\},$

$\{A\} \rightarrow \{D, E\},$

$\{B\} \rightarrow \{F\},$

$\{F\} \rightarrow \{G, H\},$

$\{D\} \rightarrow \{I, J\}$

$\}.$

Какие ключи есть в отношении R?

Декомпозируйте R в 2NF, а затем в 3NF.

Ключ отношения R: $\{A, B\}$

Отношение R в 2NF имеет вид:

- $R1 = \{A, D, E, I, J\}$, ключ $\{A\}$
- $R2 = \{B, F, G, H\}$, ключ $\{B\}$
- $R3 = \{A, B, C\}$, ключ $\{A, B\}$

Отношение R в 3NF имеет вид:

- $R11 = \{D, I, J\}$, ключ $\{D\}$
- $R12 = \{A, D, E\}$ ключ $\{A\}$
- $R21 = \{F, G, H\}$, ключ $\{F\}$
- $R22 = \{B, F\}$ ключ $\{B\}$
- $R3 = \{A, B, C\}$, ключ $\{A, B\}$

Пример приведения отношения к нормальным формам

R? Декомпозируйте R в 2NF, а затем в 3NF.

Пример приведения отношения к нормальным формам

Дана схема отношения $R(A, B, C, D, E, F, G, H, I, J)$, для которой выполняется множество функциональных зависимостей:

$S = \{$
 $AB \rightarrow C,$
 $A \rightarrow DE,$
 $B \rightarrow F,$
 $B \rightarrow G,$
 $F \rightarrow G,$
 $D \rightarrow HIJ$
 $\}.$

- Выполняются ли функциональные зависимости $AF \rightarrow BI$ и $AB \rightarrow DJ$ для R ? Ответ пояснить.
- Найти все потенциальные ключи для R .
- Показать этапы преобразования R в BCNF.

Блокировки

Блокировки

Блокировка — это механизм, с помощью которого компонент Database Engine синхронизирует одновременный доступ нескольких пользователей к одному фрагменту данных.

Каждая блокировка обладает тремя свойствами:

- гранулярностью (или размером блокировки)
- режимом (или типом блокировки)
- продолжительностью

Гранулярность блокировок

Ресурс	Описание
RID	Идентификатор строки, используемый для блокировки одной строки в куче
KEY	Блокировка строки в индексе, используемая для защиты диапазонов значений ключа в сериализуемых транзакциях.
PAGE	8-килобайтовая (КБ) страница в базе данных, например страница данных или индекса.
EXTENT	Упорядоченная группа из восьми страниц, например страниц данных или индекса.
NOBT	Куча или сбалансированное дерево. Блокировка, защищающая индекс или кучу страниц данных в таблице, не имеющей кластеризованного индекса.
TABLE	Таблица полностью, включая все данные и индексы.
FILE	Файл базы данных.
APPLICATION	Определяемый приложением ресурс.
METADATA	Блокировки метаданных.
ALLOCATION_UNIT	Единица размещения.
DATABASE	База данных, полностью.

Режимы блокировки

Режим блокировки	Описание
Совмещаемая блокировка (S)	Используется для операций считывания, которые не меняют и не обновляют данные, такие как инструкция SELECT.
Блокировка обновления (U)	Применяется к тем ресурсам, которые могут быть обновлены. Предотвращает возникновение распространенной формы взаимоблокировки, возникающей тогда, когда несколько сеансов считывают, блокируют и затем, возможно, обновляют ресурс.
Монопольная блокировка (X)	Используется для операций модификации данных, таких как инструкции INSERT, UPDATE или DELETE. Гарантирует, что несколько обновлений не будет выполнено одновременно для одного ресурса.
Блокировка с намерением	Используется для создания иерархии блокировок. Типы намеренной блокировки: с намерением совмещаемого доступа (IS), с намерением монопольного доступа (IX), а также совмещаемая с намерением монопольного доступа (SIX).
Блокировка схемы	Используется во время выполнения операции, зависящей от схемы таблицы. Типы блокировки схем: блокировка изменения схемы (Sch-S) и блокировка стабильности схемы (Sch-M).
Блокировка массового обновления (BU)	Используется, если выполняется массовое копирование данных в таблицу и указана подсказка TABLOCK.
Диапазон ключей	Защищает диапазон строк, считываемый запросом при использовании уровня изоляции сериализуемой транзакции. Запрещает другим транзакциям вставлять строки, что помогает запросам сериализуемой транзакции уточнять, были ли запросы запущены повторно.

Совместимость блокировок

Совместимость блокировок определяет, могут ли несколько транзакций одновременно получить блокировку одного и того же ресурса.

Если ресурс уже заблокирован другой транзакцией, новая блокировка может быть предоставлена только в том случае, если режим запрошенной блокировки совместим с режимом существующей. В противном случае транзакция, запросившая новую блокировку, ожидает освобождения ресурса, пока не истечет время ожидания существующей блокировки.

Продолжительность блокировки

Блокировки для выполнения инструкций `SELECT` выполняются только на время чтения ресурса, но не во время запроса.

Блокировки для инструкций `INSERT`, `UPDATE` и `DELETE` сохраняются на все время выполнения запроса. Это помогает гарантировать согласованность данных и позволяет откатывать запросы в случае необходимости.

Когда запрос выполняется в рамках транзакции, продолжительность блокировки определяется тремя факторами:

- типом запроса
- уровнем изоляции транзакции
- наличием или отсутствием подсказок блокировки

Эскалация блокировок

Эскалация (lock escalation) связана с тем, что по мере увеличения, количества отдельных малых заблокированных объектов накладные расходы, связанные с их поддержкой, начинают значительно сказываться на производительности. Блокировки длятся дольше, что приводит к спорным ситуациям — чем дольше существует блокировка, тем выше вероятность обращения к заблокированному объекту со стороны другой транзакции.

Очевидно, что на некотором этапе потребуется выполнить объединение (увеличение масштаба) блокировок, чем собственно и занимается диспетчер блокировок.

Взаимоблокировки транзакций



Как выбирается жертва взаимоблокировки?

По умолчанию в качестве жертвы взаимоблокировки выбирается сеанс, выполняющий ту транзакцию, откат которой потребует меньше всего затрат.

В качестве альтернативы пользователь может указать приоритет сеансов. Если у двух сеансов имеются различные приоритеты, то в качестве жертвы взаимоблокировки будет выбран сеанс с более низким приоритетом. Если у обоих сеансов установлен одинаковый приоритет, то в качестве жертвы взаимоблокировки будет выбран сеанс, откат которого потребует наименьших затрат.

Если сеансы, вовлеченные в цикл взаимоблокировки, имеют один и тот же приоритет и одинаковую стоимость, то жертва взаимоблокировки выбирается случайным образом.

Транзакции и уровни ИЗОЛЯЦИИ

Транзакции

Транзакция — это последовательность операций, выполняемая как единое целое.

Для поддержания целостности транзакция должна обладать четырьмя свойствами АСИД: атомарность, согласованность, изоляция и долговечность. Эти свойства называются также ACID-свойствами (от англ., atomicity, consistency, isolation, durability).

- Атомарность (Atomicity)
- Согласованность (или Непротиворечивость) (Consistency)
- Изоляция (Isolation)
- Долговечность (или Устойчивость) (Durability)

Транзакции

По признаку определения границ различают:

- автоматические транзакции
- неявные транзакции
- явные транзакции

Автоматические транзакции

Режим автоматической фиксации транзакций является режимом управления транзакциями по умолчанию. В этом режиме каждая инструкция T-SQL выполняется как отдельная транзакция. Если выполнение инструкции завершается успешно, происходит фиксация; в противном случае происходит откат.

Если возникает ошибка компиляции, то план выполнения пакета не строится и пакет не выполняется.

Автоматические транзакции

```
CREATE TABLE Tab1 (  
    Col1 int NOT NULL PRIMARY KEY,  
    Col2 char(3)  
);
```

```
INSERT INTO Tab1 VALUES (1, 'aaa');  
INSERT INTO Tab1 VALUES (2, 'bbb');  
INSERT INTO Tab1 VALUES (1, 'ccc'); -- Ошибка времени исполнения.
```

```
SELECT * FROM Tab1; -- Возвращаются две строки.
```

Неявные транзакции

Если соединение работает в режиме неявных транзакций, то после фиксации или отката текущей транзакции автоматически начинает новую транзакцию. В этом режиме явно указывается только граница окончания транзакции с помощью инструкций COMMIT TRANSACTION и ROLLBACK TRANSACTION.

Неявные транзакции

```
CREATE TABLE Tab1 (  
    Col1 int NOT NULL PRIMARY KEY,  
    Col2 char(3) NOT NULL  
)
```

-- Первая неявная транзакция, начатая оператором INSERT

```
INSERT INTO Tab1 VALUES (1, 'aaa')
```

```
INSERT INTO Tab1 VALUES (2, 'bbb')
```

```
COMMIT TRANSACTION -- Фиксация первой транзакции
```

-- Вторая неявная транзакция, начатая оператором INSERT

```
INSERT INTO Tab1 VALUES (3, 'ccc')
```

```
SELECT * FROM Tab1
```

```
COMMIT TRANSACTION -- Фиксация второй транзакции
```


Явные транзакции

Для определения явных транзакций используются следующие инструкции:

- BEGIN TRANSACTION
- COMMIT TRANSACTION или COMMIT WORK
- ROLLBACK TRANSACTION или ROLLBACK WORK
- SAVE TRANSACTION

```
BEGIN TRANSACTION royaltychange
  UPDATE titleauthor
  SET royaltypers = 65
  FROM titleauthor, titles
  WHERE royaltypers = 75 AND titleauthor.title_id = titles.title_id
      AND title = 'The Gourmet Microwave';
  UPDATE titleauthor
  SET royaltypers = 35
  FROM titleauthor, titles
  WHERE royaltypers = 25 AND titleauthor.title_id = titles.title_id
      AND title = 'The Gourmet Microwave'
SAVE TRANSACTION percentchanged
```

/* После того, как обновлено royaltypers для двух авторов, вставляется точка сохранения percentchanged, а затем определяется, насколько изменится заработок авторов после увеличения на 10 процентов цены книги */

```
UPDATE titles
SET price = price * 1.1
WHERE title = 'The Gourmet Microwave'
```

```
SELECT (price * royalty * ytd_sales) * royaltypers
FROM titles, titleauthor
WHERE title = 'The Gourmet Microwave' AND titles.title_id = titleauthor.title_id
```

```
/* Откат транзакции до точки сохранения и фиксация транзакции в целом */
ROLLBACK TRANSACTION percentchanged
COMMIT TRANSACTION
```

Явные транзакции

Управлением параллельным выполнением транзакций

Когда множество пользователей одновременно пытаются модифицировать данные в базе данных, необходимо создать систему управления, которая защитила бы модификации, выполненные одним пользователем, от негативного воздействия модификаций, сделанных другими. Выделяют два типа управления параллельным выполнением:

- **Пессимистическое** управление параллельным выполнением.
- **Оптимистическое** управление параллельным выполнением.

Управлением параллельным выполнением транзакций

Если в случае пессимистического управления в СУБД не реализованы механизмы блокирования, то при одновременном чтении и изменении одних и тех же данных несколькими пользователями могут возникнуть следующие проблемы одновременного доступа:

- **проблема последнего изменения**
- **проблема "грязного" чтения (Dirty Read)**
- **проблема неповторяемого чтения (Non-repeatable or Fuzzy Read)**
- **проблема чтения фантомов (Phantom)**

Управлением параллельным выполнением транзакций

уровень 0 – запрещение «загрязнения» данных. Этот уровень требует, чтобы изменять данные могла только одна транзакция; если другой транзакции необходимо изменить те же данные, она должна ожидать завершения первой транзакции;

уровень 1 – запрещение «грязного» чтения. Если транзакция начала изменение данных, то никакая другая транзакция не сможет прочитать их до завершения первой;

уровень 2 – запрещение неповторяемого чтения. Если транзакция считывает данные, то никакая другая транзакция не сможет их изменить. Таким образом, при повторном чтении они будут находиться в первоначальном состоянии;

уровень 3 – запрещение фантомов. Если транзакция обращается к данным, то никакая другая транзакция не сможет добавить новые или удалить имеющиеся строки, которые могут быть считаны при выполнении транзакции. Реализация этого уровня блокирования выполняется путем использования блокировок диапазона ключей. Подобная блокировка накладывается не на конкретные строки таблицы, а на строки, удовлетворяющие определенному логическому условию.

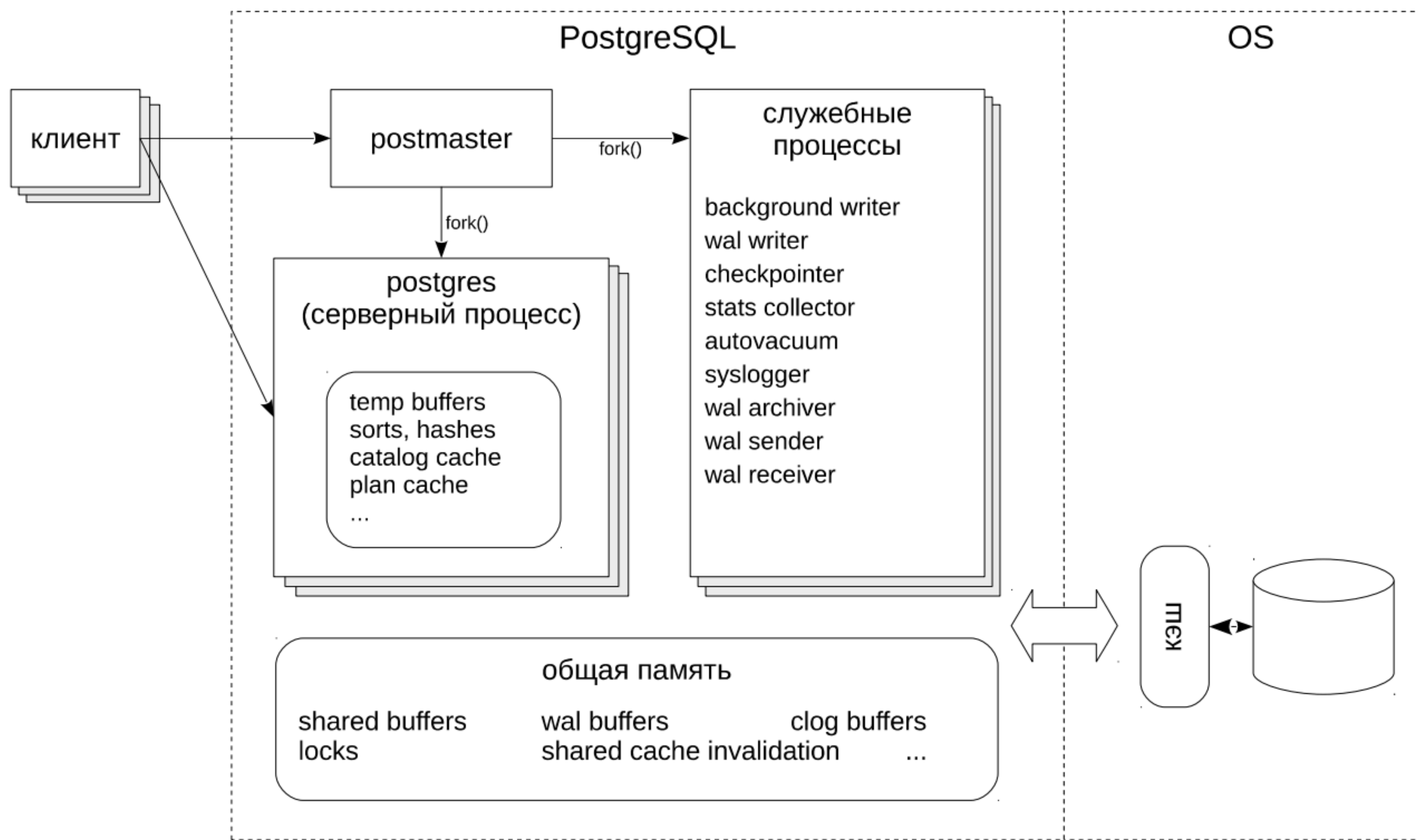
Управлением параллельным выполнением транзакций

Уровень изоляции	«Грязное» чтение	Неповторяющееся чтение	Фантомное чтение
read uncommitted	Да	Да	Да
read committed	Нет	Да	Да
repeatable read	Нет	Нет	Да
snapshot	Нет	Нет	Нет
serializable	Нет	Нет	Нет

Оптимизация запросов

План запроса

PostgreSQL



Пять стадий выполнения запроса

Клиентский запрос проходит следующие стадии:

- Прикладная программа устанавливает подключение к серверу PostgreSQL.
- На этапе разбора запроса сервер выполняет синтаксическую проверку запроса, переданного прикладной программой, и создаёт дерево запроса.
- Система правил принимает дерево запроса, созданное на стадии разбора, и ищет в системных каталогах правила для применения к этому дереву.
- Планировщик/оптимизатор принимает дерево запроса (возможно, переписанное) и создаёт план запроса, который будет передан исполнителю. Он выбирает план, сначала рассматривая все возможные варианты получения одного и того же результата.
- Исполнитель рекурсивно проходит по дереву плана и получает строки тем способом, который указан в плане.

Дерево разбора

Дерево разбора состоит из узлов двух типов:

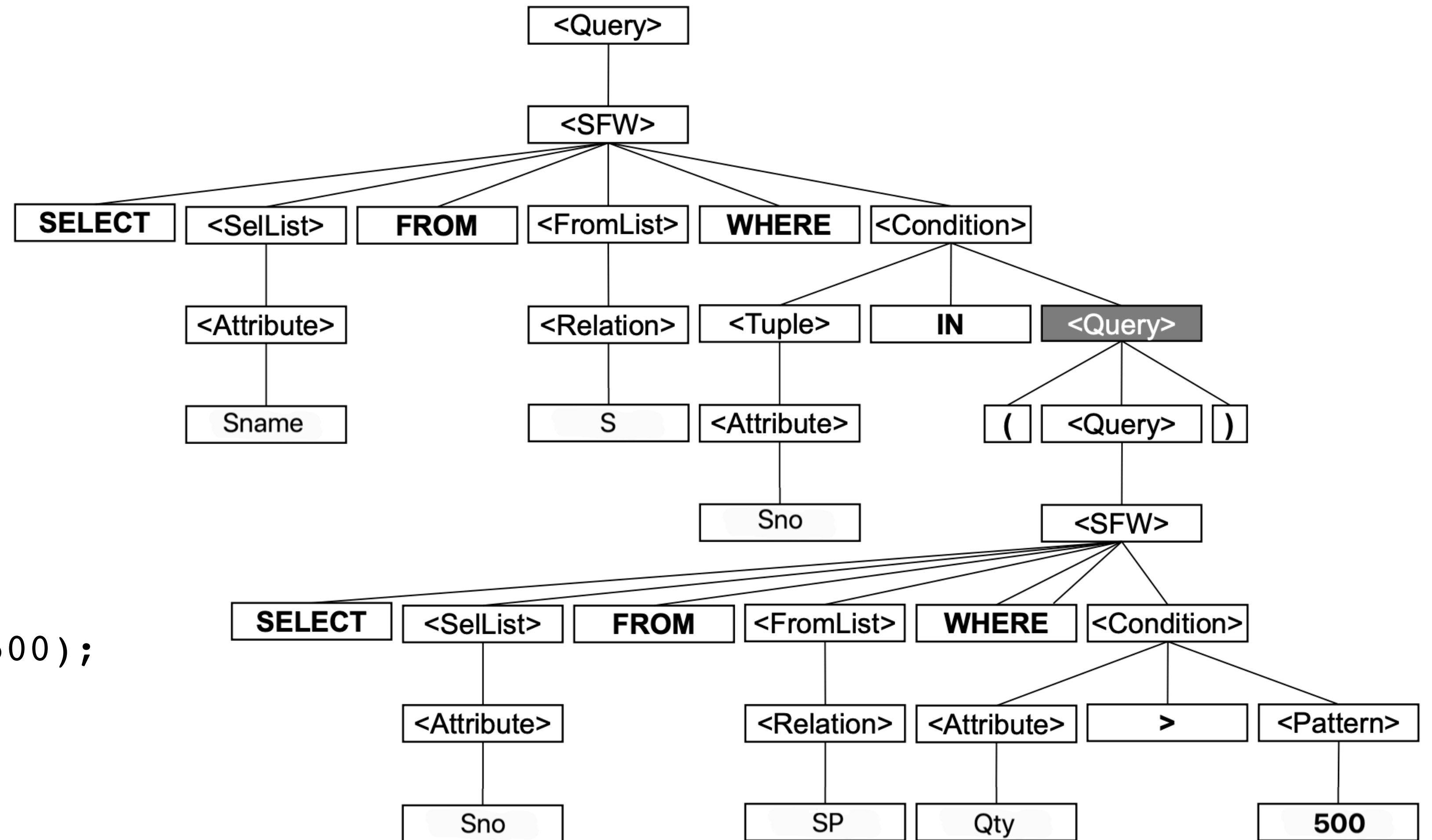
- Атомы - лексические элементы следующих типов:
ключевые слова (например, SELECT); имена атрибутов или отношений; константы; скобки; операторы (например, + или >);
 - Синтаксические категории - имена семейств, представляющих часть запроса. Заключаются в угловые скобки: <SFW>, <Condition>
-
- [1, 2, 3, 4]
 - 1 [2, 3, 4]
 - 2 [3, 4]
 - 3 [4]
 - 4

Граматику языка SQL можно описать с помощью следующих правил:

```
<Query> ::= <SFW>
<Query> ::= (<Query>)
<SFW>   ::= SELECT <SelList> FROM <FromList> WHERE <Condition>
<SelList> ::= <Attribute>, <SelList>
<SelList> ::= <Attribute>
<FromList> ::= <Relation>, <FromList>
<FromList> ::= <Relation>
<Condition> ::= <Condition> AND <Condition>
<Condition> ::= <Condition> OR <Condition>
<Condition> ::= NOT <Condition>
<Condition> ::= <Tuple> IN <Query>
<Condition> ::= <Attribute> = <Attribute>
<Condition> ::= <Attribute> > <Attribute>
<Condition> ::= <Attribute> < <Attribute>
<Condition> ::= <Attribute> LIKE <Pattern>
<Condition> ::= EXISTS <Query>
<Tuple> ::= <Attribute>
```

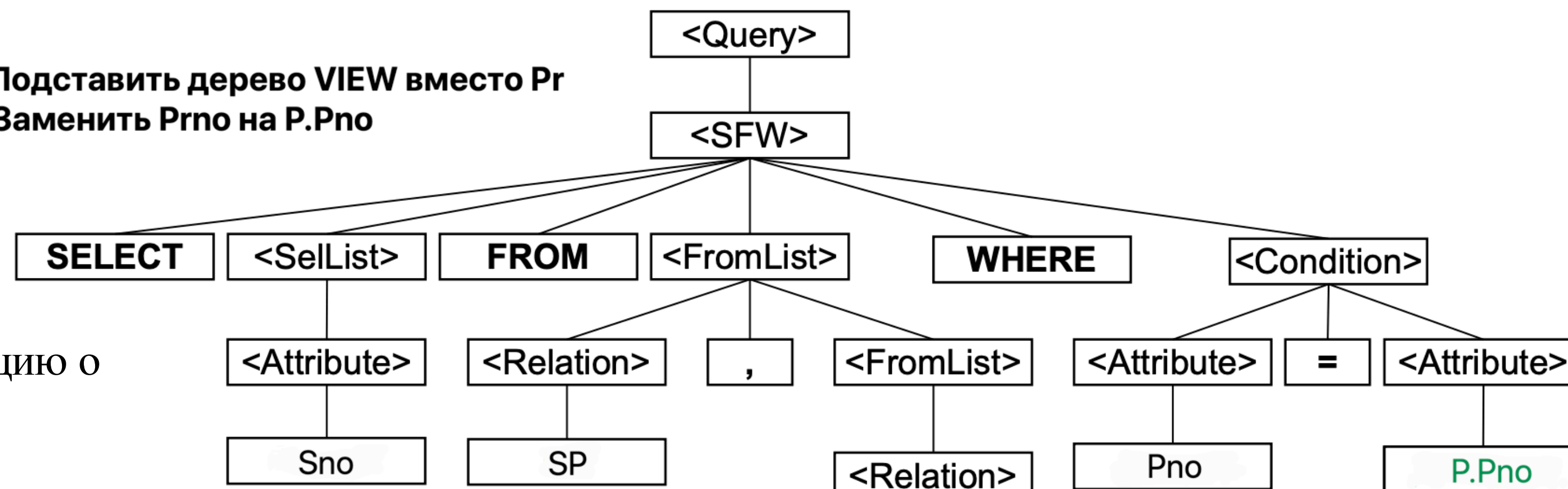

Дерево разбора

```
SELECT Sname FROM S
WHERE Sno IN ( SELECT Sno
                FROM SP
                WHERE Qty > 500 );
```



Система правил

1. Подставить дерево VIEW вместо Pr
2. Заменить Prno на P.Pno

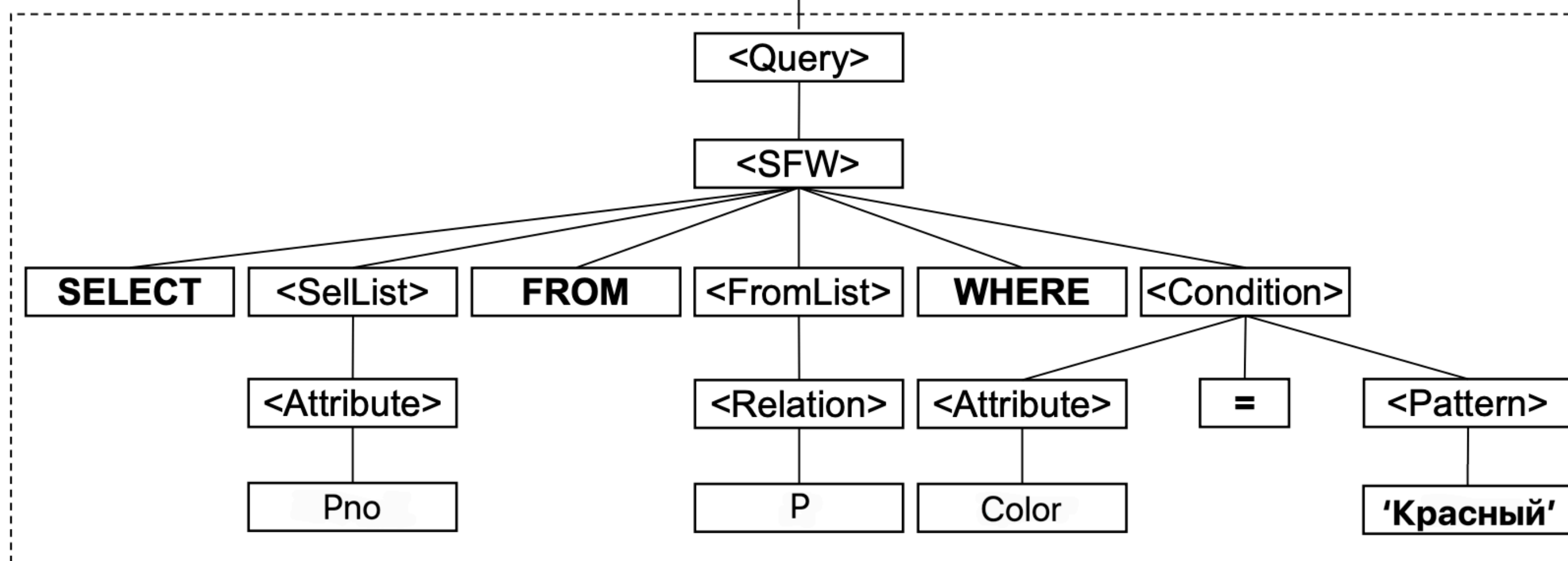


Создадим представление, хранящую информацию о красных деталях:

```
CREATE VIEW Pr (Prno) AS
  SELECT Pno FROM P
  WHERE Color = 'Красный';
```

Теперь найдем все коды поставщиков, поставляющих хотя бы одну красную деталь:

```
SELECT Sno FROM SP, Pr
WHERE Pno = Prno;
```



MPP системы и колоночное хранение

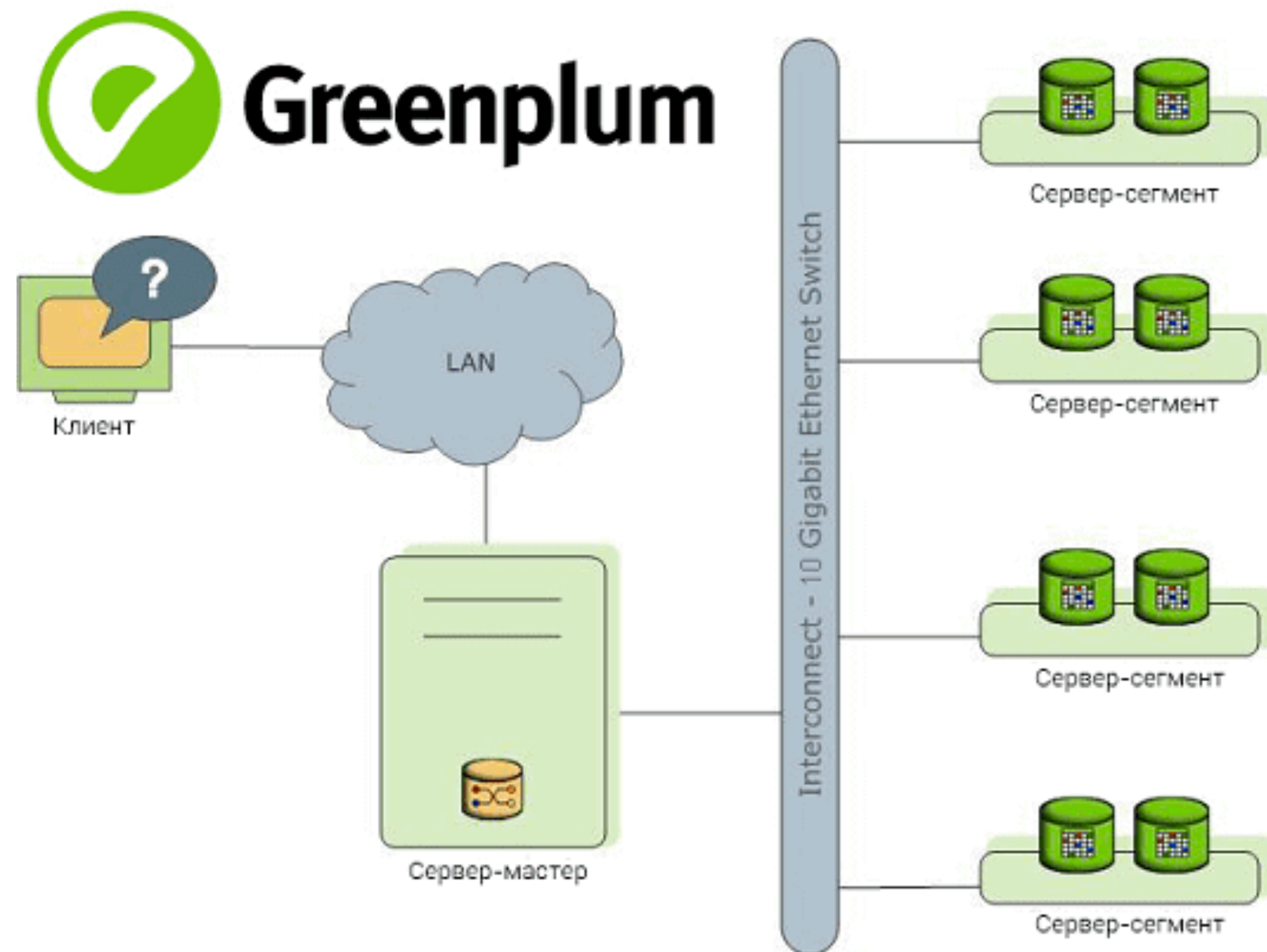
Массивно-параллельные системы обработки

При использовании массивно-параллельной архитектуры данные разделяются на фрагменты, обрабатываемые независимыми центральными процессорами (CPU) и хранящиеся на разных носителях.

Это похоже на загрузку разных фрагментов данных на несколько объединенных в сеть персональных компьютеров.



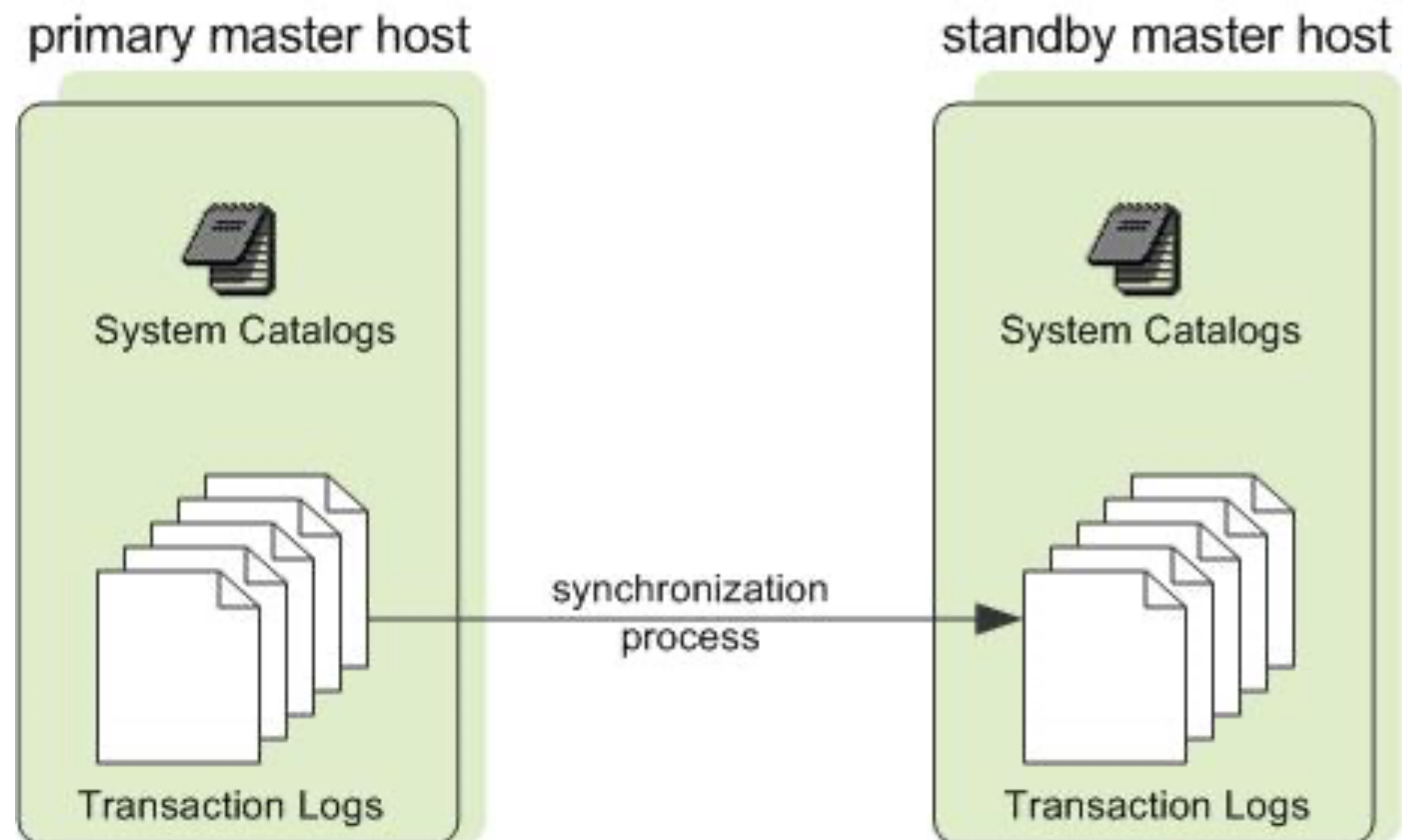
Аппаратная архитектура на примере Greenplum



Компоненты кластера Greenplum:

- Мастер-сервер (Master host), где развернут главный инстанс PostgreSQL (Master instance). Он является точкой входа в Greenplum и представляет собой экземпляр базы данных, к которому подключаются клиенты, отправляя SQL-запросы. Мастер координирует свою работу с сегментами – другими экземплярами базы данных в этой Big Data системе
- Резервный мастер (Secondary master instance) — инстанс PostgreSQL, который вручную включается в работу при отказе основного мастера
- Сервер-сегмент (Segment host), который хранит и обрабатывает данные. Обычно 1 хост-сегмент содержит 2-8 сегментов Greenplum.

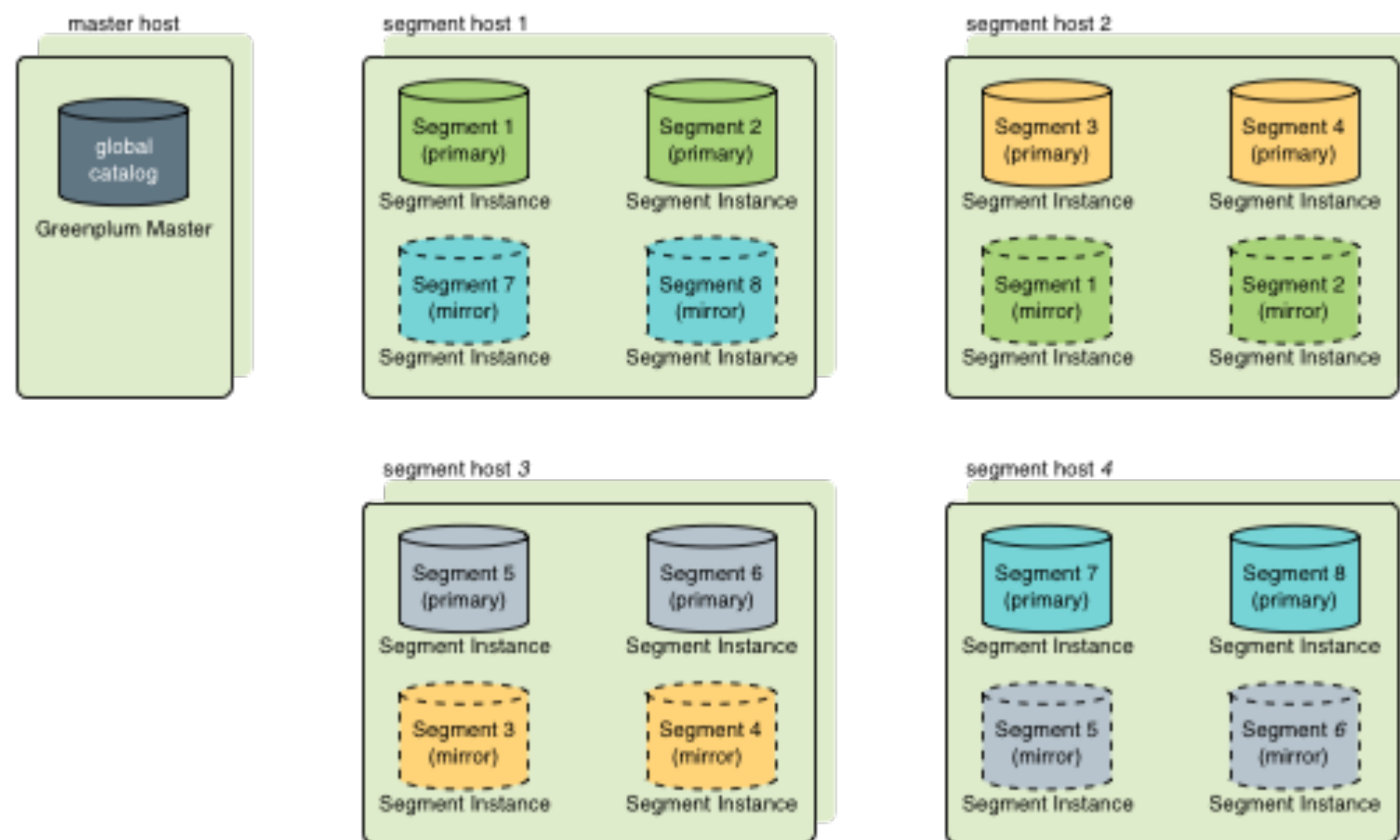
Обеспечение отказоустойчивости мастер-сегмента



- Если основной координатор выходит из строя, система Greenplum Database завершает работу, а процесс репликации координатора останавливается.
- Администратор вручную переназначает резервный координатор основным координатором.
- После активации резервного координатора реплицированные журналы восстанавливают состояние основного координатора на момент последней успешно зафиксированной транзакции.
- Активированный резервный координатор теперь принимает соединения через порт, указанный при инициализации резервного координатора.

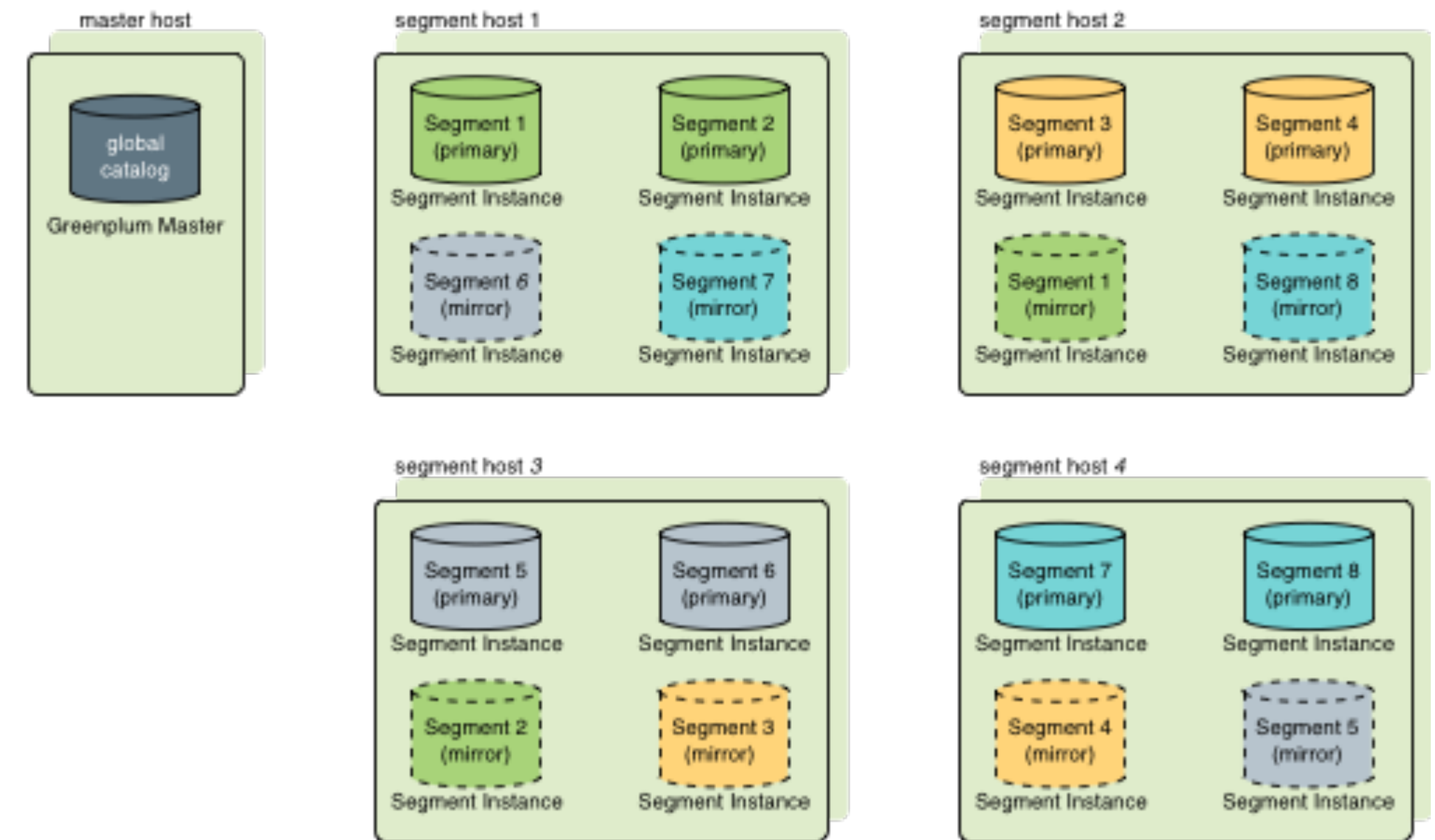
Обеспечение отказоустойчивость

Групповое зеркалирование



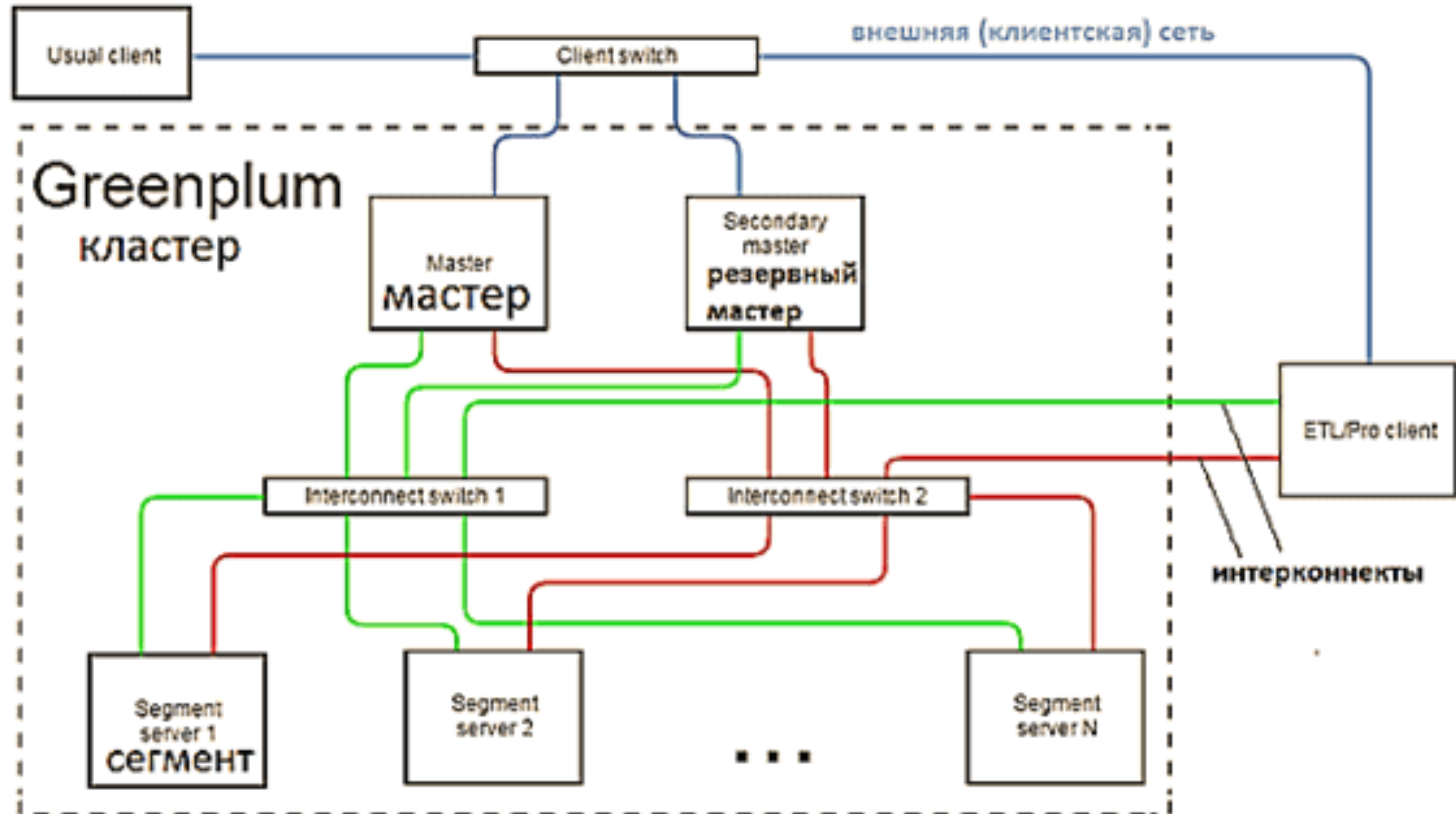
Конфигурация зеркалирования по умолчанию.
Зеркальные сегменты для основных сегментов каждого хоста размещаются на другом одном хосте. Если один из хостов выходит из строя, количество активных основных сегментов на хосте, который поддерживает вышедший из строя хост, удваивается.

Распределенное зеркалирование



При такой конфигурации зеркала каждого хоста распределяются между несколькими хостами, так что в случае сбоя одного хоста ни на одном другом хосте не будет более одного зеркала, преобразованного в активный первичный сегмент.

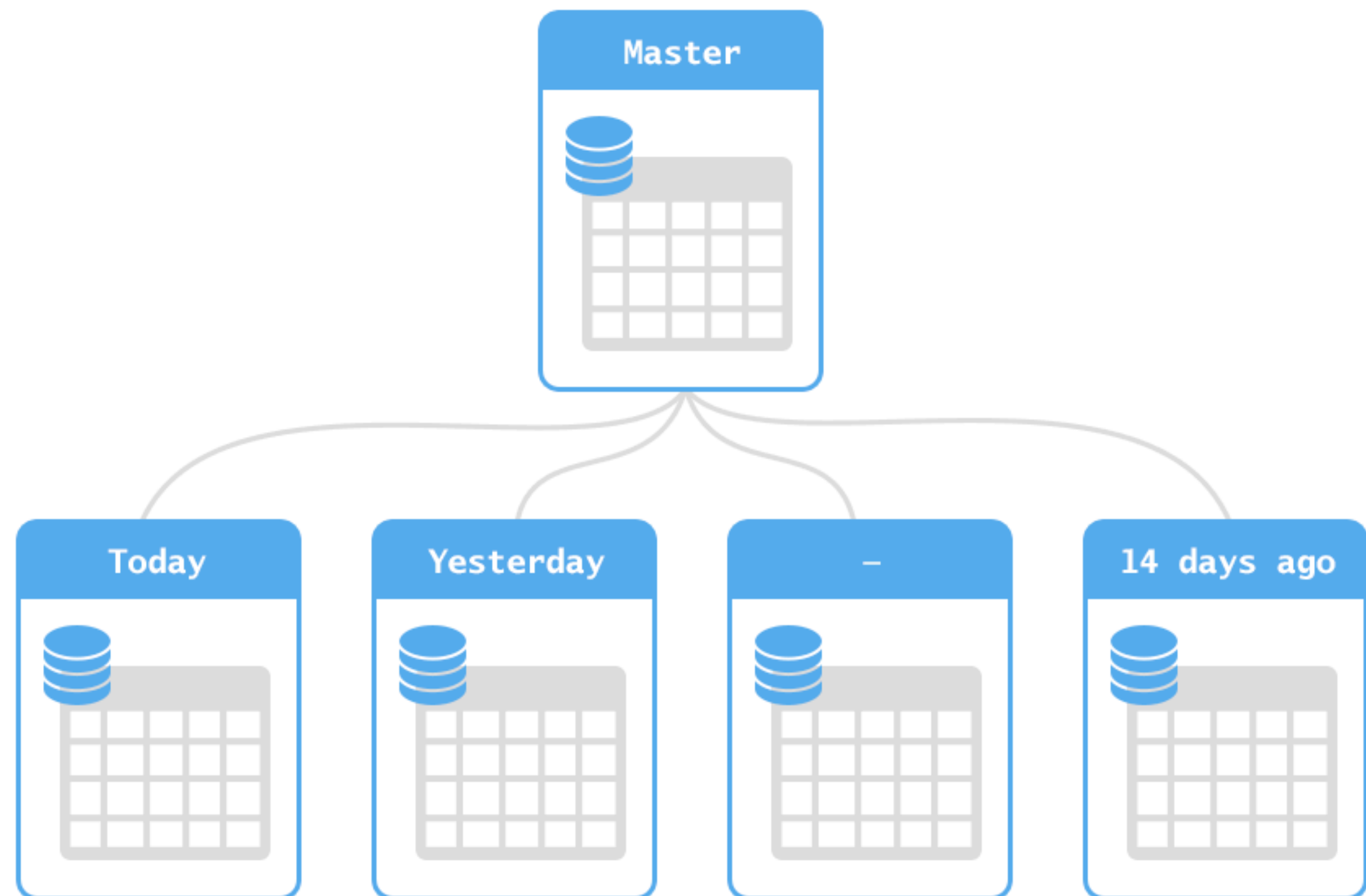
Архитектура GreenPlum



Партиционирование

Партиционирование – это метод разделения больших (исходя из количества записей, а не столбцов) таблиц на много маленьких. И желательно, чтобы это происходило прозрачным для приложения способом.

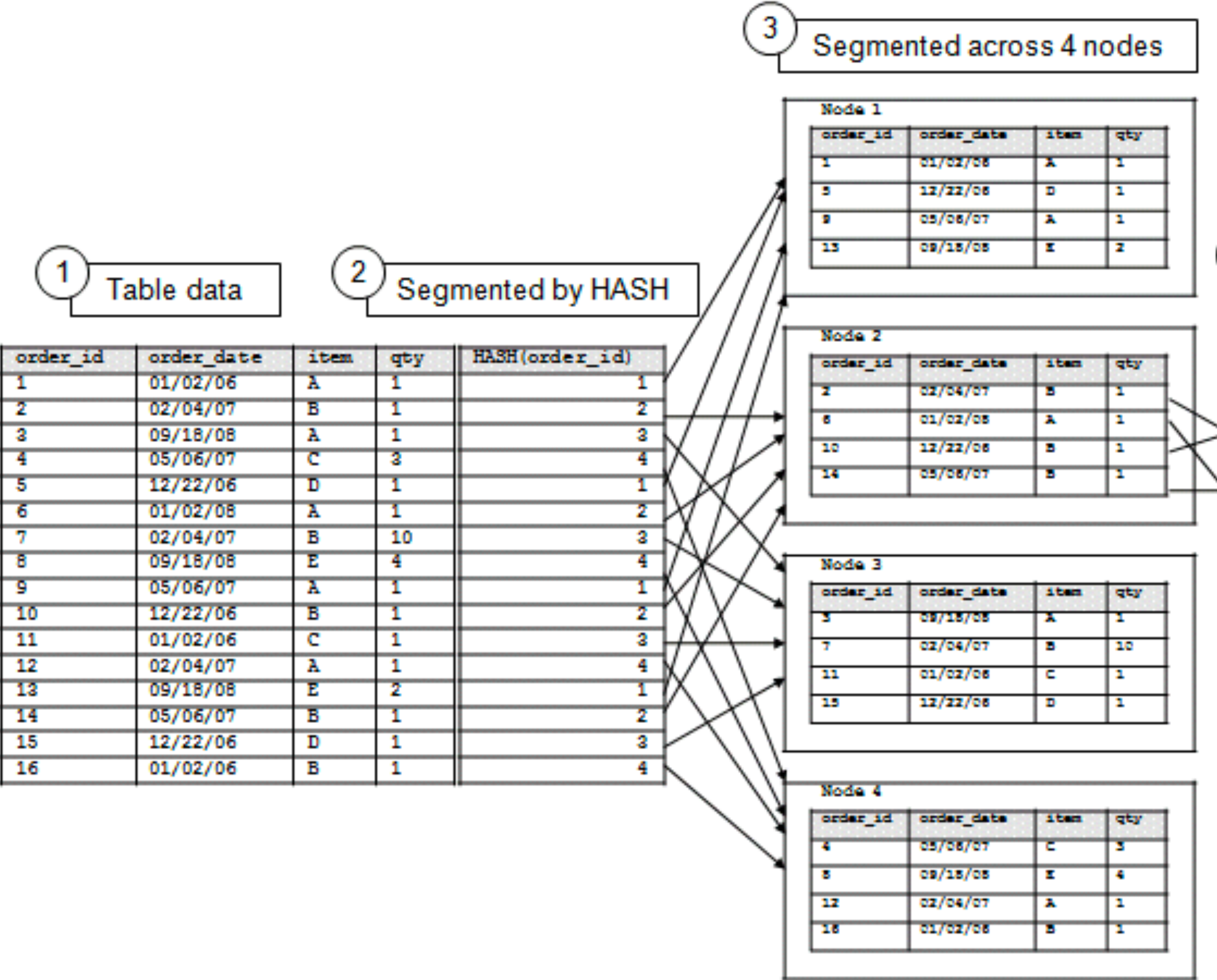
```
CREATE TABLE measurement (  
  city_id      int not null,  
  logdate     date not null,  
  peaktemp    int,  
  unitsales   int  
) PARTITION BY RANGE (logdate);
```



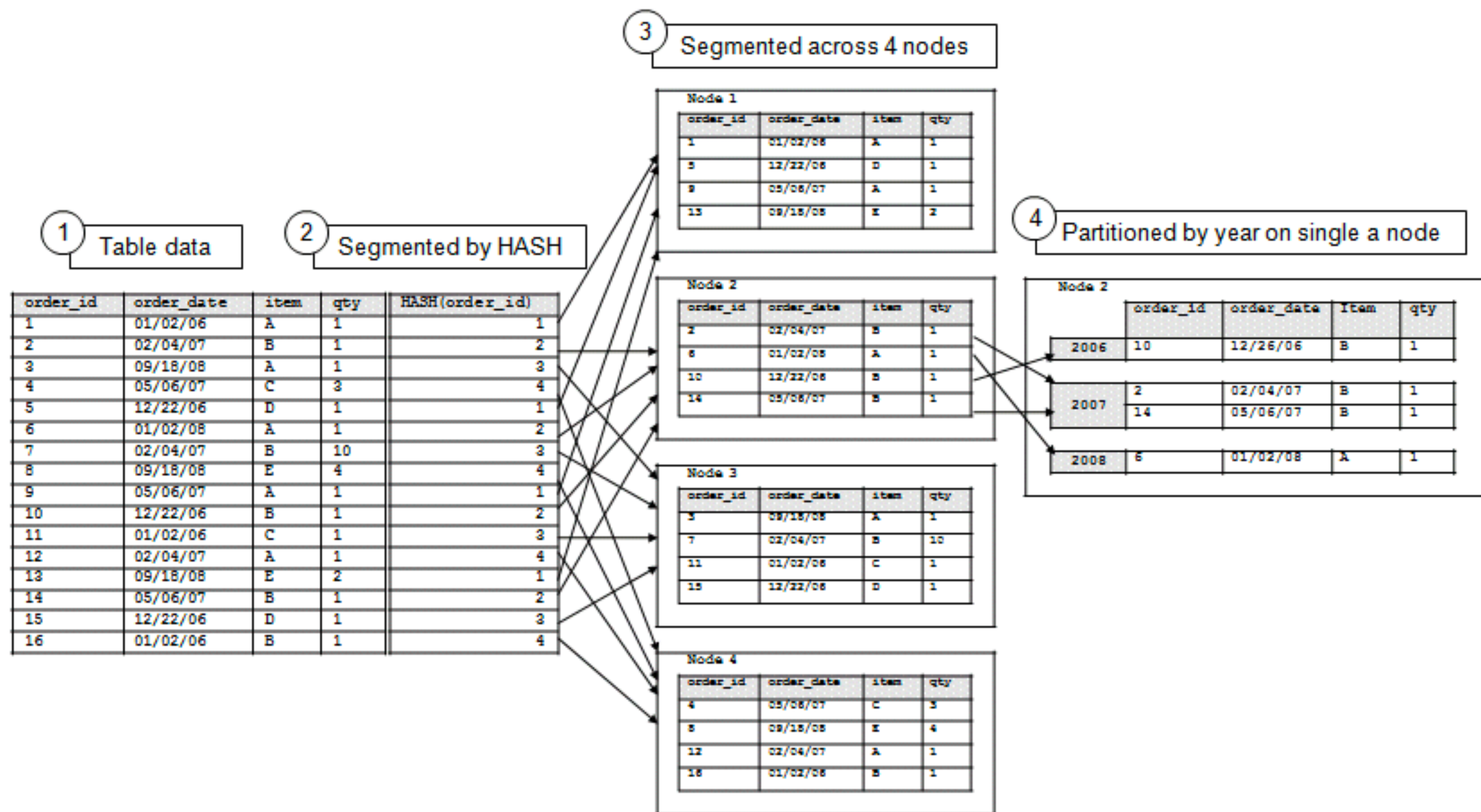
Сегментирование (дистрибьюция)

Для распределения данных по кластерам используется их сегментация (Segmentation).

Задача разработчика — подобрать такой список полей и/или такую функцию(например, хэш-функцию), благодаря которым данные равномерно распределятся по нодам кластера.



Объединение процессов сегментации и партиционирования



Аппаратная архитектура на примере VERTICA

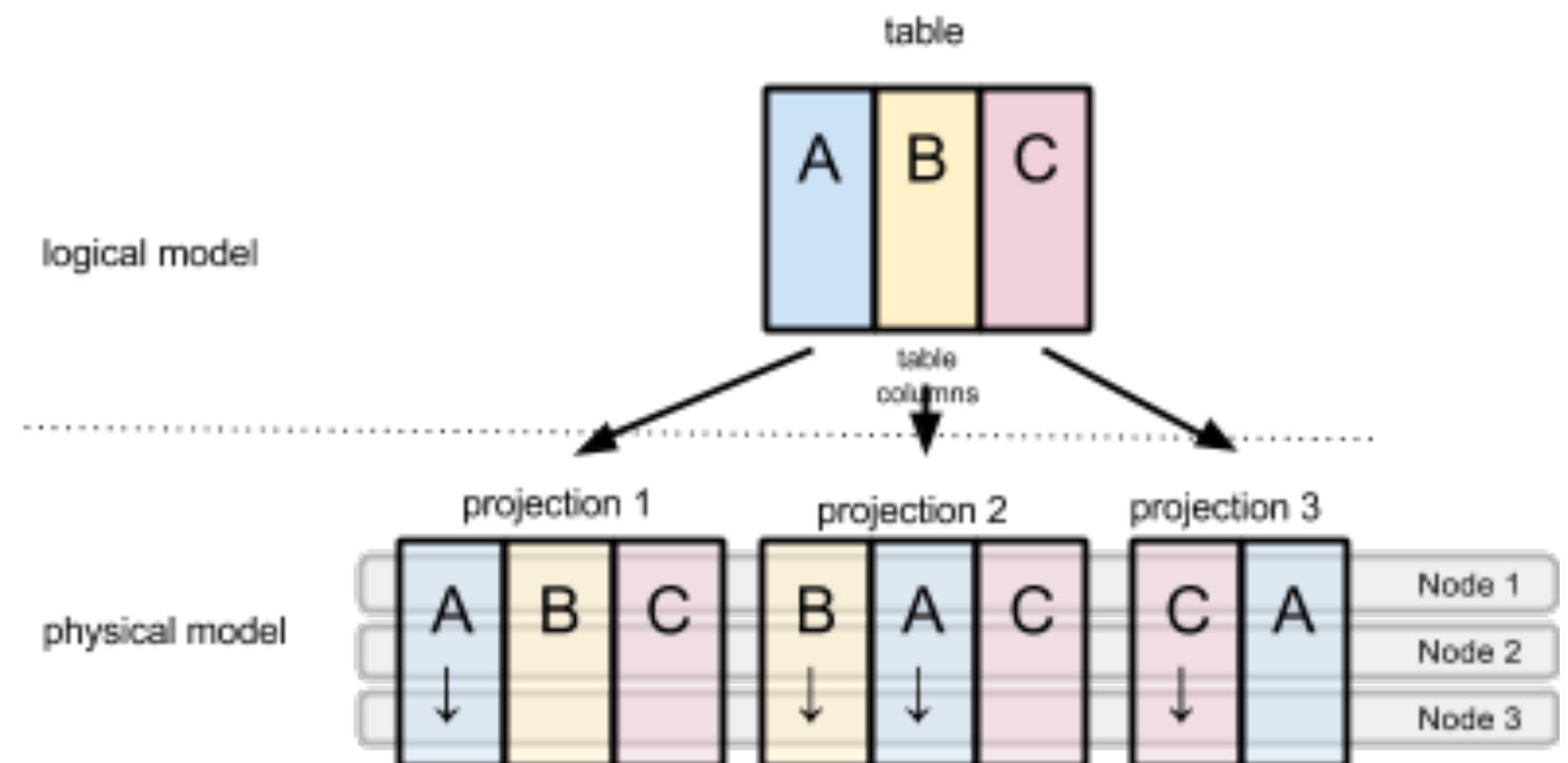
Основные свойства:

- отсутствует единая точка отказа,
- каждый узел независим и самостоятелен,
- отсутствует единая для всей системы точка подключения,
- узлы инфраструктуры дублируются,
- данные на узлах кластера автоматически копируются.

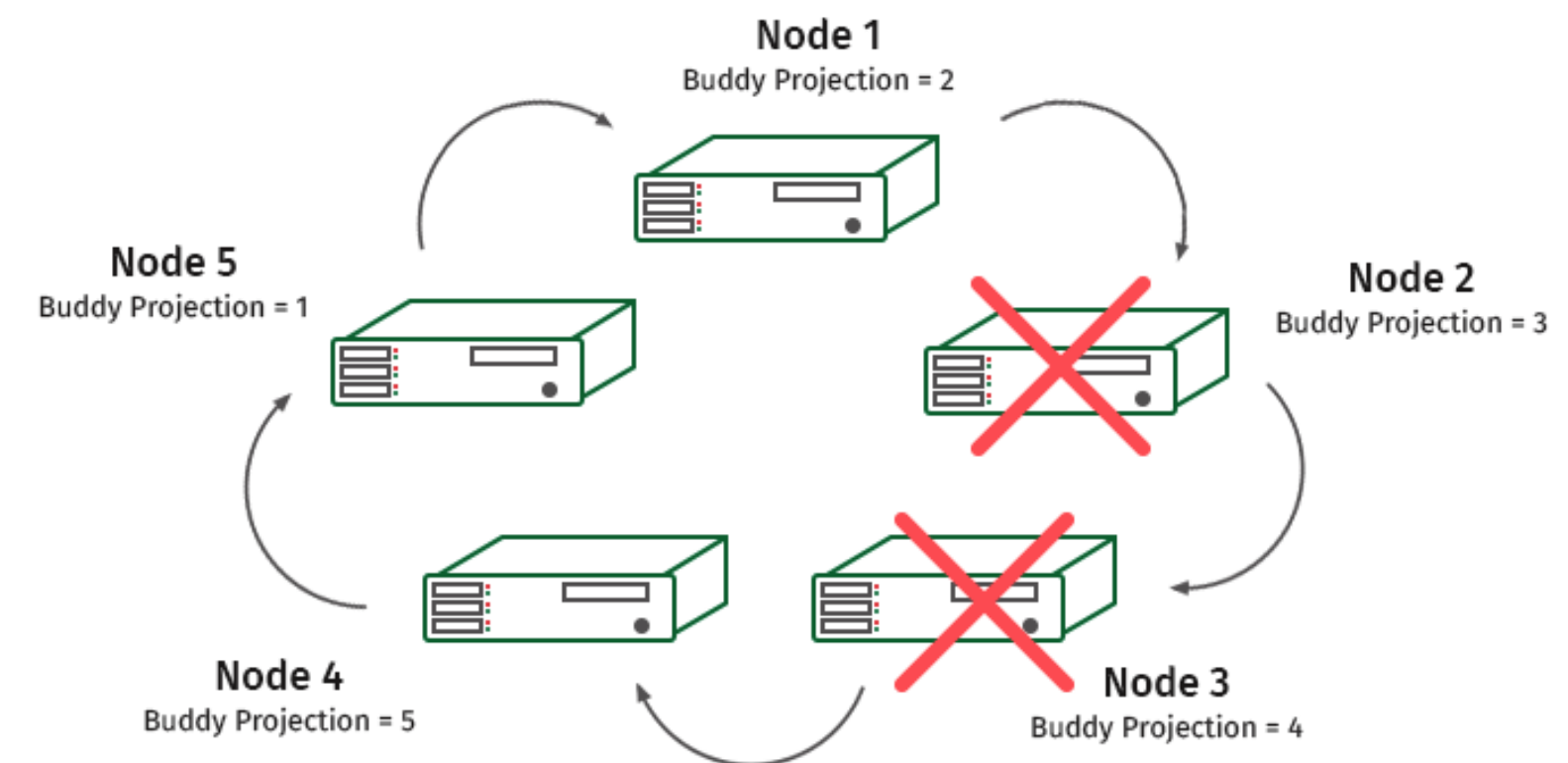
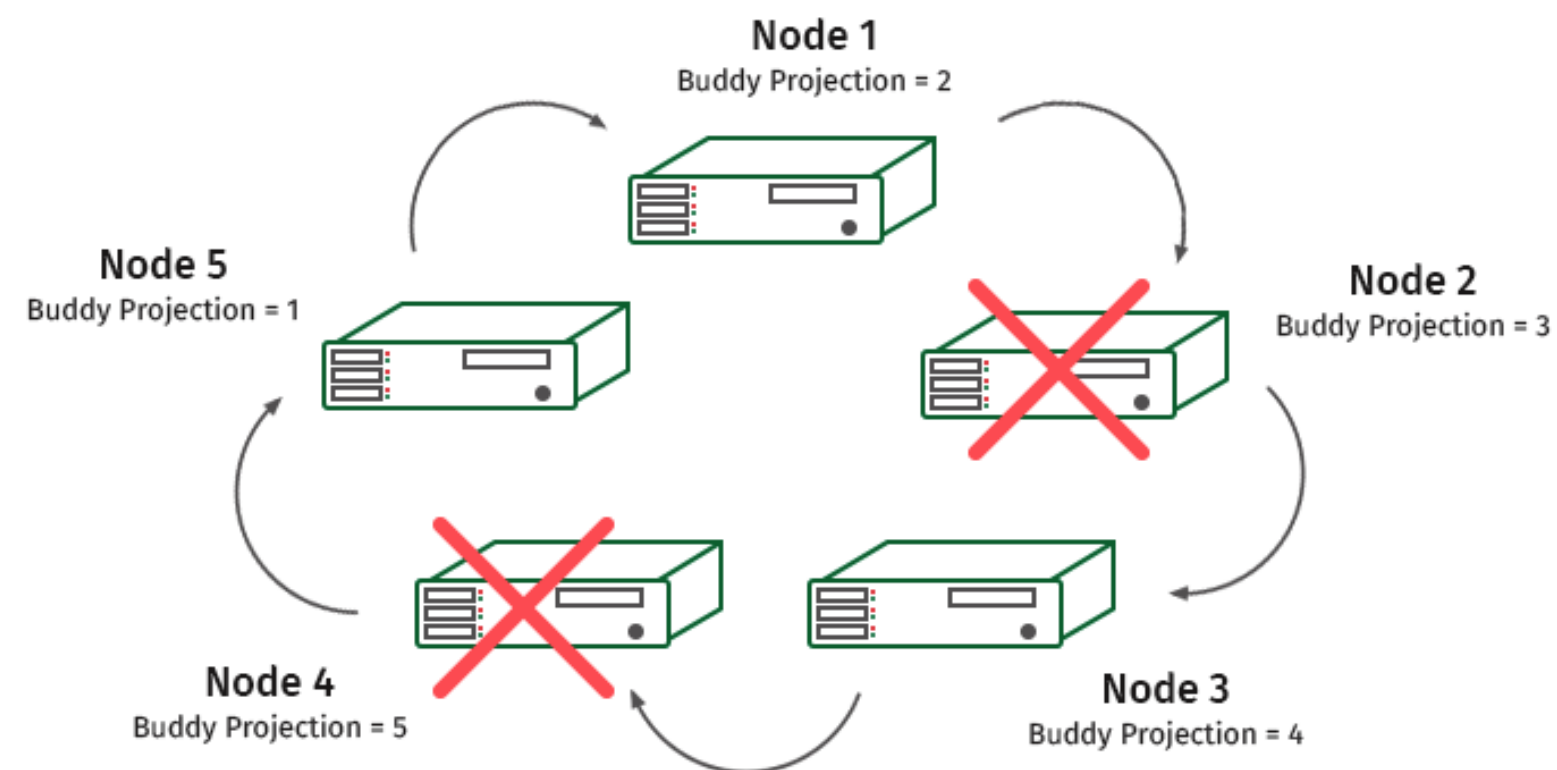
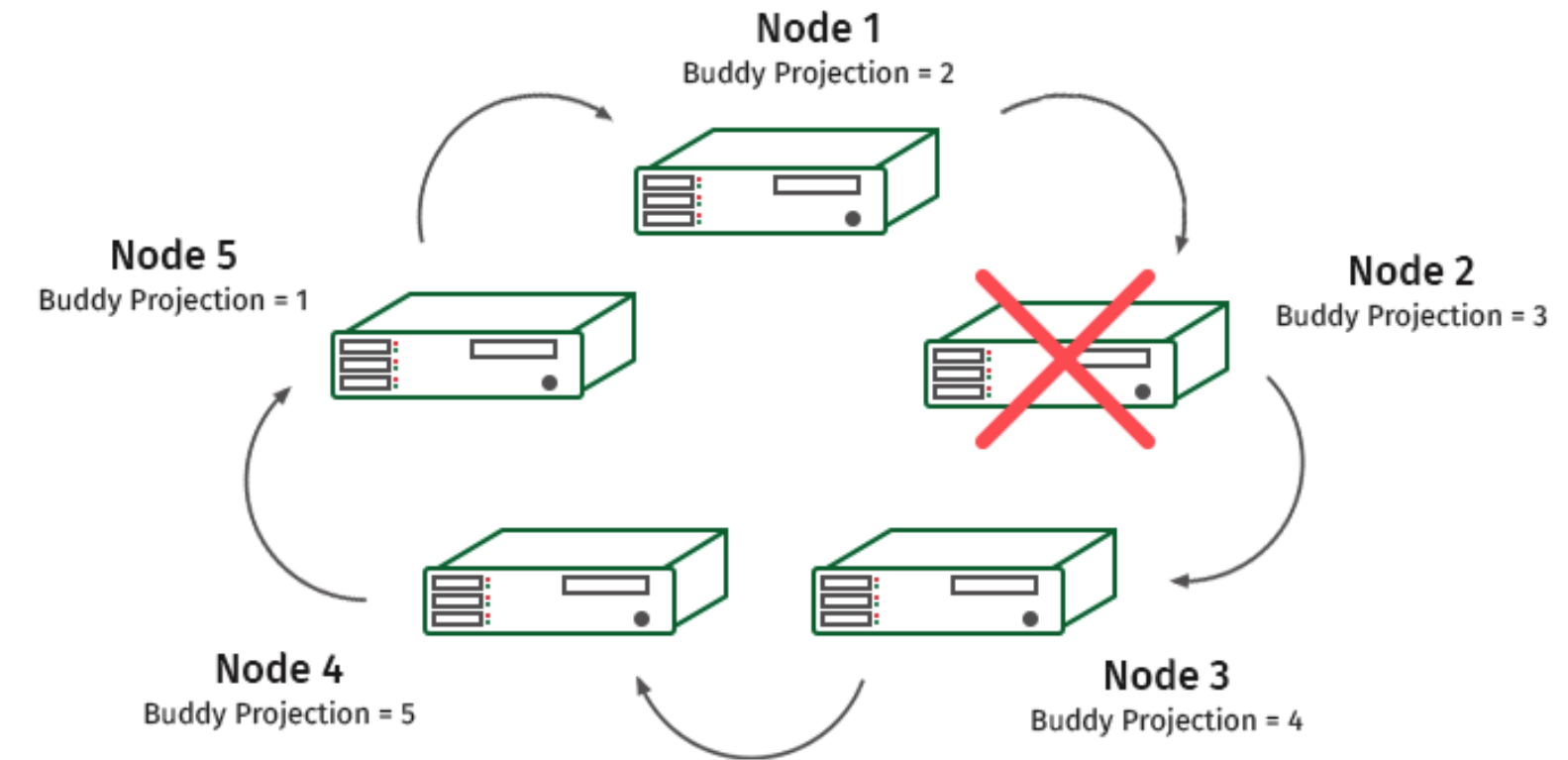
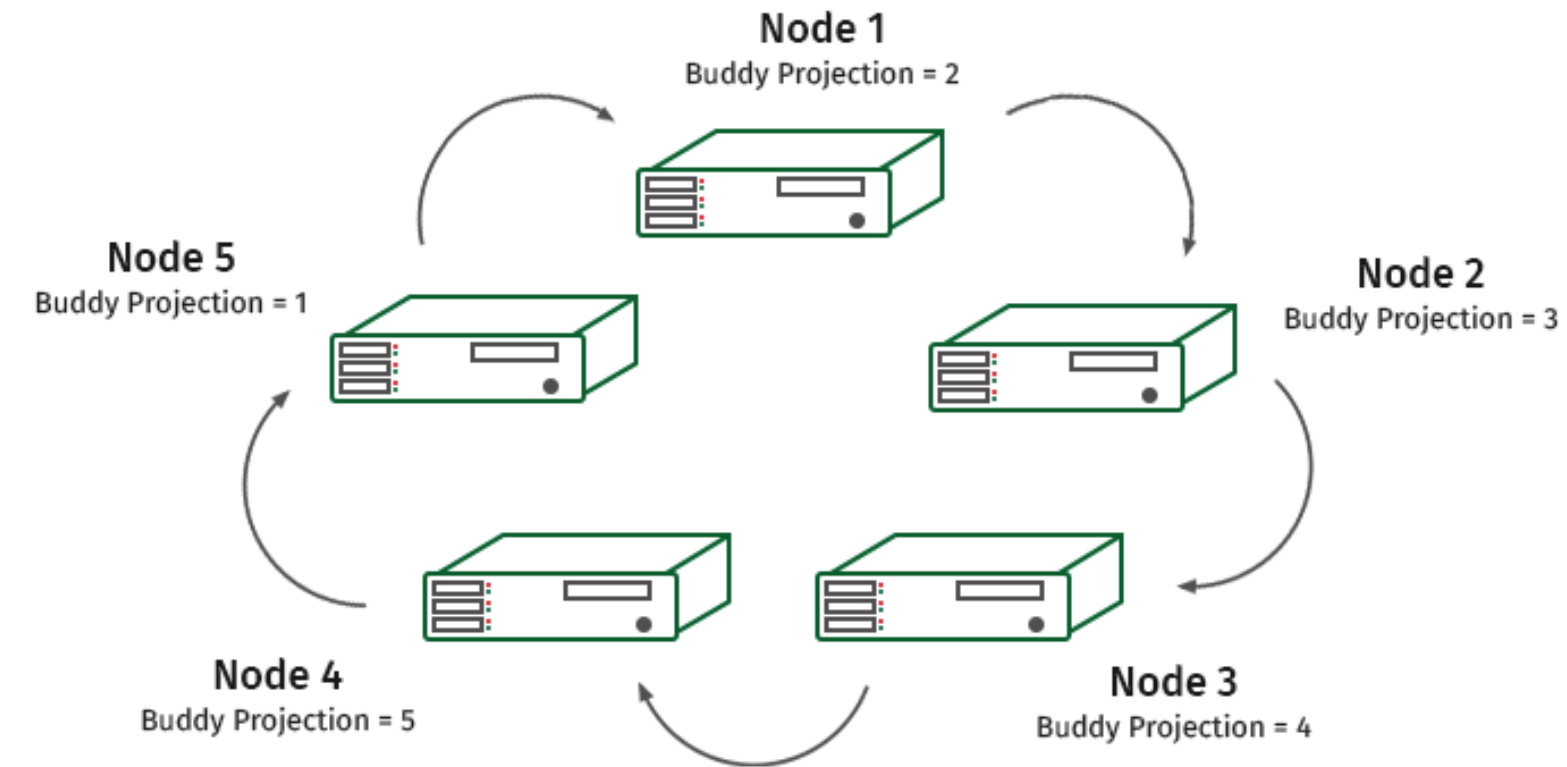
Логические единицы хранения информации — это схемы, таблицы и представления. Физические единицы — это проекции.

Проекции бывают нескольких типов:

- суперпроекции (Superprojection),
- запрос-ориентированные проекции (Query-Specific Projections),
- агрегированные проекции (Aggregate Projections).



Обеспечение отказоустойчивость



Движение данных в MRP системах

Этапы выполнения параллельного запроса

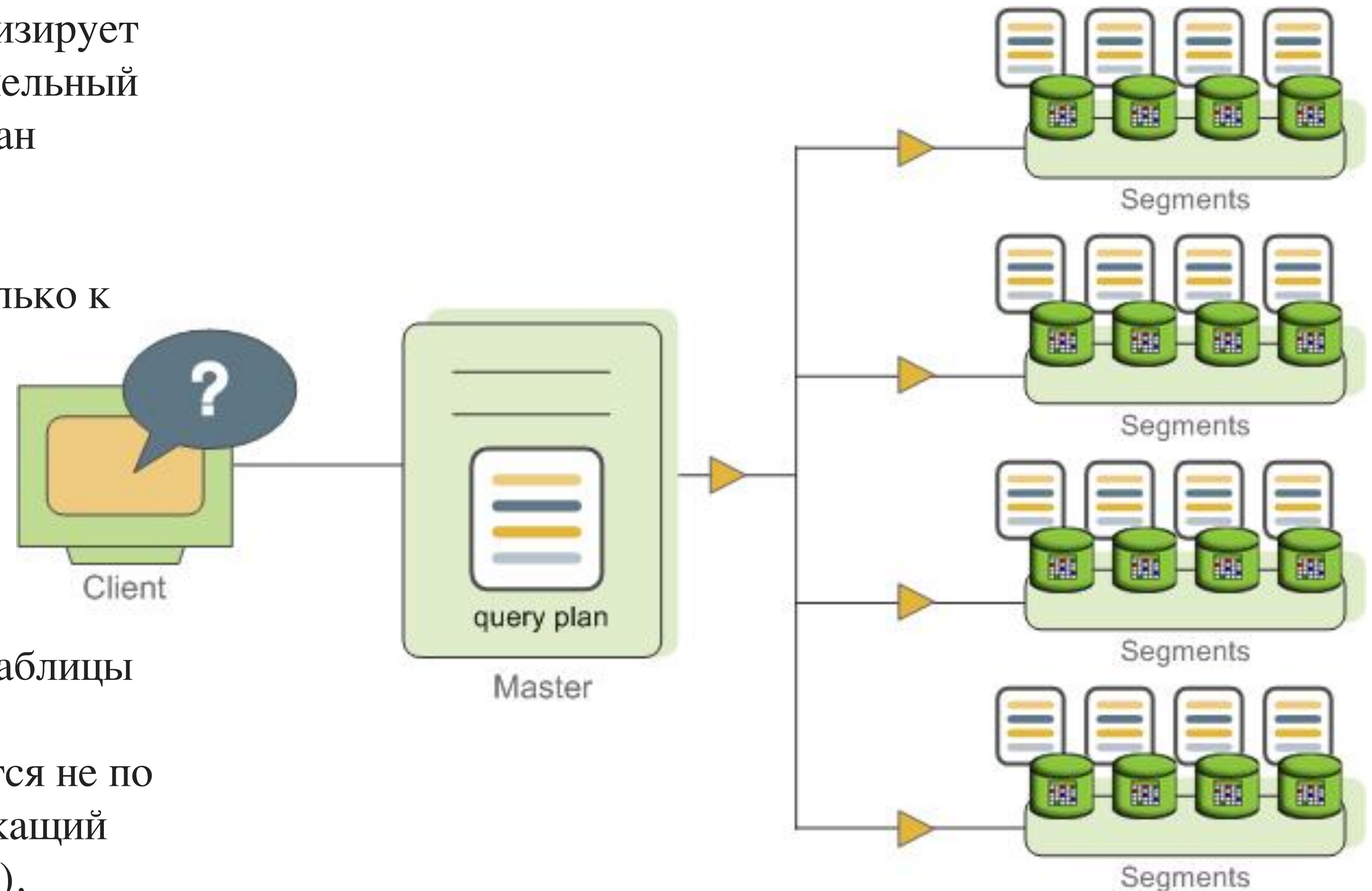
Координатор получает, анализирует и оптимизирует запрос. В результате получается либо параллельный (для нескольких сегментов), либо целевой план запроса (для одного сегмента).

Определенные запросы могут обращаться только к данным в одном сегменте.

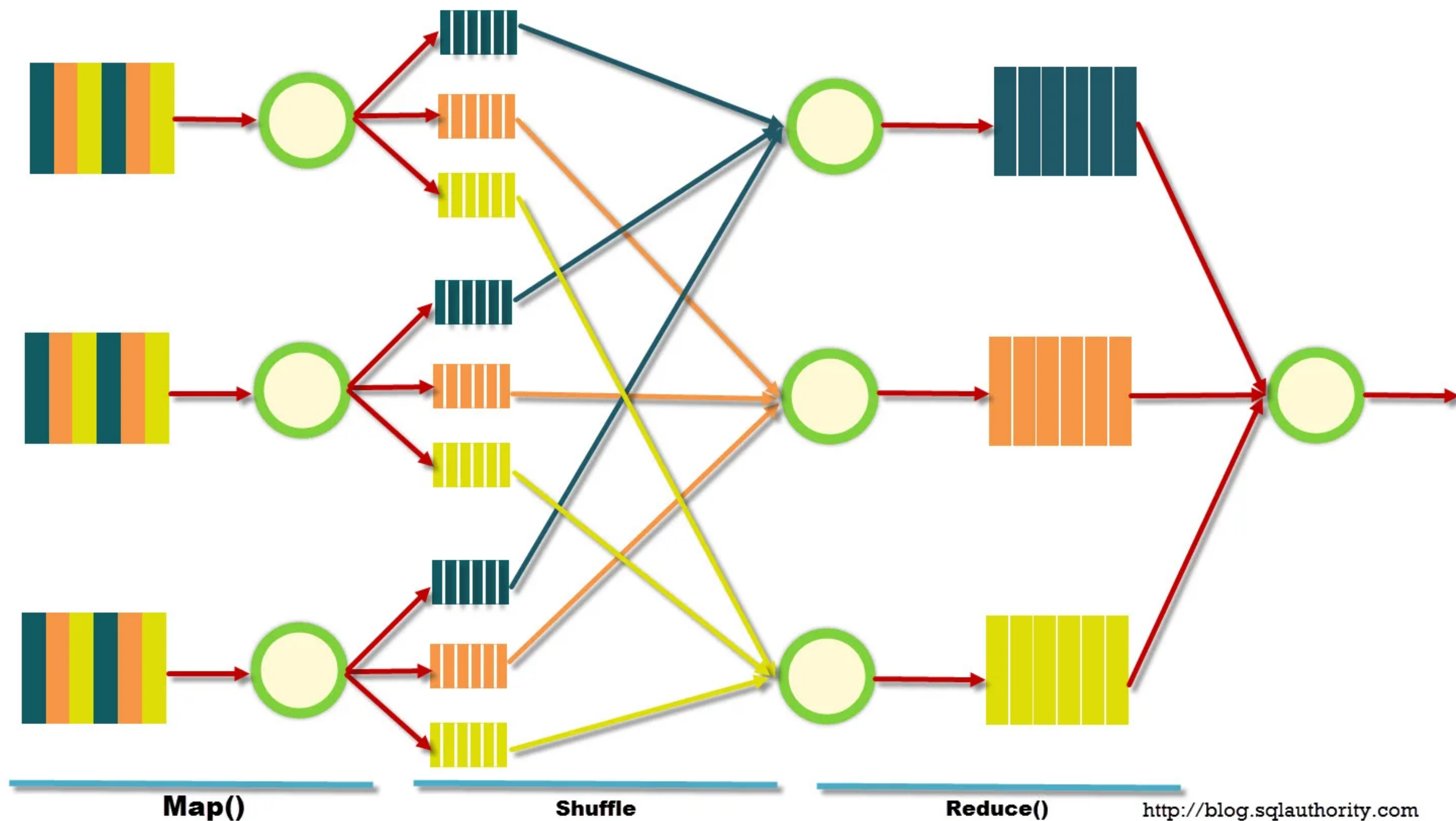
Например:

- INSERT
- UPDATE
- DELETE
- SELECT, который фильтрует по столбцу(столбцам) ключа распределения таблицы

В подобных запросах план запроса рассылается не по всем сегментам, а нацелен на сегмент, содержащий затронутую или релевантную строку (строки).



MapReduce в МРР системах



Какие бывают движения данных

Основная задача распределенных систем - равномерно нагрузить каждый сегмент в MPP, что означает добиться локальности выполнения операций. Так как обрабатываемые данные могут находиться на разных сегментах, то необходимо эффективно перемещать данные из сегмента в сегмент.

Какие операции могут вызывать движение данных:

- Операции агрегации
- Операции соединения (джоины)

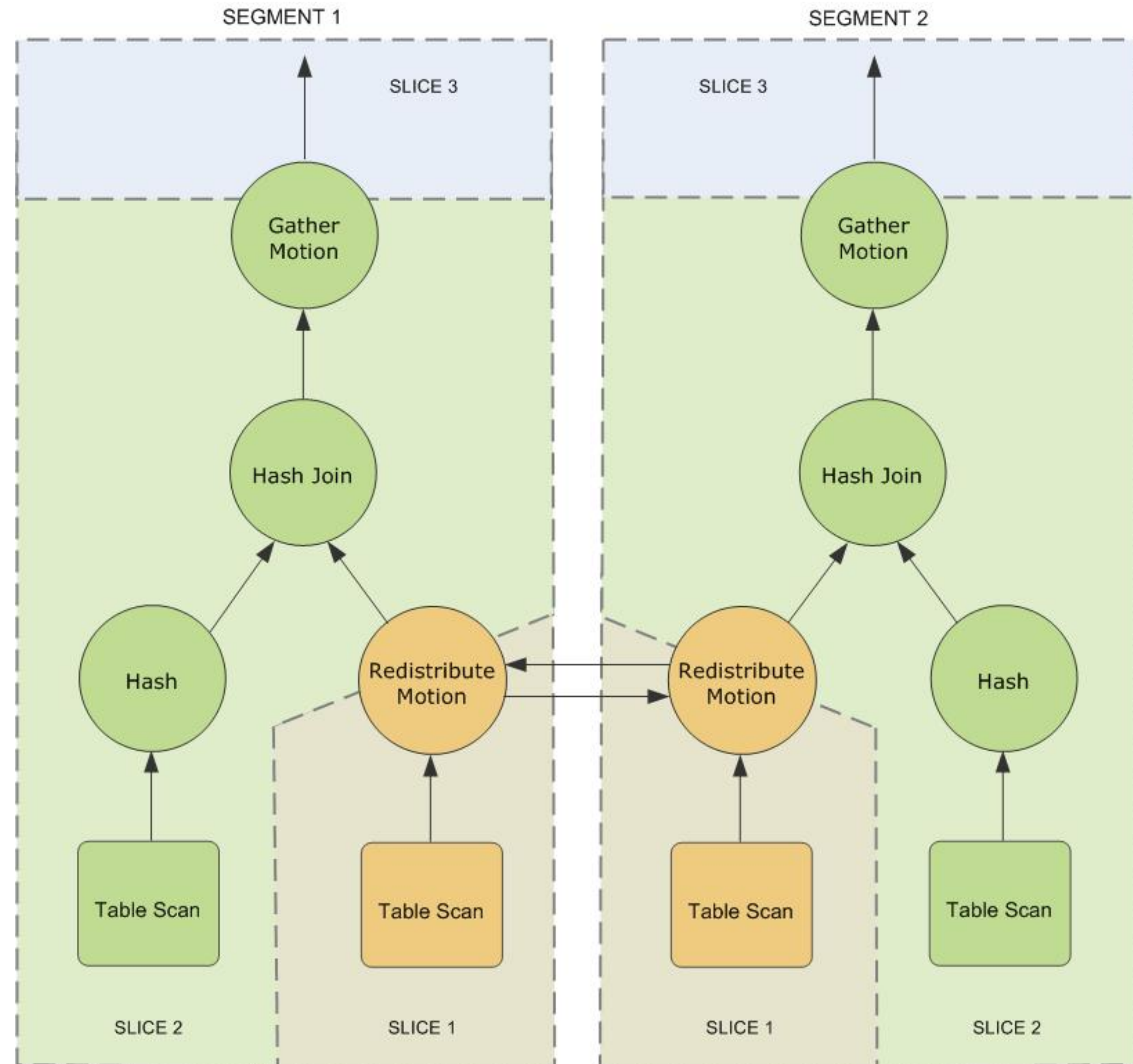
Какие движения данных существуют:

- Redistribute Motion
- Broadcast Motion
- Gather Motion

Redistribute Motion

Если ключ соединения таблиц не соответствует ключу распределения, то возможна ситуация при которых соединяемые данные физически расположены на разных серверах.

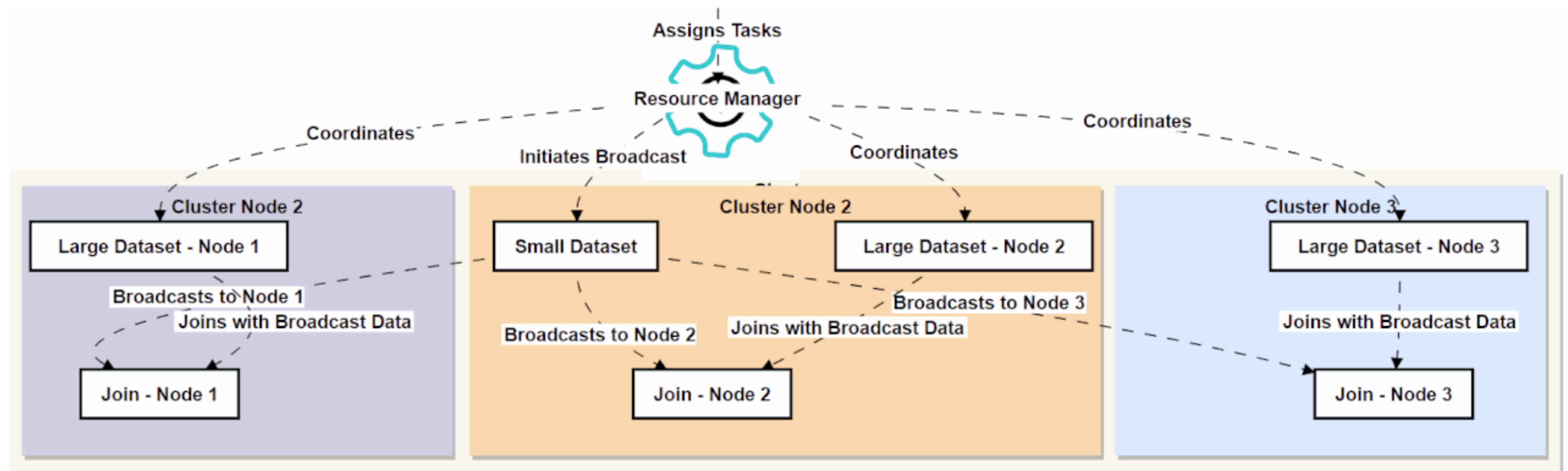
В этом случае для выполнения запроса системе необходимо автоматически перераспределить данные по другому ключу дистрибьюции.



Broadcast Motion

Broadcast Motion выбирает наименьшую из таблиц и дублирует данные на все сегменты.

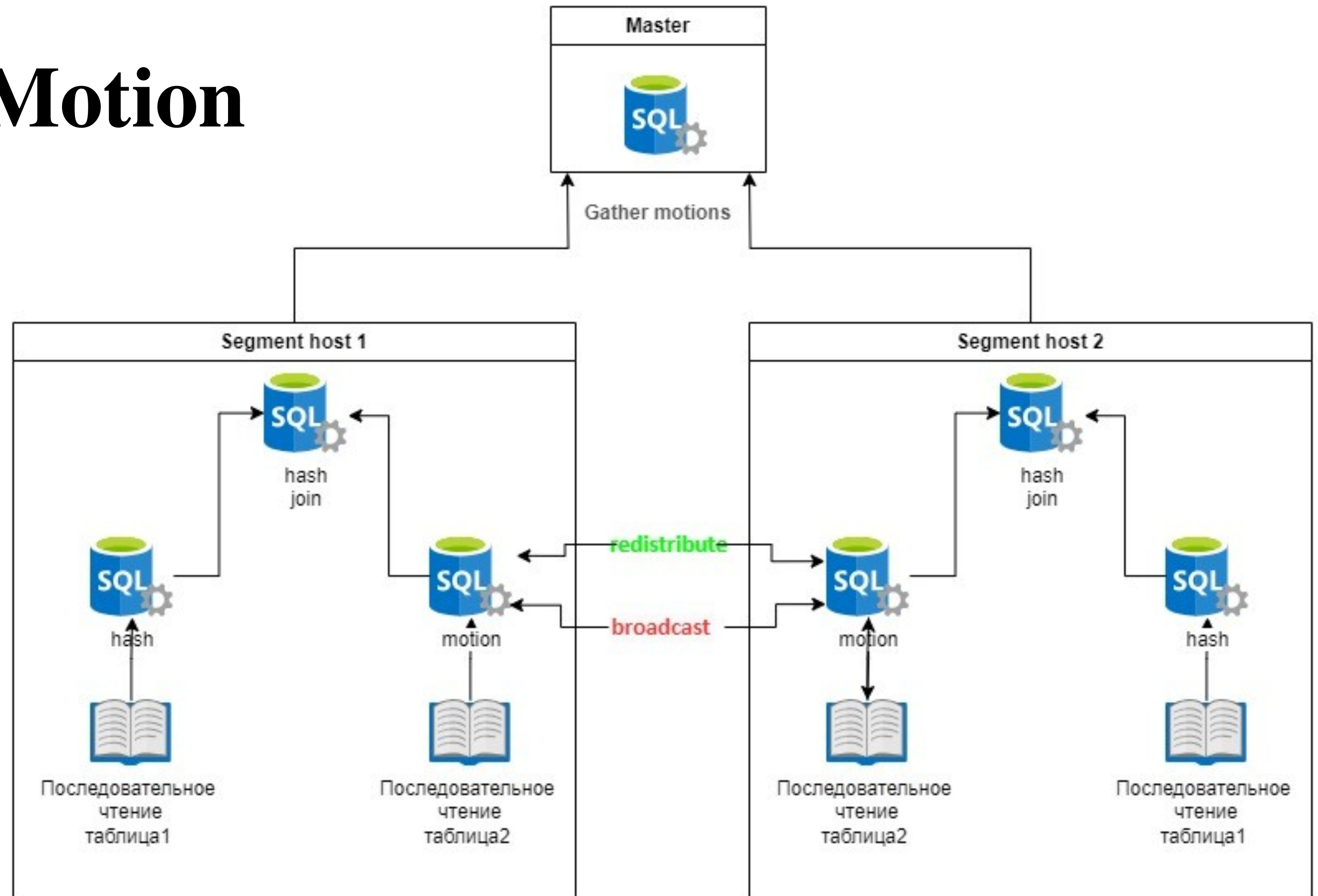
Этот способ может быть не таким оптимальным, как предыдущий, поэтому оптимизатор обычно выбирает его для небольших таблиц.



Gather Motion

Gather Motion -
объединяет на мастере
результаты с сегментов.

Данный шаг
присутствует в каждом
запросе и является
последним шагом плана.



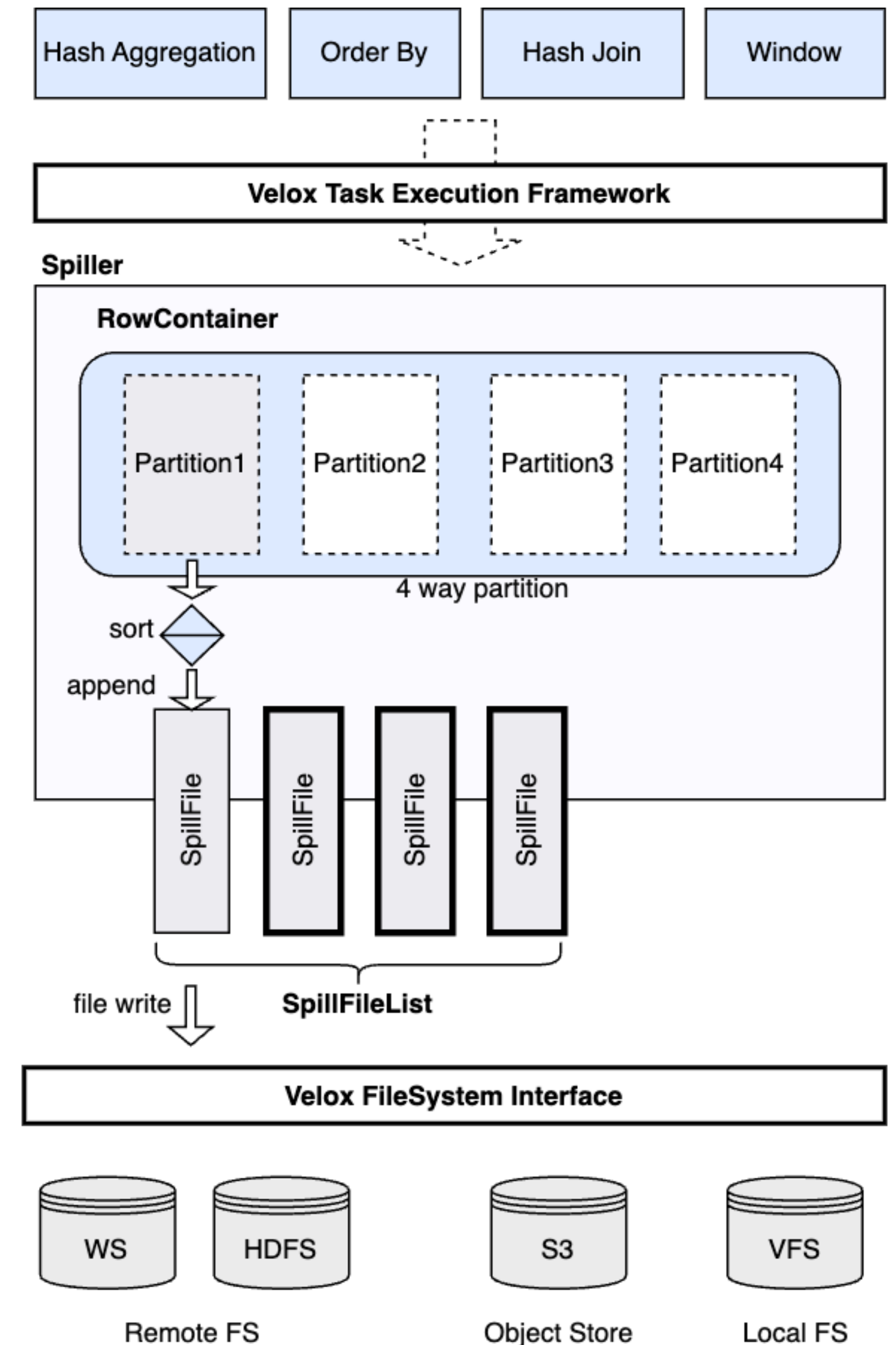
Spill-файлы

Спиллы — это некоторый генерируемый дополнительный объем данных на жестком диске, который используется для выполнения запроса.

Спиллы появляются, когда для запроса нужно хранить данных больше, чем предоставлено оперативной памятью.

Причины генерации Spill-файлов:

- обработка огромного объема данных без фильтров;
- создание хэш-таблицы на основе таблицы большого объема;
- DISTINCT на большом количестве полей;
- использование ORDER BY;
- Использование оконных функций.



**Что делать, чтобы избежать плохих
движений данных и спиллов?**

- 1) **EXPLAIN ANALYSE**
- 2) **Умение читать план запроса**

Пример с распределением не по ключу соединения

Таблица клиентов

```
CREATE TABLE customers
(
  customer_id  INTEGER,
  customer_name VARCHAR(25)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (customer_id);
```

Таблица заказов

```
CREATE TABLE orders
(
  order_id  INTEGER,
  customer_id INTEGER,
  amount    DECIMAL(6, 2)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (order_id);
```


Пример с распределением не по ключу соединения

EXPLAIN

```
SELECT c.customer_id,  
       c.customer_name AS customer_name,  
       SUM(o.amount) AS total_purchases  
FROM customers c  
      JOIN orders o USING (customer_id)  
GROUP BY c.customer_id,  
         c.customer_name  
ORDER BY total_purchases DESC  
LIMIT 5;
```

QUERY PLAN

Limit

-> Gather Motion 4:1 (slice3; segments: 4)

Merge Key: (pg_catalog.sum((sum(o.amount))))

-> Limit

-> Sort

Sort Key: (pg_catalog.sum((sum(o.amount))))

-> HashAggregate

Group Key: c.customer_id, c.customer_name

-> Redistribute Motion 4:4 (slice2; segments: 4)

Hash Key: c.customer_id, c.customer_name

-> Result

-> HashAggregate

Group Key: c.customer_id, c.customer_name

-> Hash Join

Hash Cond: (o.customer_id = c.customer_id)

-> Seq Scan on orders o

-> Hash

-> Broadcast Motion 4:4 (slice1; segments: 4)

-> Seq Scan on customers c

Optimizer: Pivotal Optimizer (GPORCA)

(20 rows)

Пример с распределением по ключу соединения

Таблица клиентов

```
CREATE TABLE customers
(
  customer_id  INTEGER,
  customer_name VARCHAR(25)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (customer_id);
```

Таблица заказов

```
CREATE TABLE orders
(
  order_id  INTEGER,
  customer_id INTEGER,
  amount    DECIMAL(6, 2)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (customer_id);
```

Пример с распределением по ключу соединения

```
EXPLAIN
SELECT c.customer_id,
       c.customer_name AS customer_name,
       SUM(o.amount) AS total_purchases
FROM customers c
      JOIN orders o USING (customer_id)
GROUP BY c.customer_id,
         c.customer_name
ORDER BY total_purchases DESC
LIMIT 5;
```

QUERY PLAN

Limit

-> Gather Motion 4:1 (slice1; segments: 4)

Merge Key: (sum(o.amount))

-> Limit

-> Sort

Sort Key: (sum(o.amount))

-> HashAggregate

Group Key: c.customer_id, c.customer_name

-> Hash Join

Hash Cond: (o.customer_id = c.customer_id)

-> Seq Scan on orders o

-> Hash

-> Seq Scan on customers c

Optimizer: Pivotal Optimizer (GPORCA)

(14 rows)

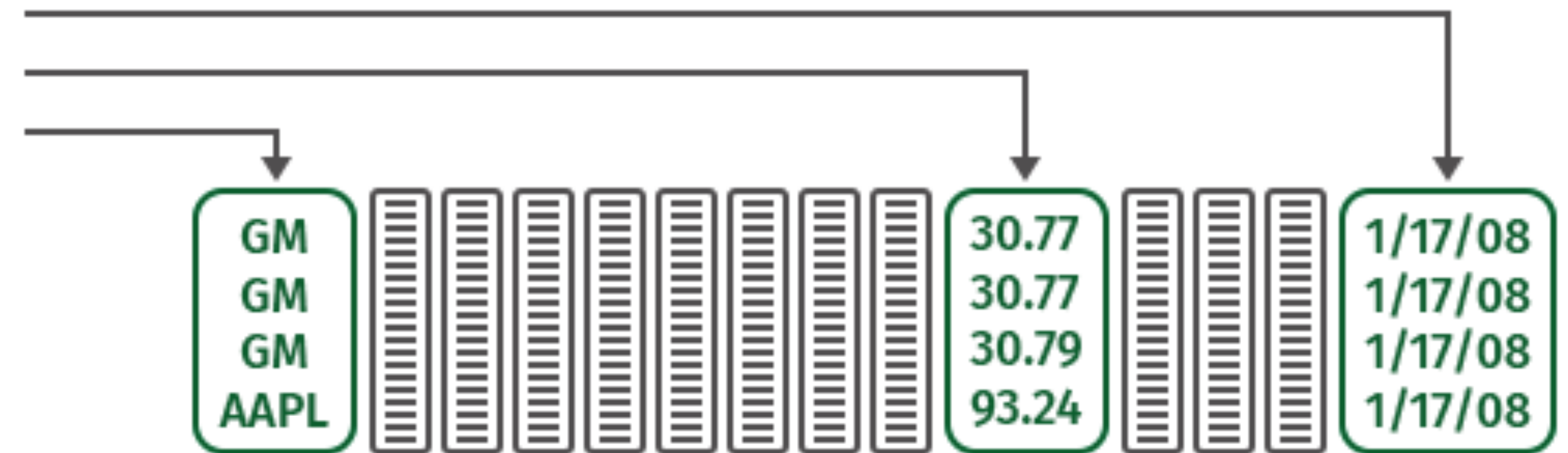
Чем еще отличаются MPP-системы от локальных?

Если мы читаем строки, то, например, для выполнения команды:

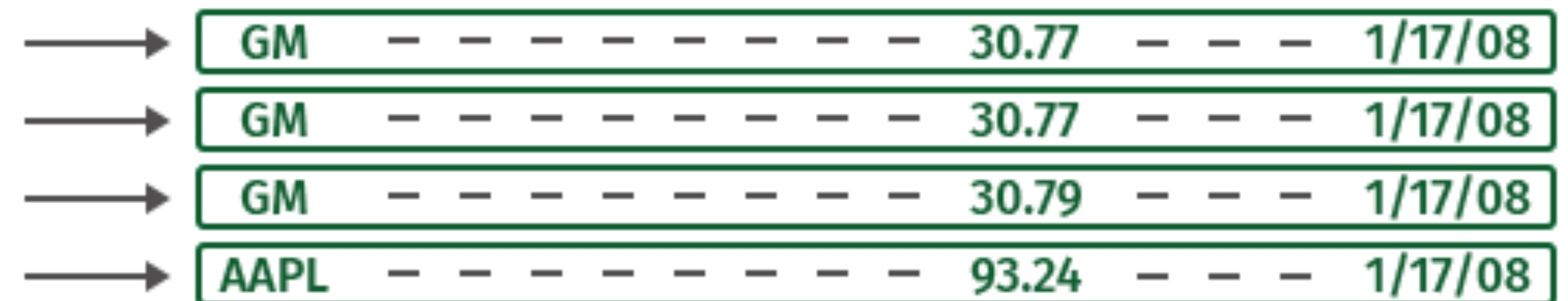
```
SELECT 1,11,15  
FROM table1
```

нам придется читать всю таблицу.
Это огромный объем информации.
В данном случае колоночный подход выгоднее. Он позволяет считать только три нужных нам столбца, экономя память и время

**Хранение
по столбцам**
Чтение трёх столбцов



**Хранение
по строкам**
Чтение всех столбцов



Что такое Parquet

Это бинарный, колонно-ориентированный формат хранения данных, изначально созданный для экосистемы hadoop. Разработчики уверяют, что данный формат хранения идеален для big data (неизменяемых).

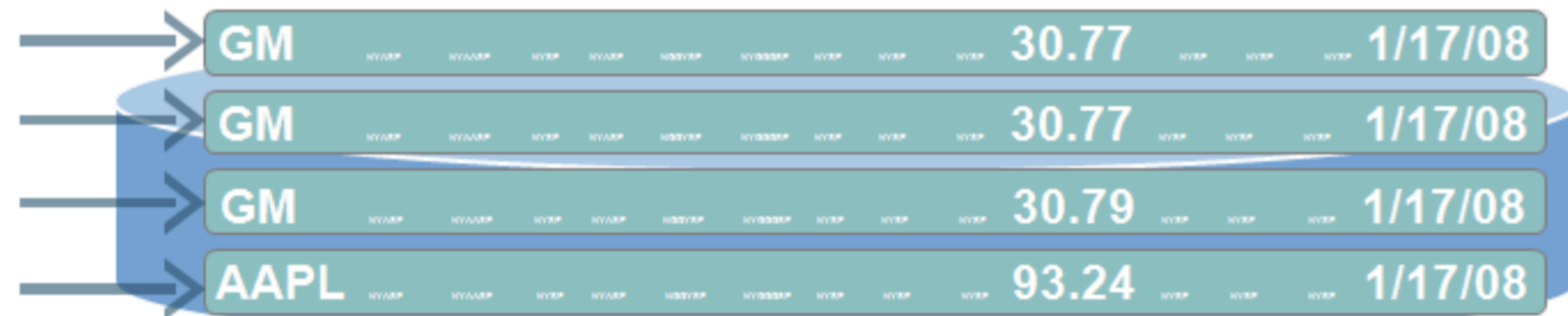
Хранение по столбцам

Чтение 3х столбцов



Хранение по строкам

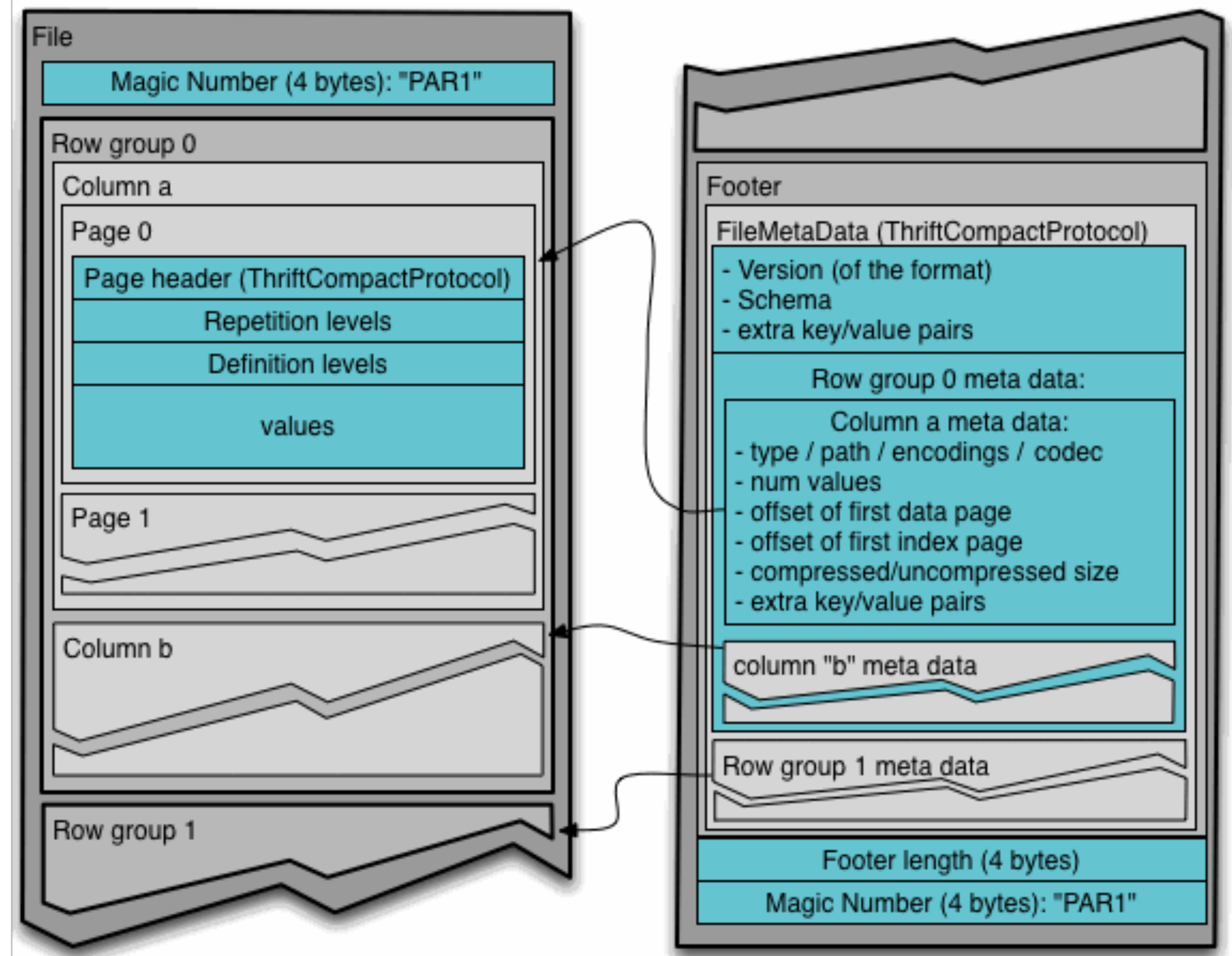
Чтение всех столбцов



Как выглядит структура Parquet файлов

Файлы имеют несколько уровней разбиения на части, благодаря чему возможно довольно эффективное параллельное исполнение операций поверх них:

- Row-group — это разбиение, позволяющее параллельно работать с данными на уровне Map-Reduce
- Column chunk — разбиение на уровне колонок, позволяющее распределять IO операции
- Page — Разбиение колонок на страницы, позволяющее распределять работу по кодированию и сжатию



Поколоночное vs Построчное хранение данных

CSV - пример построчного хранения, Parquet - пример поколоночного хранения данных.
Рассмотрим пример формирования файлов:

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

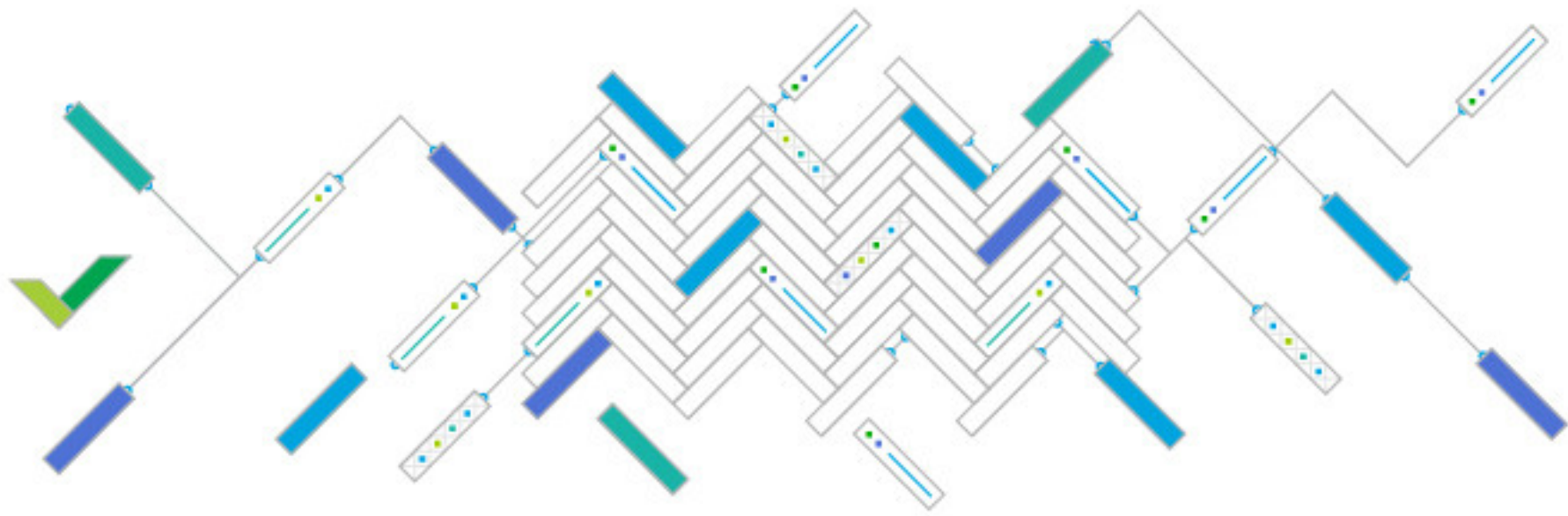
Тогда в текстовом файле, скажем, csv мы бы хранили данные на диске примерно так:

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

В случае с Parquet:

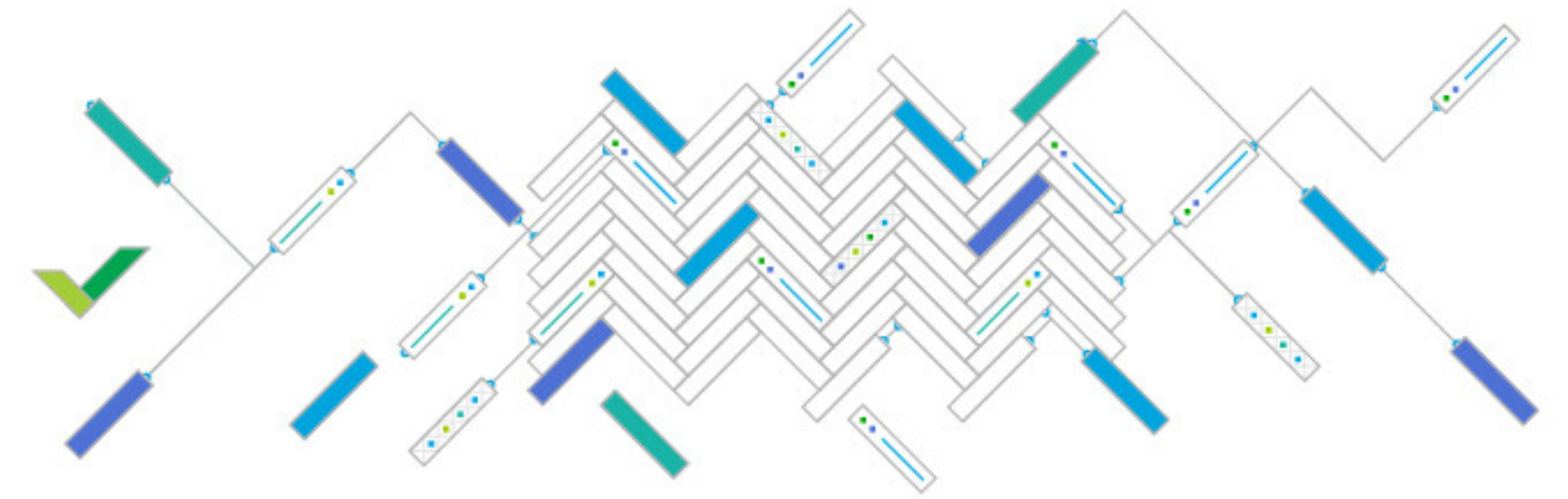
A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

Достоинства и недостатки колодочного хранения



Достоинства:

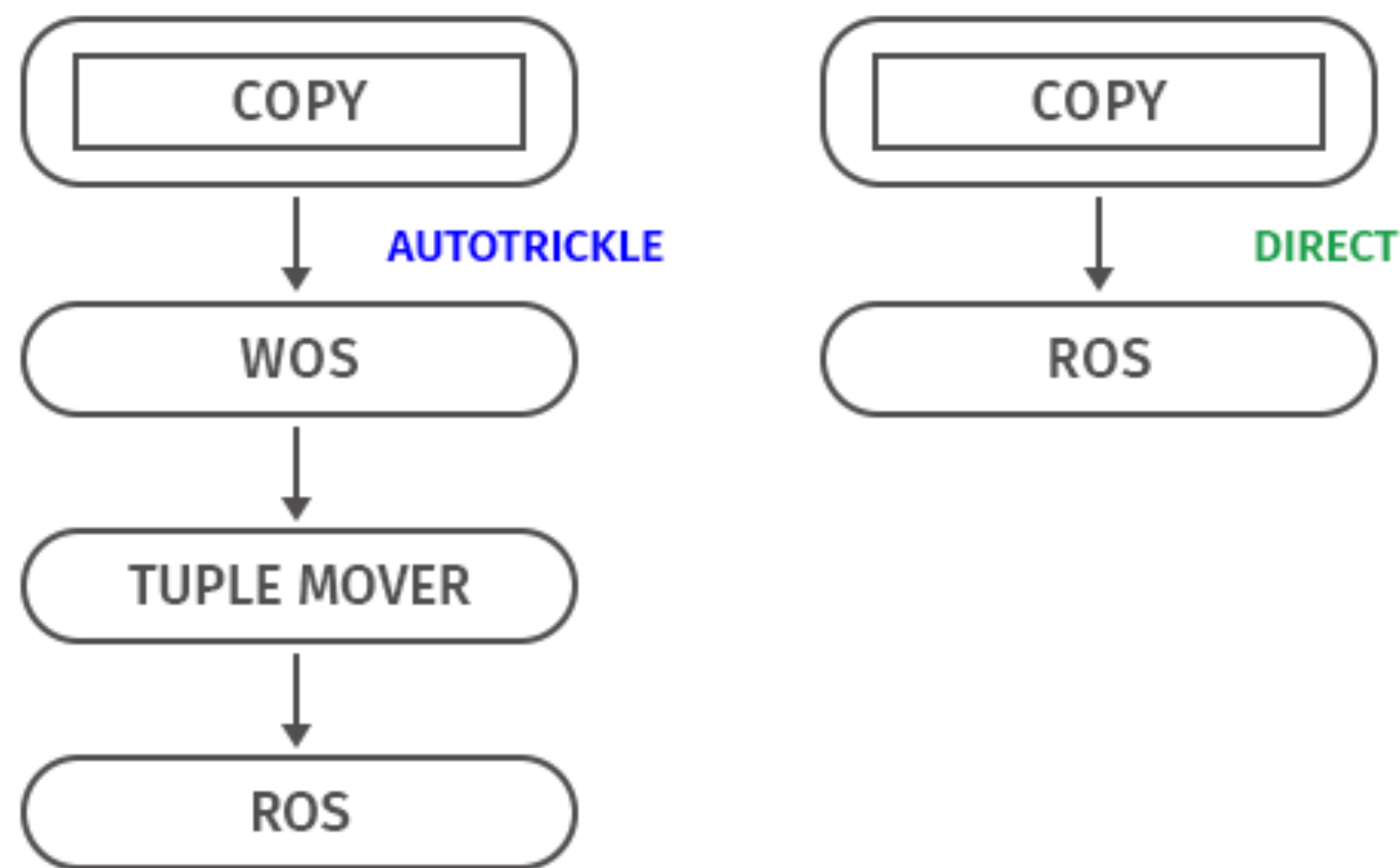
- Несмотря на то, что они и созданы для hdfs, данные могут храниться и в других файловых системах, таких как GlusterFs или поверх NFS
- По сути это просто файлы, а значит с ними легко работать, перемещать, бэкапить и реплицировать.
- Колончатый вид позволяет значительно ускорить работу аналитика, если ему не нужны все колонки сразу.
- Минимизированное хранение с точки зрения занимаемого места.
- Скорость чтения по сравнению с использованием других файловых форматов.



Недостатки:

- Колончатый вид заставляет задумываться о схеме и типах данных.
- Parquet не всегда обладает нативной поддержкой в других продуктах.
- Не поддерживает изменение данных и эволюцию схемы. Чтобы что-то изменить в уже существующим файле, нельзя обойтись без перезаписи, разве что можно добавить новую колонку.
- Не поддерживаются транзакции, так как это обычные файлы а не БД.

С какими проблемами сталкивается колоночное хранение



Вне зависимости от того, куда записаны данные — в WOS или в ROS — они доступны сразу. Но из WOS чтение идет медленнее, потому что данные там не сгруппированы.

В таких системах должен присутствовать инструмент-уборщик, который выполняет две операции:

- Moveout — сжимает и сортирует данные в WOS, перемещает их в ROS и создает для них в ROS новые контейнеры.
- Mergeout — подметает за нами, когда мы используем DIRECT. Мы не всегда способны грузить столько информации, чтобы получались большие ROS-контейнеры. Поэтому он периодически объединяет небольшие контейнеры ROS в более крупные, очищает данные с пометкой на удаление, работая при этом в фоновом режиме (по времени, заданному в конфигурации).

Безопасность и аудит

Основные функции системы безопасности

- **Проверка подлинности (аутентификация)** – процедура проверки соответствия некоего лица и его учетной записи в компьютерной системе.
 - Например: Пользовательский идентификатор и пароль = некоторая конфиденциальная информации, знание которой обеспечивает владение определенным ресурсом.
 - Аутентификацию != Идентификация.
- **Авторизация** – это предоставление лицу прав на какие-то действия в системе.

Дополнительные функции безопасности

- Шифрование
- Контекстное переключение
- Олицетворение
- Встроенные средства управления ключами

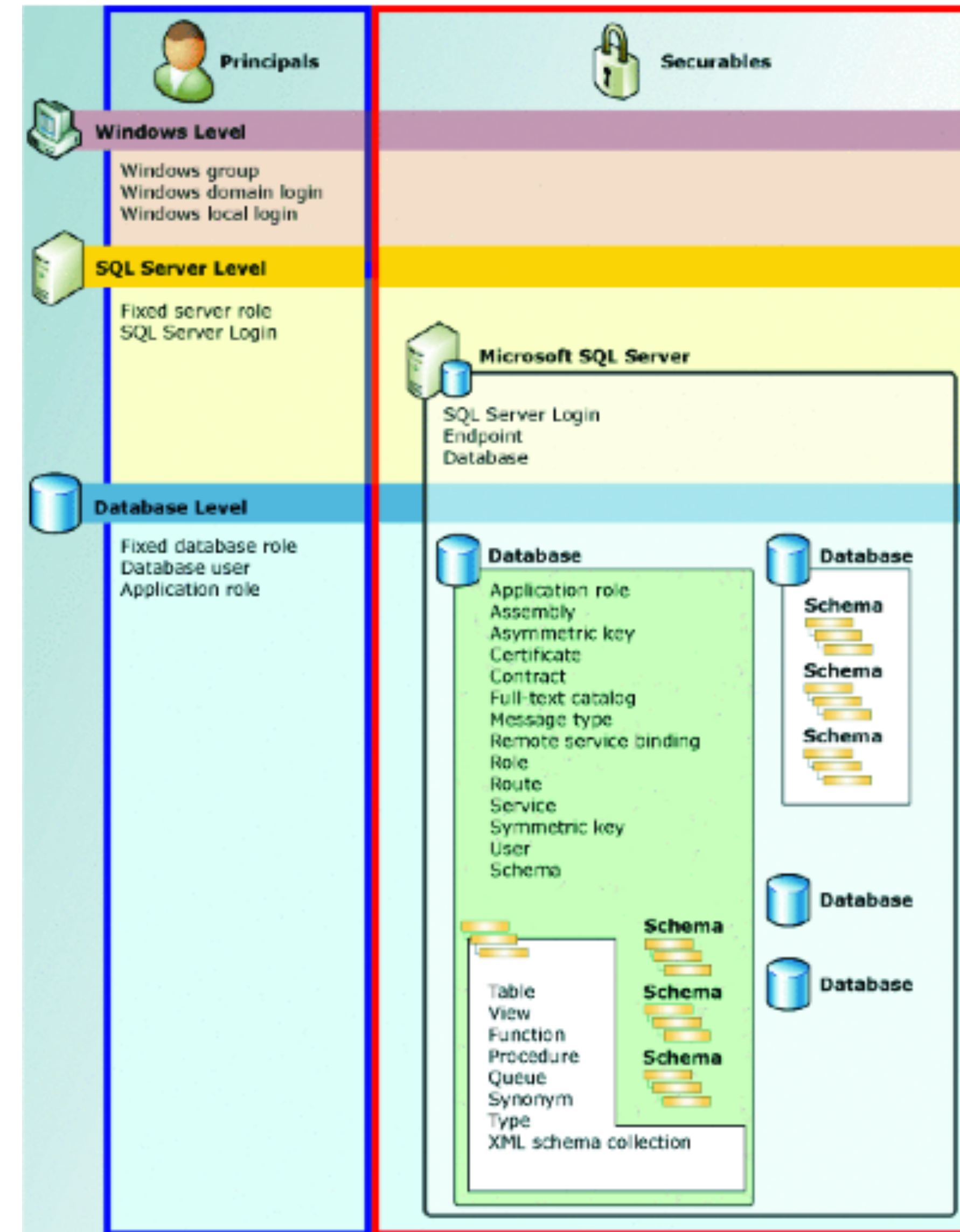
Три ключевых понятия системы безопасности

- Участники системы безопасности (Principals)
- Защищаемые объекты (Securables)
- Система разрешений (Permissions)

Участники системы безопасности

Область влияния принципала зависит:

- от области его определения:
 - Операционная система
 - СУБД
 - База данных
- от количества пользователей области:
 - Индивидуальный (неделимый) участник, например, логин в ОС
 - Коллективный участник - группа пользователей ОС



Защищаемые объекты

Защищаемые объекты – это ресурсы, доступ к которым регулируется системой авторизации.

К областям защищаемых объектов относятся:

- Сервер - управление учетными записями подключения (login)
- База данных - управление:
 - учетными записями пользователей (user)
 - ролями (role)
- Схема - управление разрешениями на объекты БД:
 - функции (function)
 - процедуры (stored procedure)
 - таблицы (table)
 - представления (view)

Пользователи и группы пользователей

Работа с пользователем:

- Создать пользователя
 - CREATE USER Jonathan;
 - CREATE USER davide WITH PASSWORD 'jw8s0F4';
 - CREATE USER miriam WITH PASSWORD 'jw8s0F4' VALID UNTIL '2005-01-01';
- Удалить пользователя
 - DROP USER Jonathan

Группы упрощают процедуру назначения прав пользователям. Обычные привилегии должны назначаться каждому пользователю по отдельности.

- Добавить пользователя в группу
 - ALTER GROUP workers DROP USER beth;

Роли

Права можно выдавать для всей группы и у всей группы забирать. Для этого создаётся роль, представляющая группу, а затем членство в этой группе выдаётся ролям индивидуальных пользователей.

Для настройки групповой роли сначала нужно создать саму роль:

- `CREATE ROLE имя;`
- `GRANT групповая_роль TO роль1, ... ;`
- `REVOKE групповая_роль FROM роль1, ... ;`

Обычно групповая роль не имеет атрибута `LOGIN`, хотя при желании его можно установить.

Членом роли может быть и другая групповая роль. При этом база данных не допускает замыкания членства по кругу. Также не допускается управление членством роли `PUBLIC` в других ролях.

Члены групповой роли могут использовать её права двумя способами:

- Каждый член группы может явно выполнить `SET ROLE`, чтобы временно «стать» групповой ролью.
- Роли, имеющие атрибут `INHERIT`, автоматически используют права всех ролей, членами которых они являются, в том числе и унаследованные этими ролями права.

Роли. Пример

```
CREATE ROLE joe LOGIN INHERIT;  
CREATE ROLE admin NOINHERIT;  
CREATE ROLE wheel NOINHERIT;  
GRANT admin TO joe;  
GRANT wheel TO admin;
```

После подключения с ролью joe сеанс базы данных будет использовать права, выданные напрямую joe, и права, выданные роли admin, так как joe «наследует» права admin.

Однако права, выданные wheel, не будут доступны, потому что, хотя joe неявно и является членом wheel, это членство получено через роль admin, которая имеет атрибут NOINHERIT.

После выполнения команды:

```
SET ROLE admin;
```

сеанс будет использовать только права, назначенные admin, а права, назначенные роли joe, не будут доступны.

После выполнения команды:

```
SET ROLE wheel;
```

сеанс будет использовать только права, выданные wheel, а права joe и admin не будут доступны.

Система разрешений

У субъекта системы есть только один путь получения доступа к объектам - иметь назначенные непосредственно или опосредовано разрешения.

- При непосредственном управлении разрешениями они назначаются субъекту явно
- При опосредованном разрешения назначаются через членство в группах, ролях или наследуются от объектов, лежащих выше по цепочке иерархии.

Управление разрешениями производится путем выполнения инструкций языка DCL:

- GRANT (разрешить)
- DENY (запретить)
- REVOKE (отменить)

Система разрешений. Примеры

Какие задачи выполняют следующие запросы:

- GRANT CONNECT ON DATABASE database_name TO user_name;
- GRANT ALL PRIVILEGES ON kinds TO Manuel;
- GRANT CREATE TABLE ON table_name TO role_name.
- GRANT INSERT ON films TO PUBLIC;

- GRANT USAGE ON SCHEMA schema1 TO readonly;
- GRANT SELECT ON ALL TABLES IN SCHEMA schema1 TO readonly;

- REVOKE CONNECT ON DATABASE your_database FROM PUBLIC;

Модели управления доступом

- **MAC** — мандатная модель управления доступом
- **DAC** — прямое управление доступом
- **RBAC** — ролевая модель управления доступом
- **ABAC** — атрибутивная модель управления доступом

Мандатная модель управления доступом

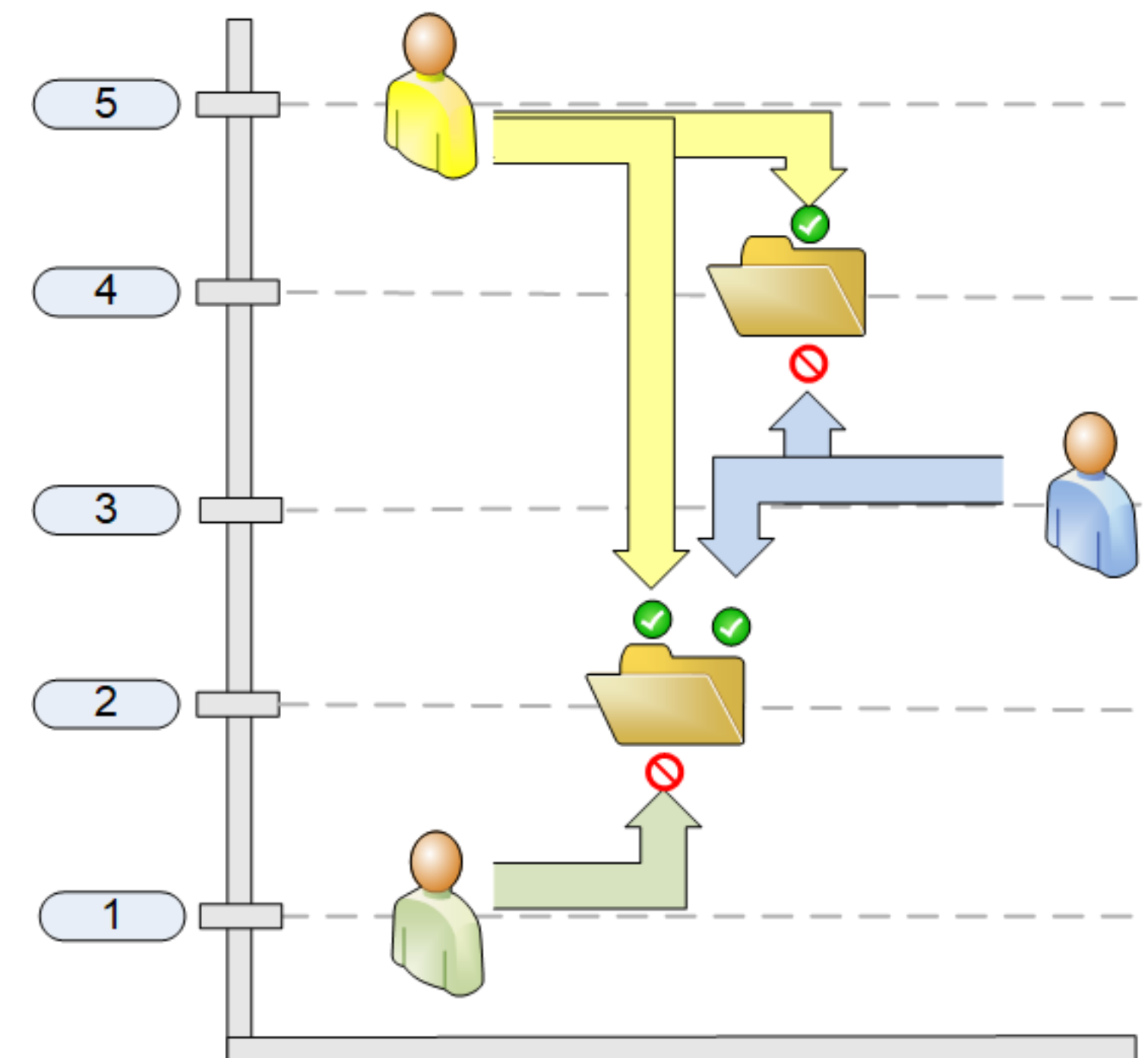
Доступ субъекта к объекту определяется его уровнем доступа: уровень доступа субъекта должен быть не ниже уровня секретности объекта.

Достоинства:

- простота построения общей схемы доступа
- простота администрирования

Недостатки:

- проблема разграничения пользователей одного уровня
- пользователь не может назначать доступ к объекту



Прямое управление доступом

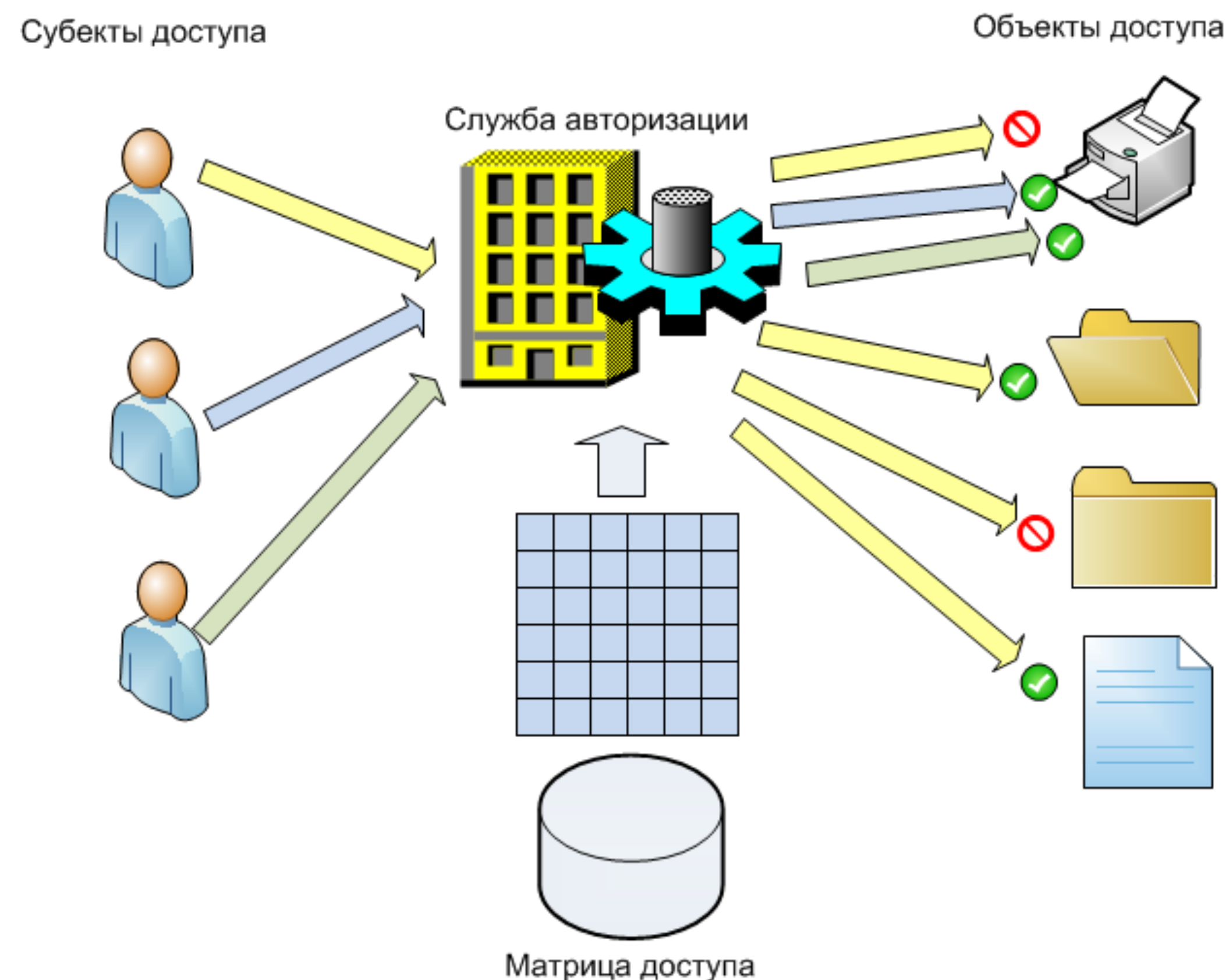
Доступ субъекта к объекту определяется наличием субъекта в списке доступа (ACL) объекта.

Достоинства:

- простота реализации
- гибкость (пользователь может описать доступ к своим ресурсам)

Недостатки:

- излишняя детализированность (приводит к запутанности)
- сложность администрирования
- пользователь может допустить ошибку при назначении прав



Ролевая модель управления доступом

Доступ субъекта к объекту определяется наличием у субъекта роли, содержащей полномочия, соответствующие запрашиваемому доступу.

Формирование ролей:

- Ролевое разграничение доступа позволяет реализовать гибкие, изменяющиеся динамически в процессе функционирования компьютерной системы правила
- Как правило, данный подход применяется в системах защиты СУБД
- Ролевой подход часто используется в системах, для пользователей которых чётко определён круг их должностных полномочий и обязанностей



Атрибутивная модель управления доступом

Доступ субъекта к объекту определяется динамически на основании анализа политик учитывающих значения атрибутов субъекта, объекта и окружения.

Политика на основе атрибутов:

- Делает управление доступом более эффективным, уменьшая сложность нормативных требований.
- Одна и та же политика может использоваться в разных системах. Это помогает управлять согласованностью доступа к ресурсам в пределах одной компании
- Централизованное управление доступом предполагает единственный авторитетный источник для правил доступа



Журнализация

Типы используемых файлов

Базы данных SQL Server содержат файлы трех типов:

- Первичные файлы данных
- Вторичные файлы данных
- Файлы журналов

Операции журнала транзакций

Журнал транзакций поддерживает следующие операции:

- Восстановление отдельных транзакций
- Восстановление всех незавершенных транзакций при запуске SQL Server
- Накат восстановленной базы данных, файла, файловой группы или страницы до момента сбоя
- Поддержка репликации транзакций
- Поддержка решений с резервными серверами

Восстановление данных

SQL Server поддерживает восстановление данных на следующих уровнях:

- База данных (полное восстановление базы данных)
- Файл данных (восстановление файла)
- Страница данных (восстановление страницы)

Структура Лог-файла

На логическом уровне журнал транзакций состоит из последовательности записей. Двумя основными типами записи Log-файла являются:

- код выполненной логической операции
 - Для наката логической операции выполняется эта операция.
 - Для отката логической операции выполняется логическая операция, обратная зарегистрированной.
- исходный и результирующий образ измененных данных
 - Для наката операции применяется результирующий образ
 - Для отката операции применяется исходный образ.

Модель восстановления

Модель восстановления — это свойство базы данных, которое управляет процессом регистрации транзакций, определяет, требуется ли для журнала транзакций резервное копирование, а также определяет, какие типы операций восстановления доступны.

Есть три модели восстановления:

- Модель полного восстановления (FULL)
- Модель восстановления с неполным протоколированием (BULK_LOGGED)
- Простая модель восстановления (SIMPLE)

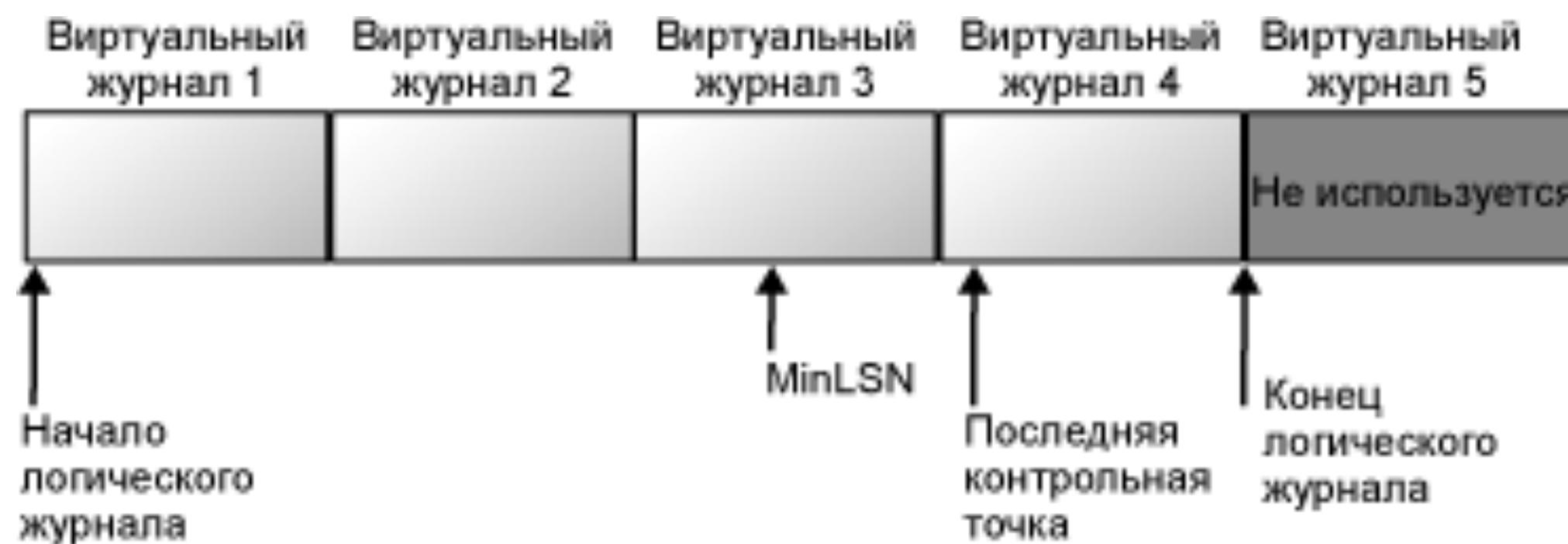
Логическая архитектура журнала транзакций

На логическом уровне журнал транзакций состоит из последовательности записей. Каждая запись содержит:

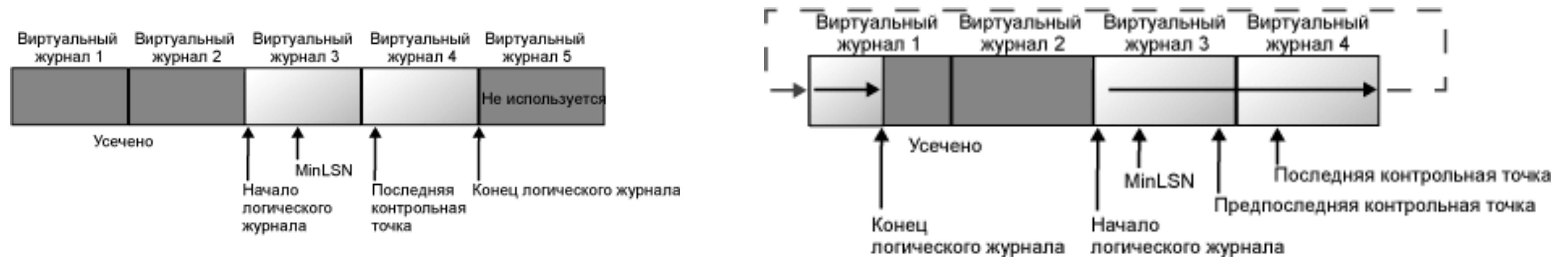
- Log Sequence Number: регистрационный номер транзакции (LSN). Каждая новая запись добавляется в логический конец журнала с номером LSN, который больше номера LSN предыдущей записи.
- Prev LSN: обратный указатель, который предназначен для ускорения отката транзакции.
- Transaction ID number: идентификатор транзакции.
- Type: тип записи Log-файла.
- Other information: Прочая информация.

Управление журналом транзакций

Чтобы логический журнал не увеличивался до размера физических файлов журнала, следует периодически выполнять его усечение. Процесс усечения журнала уменьшает размер файла логического журнала, помечая виртуальные файлы журнала, которые не содержат частей логического журнала, как неактивные. В некоторых случаях может оказаться полезным физическое сжатие или расширение размера реального файла журнала.



Усечение журнала транзакций

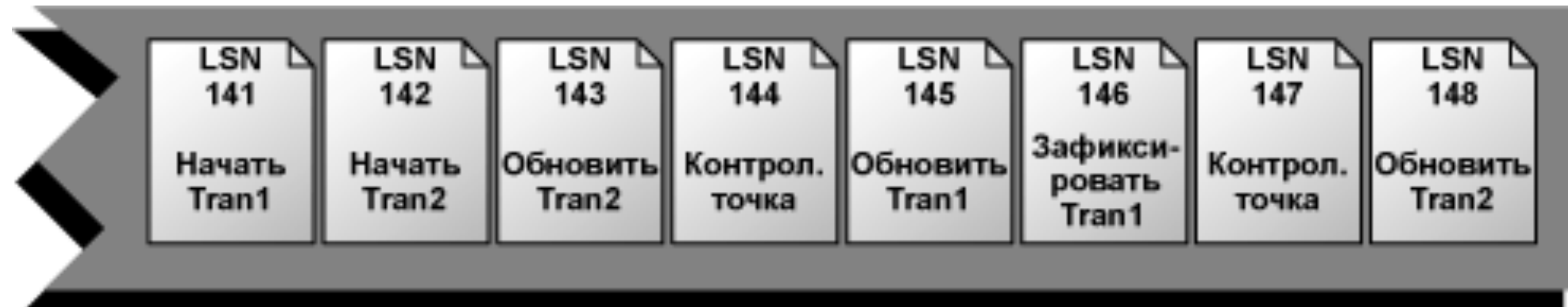


MinLSN - регистрационный номер самой старой записи, которая необходима для успешного отката на уровне всей базы данных.

Так как все изменения страниц данных происходят в страничных буферах, то изменения данных в памяти не обязательно отражаются в этих страницах на диске. Чтобы эти страницы были записаны на диск применяется контрольная точка. Все грязные страницы должны быть сохранены на диске в обязательном порядке.

Активный журнал

Часть журнала, начинающаяся с номера MinLSN и заканчивающаяся последней записью, называется активной частью журнала, или активным журналом. Этот раздел журнала необходим для выполнения полного восстановления базы данных. Ни одна часть активного журнала не может быть усечена. Все записи журнала до номера MinLSN должны быть удалены из частей журнала.



Контрольные точки и активная часть журнала

Контрольная точка выполняет в базе данных следующее:

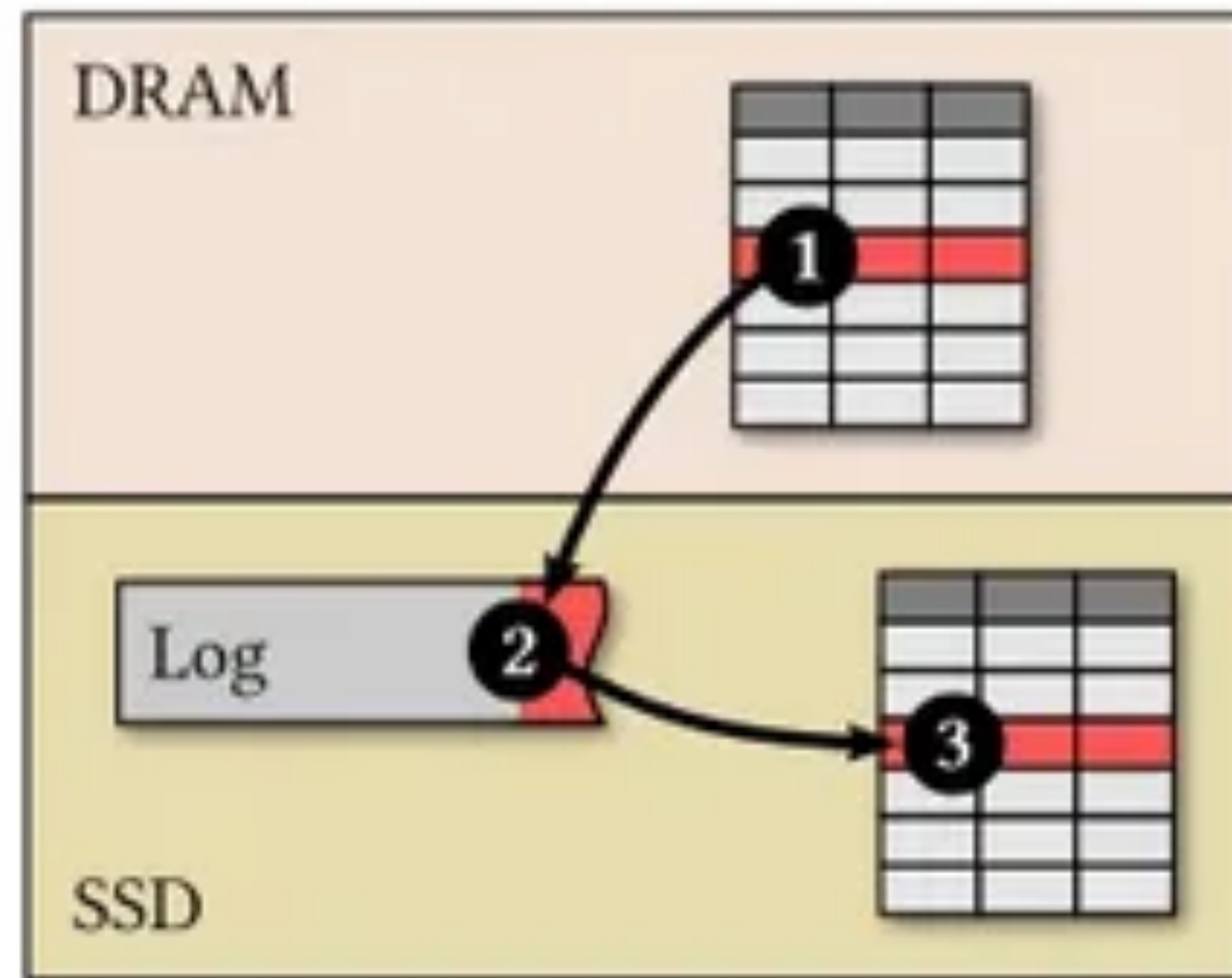
- Записывает в файл журнала запись, отмечающую начало контрольной точки
- Сохраняет данные, записанные для контрольной точки в цепи записей журнала контрольной точки
- Если база данных использует простую модель восстановления, помечает для повторного использования пространство, предшествующее номеру MinLSN
- Записывает все измененные страницы журналов и данных на диск
- Записывает в файл журнала запись, отмечающую конец контрольной точки
- Записывает в страницу загрузки базы данных номер LSN начала соответствующей цепи

Действия, приводящие к срабатыванию контрольных точек

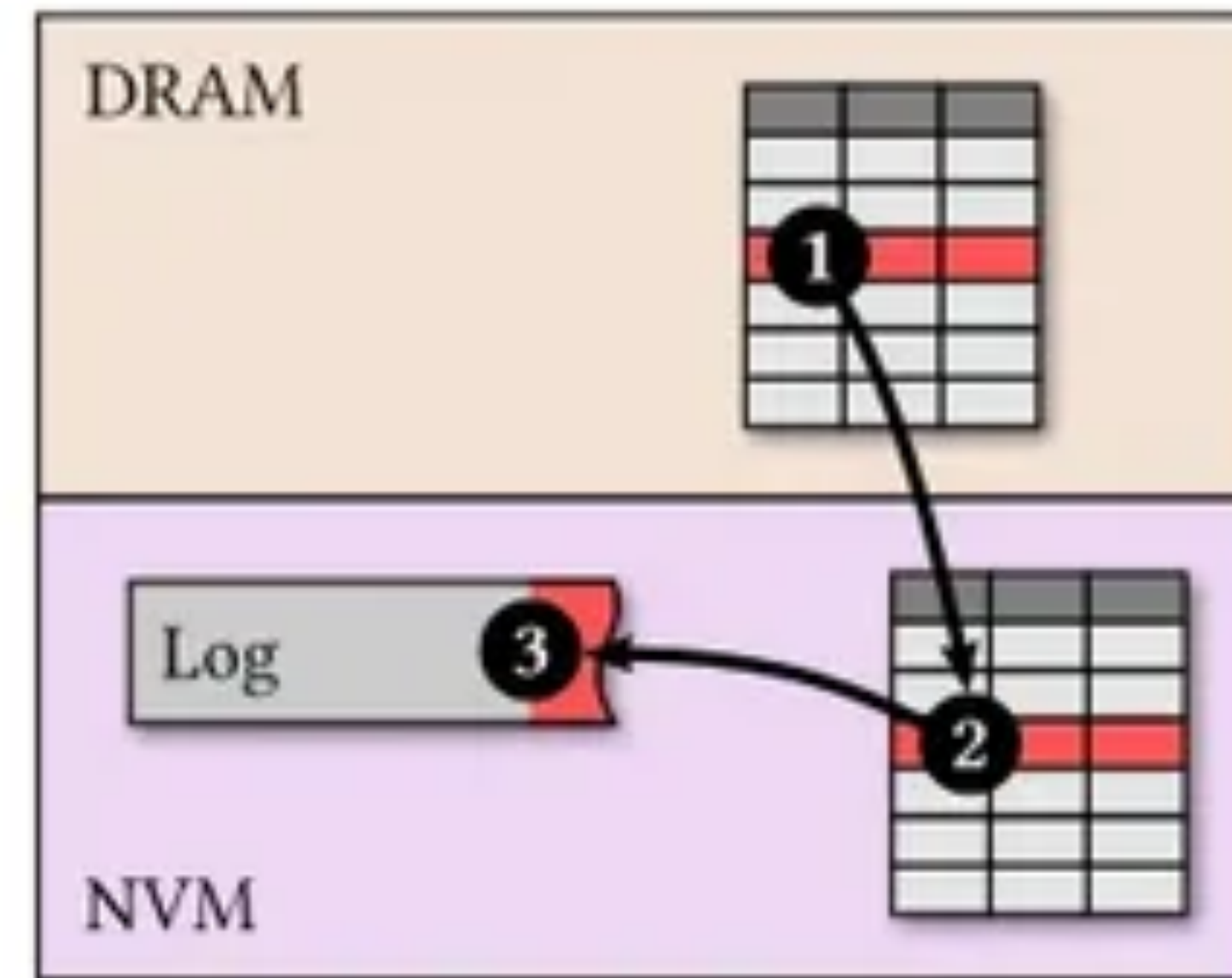
Контрольные точки срабатывают в следующих ситуациях:

- При явном выполнении инструкции CHECKPOINT. Контрольная точка срабатывает в текущей базе данных соединения
- При выполнении операции массового копирования для базы данных, на которую распространяется модель восстановления с неполным протоколированием
- При добавлении или удалении файлов баз данных с использованием инструкции ALTER DATABASE
- При остановке экземпляра SQL Server с помощью инструкции SHUTDOWN или при остановке службы SQL Server (MSSQLSERVER)
- Если экземпляр SQL Server периодически создает в каждой базе данных автоматические контрольные точки для сокращения времени восстановления базы данных
- При создании резервной копии базы данных
- При выполнении действия, требующего отключения базы данных

Механизм записи транзакции в лог



Write-ahead logging



Write-behind logging

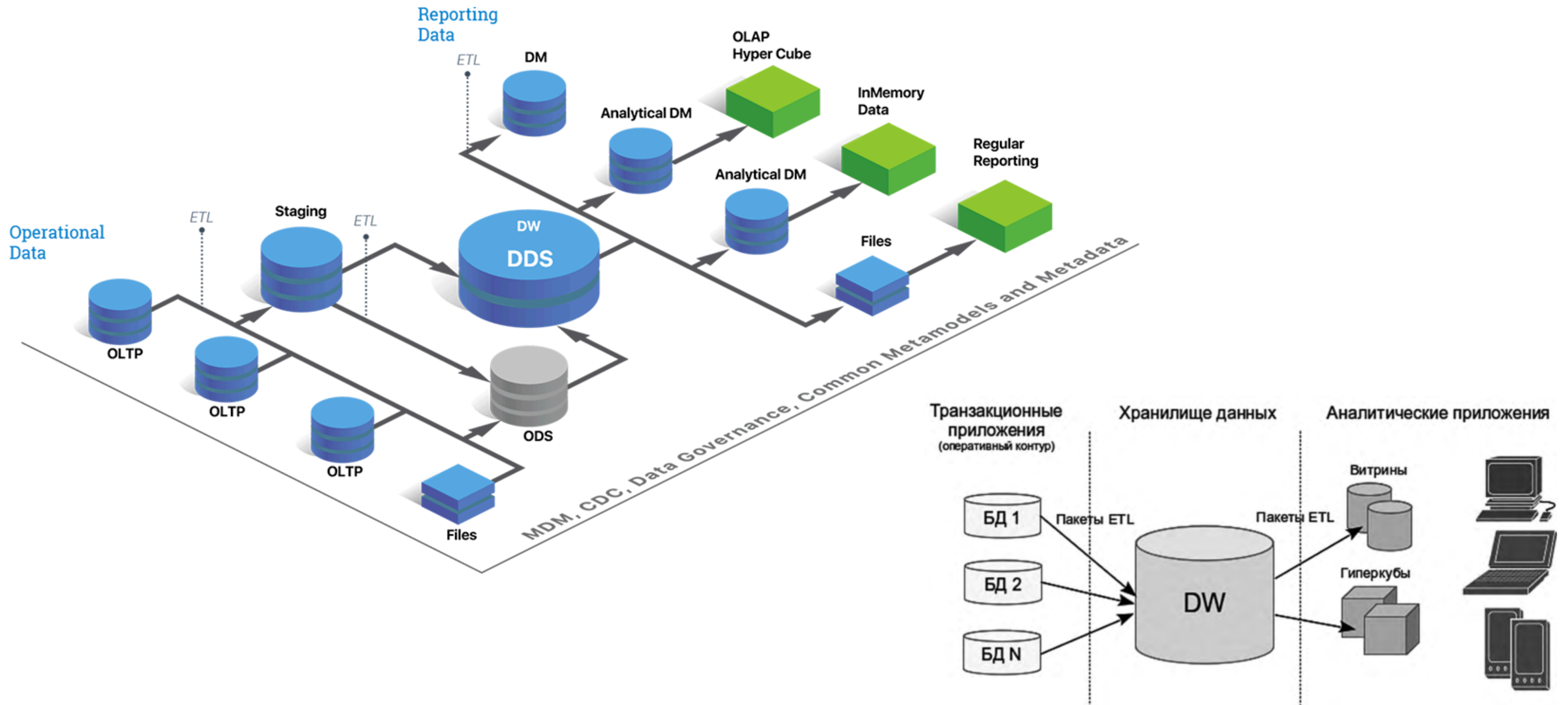
Большинство РСУБД используют журнал с упреждающей записью, который гарантирует, что до занесения на диск записи, связанной с журналом, никакие изменения данных записаны не будут. Таким образом обеспечиваются свойства ACID для транзакции.

Аналитические хранилища

Пространственная модель

ETL и ELT

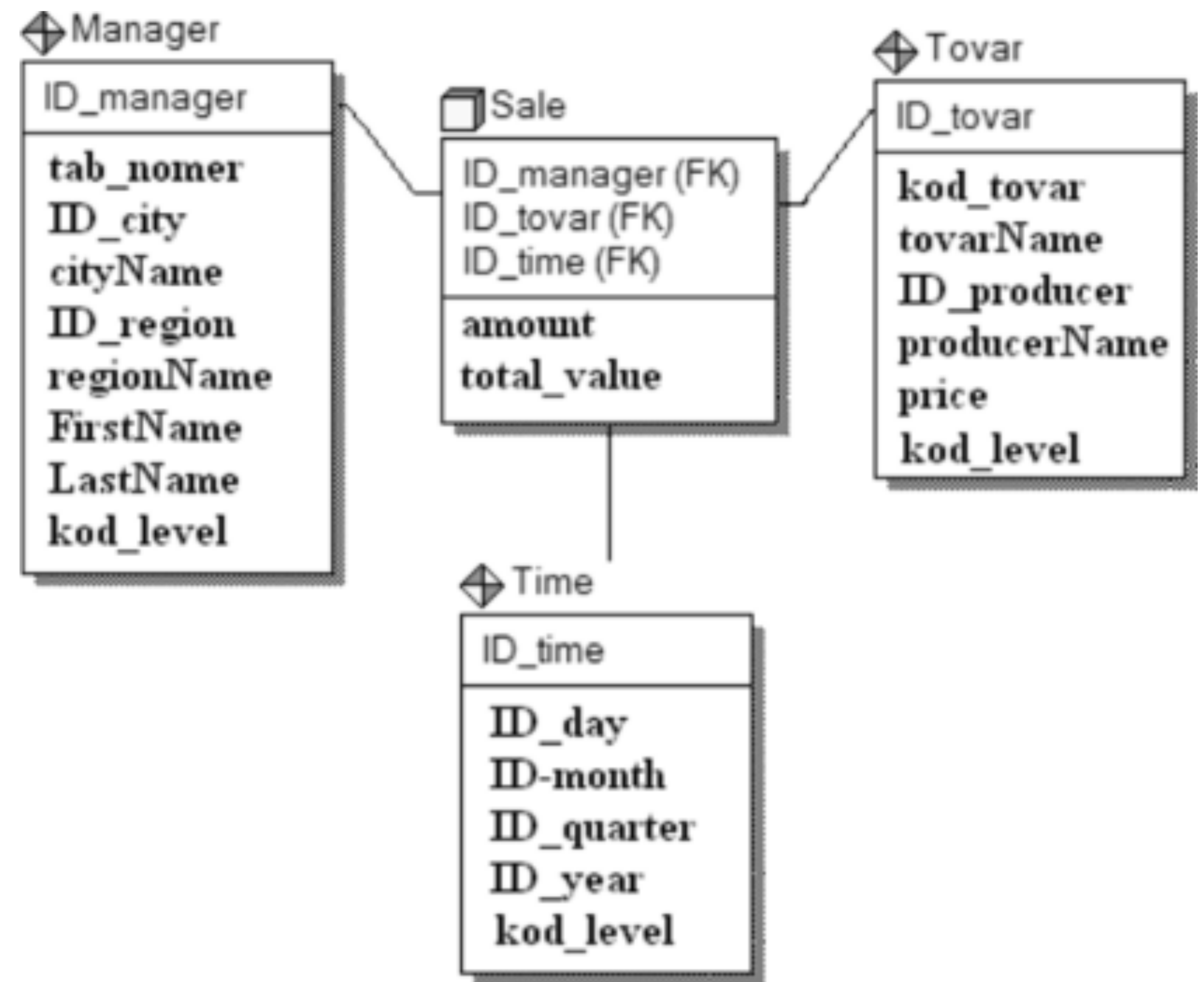
Аналитические хранилища данных



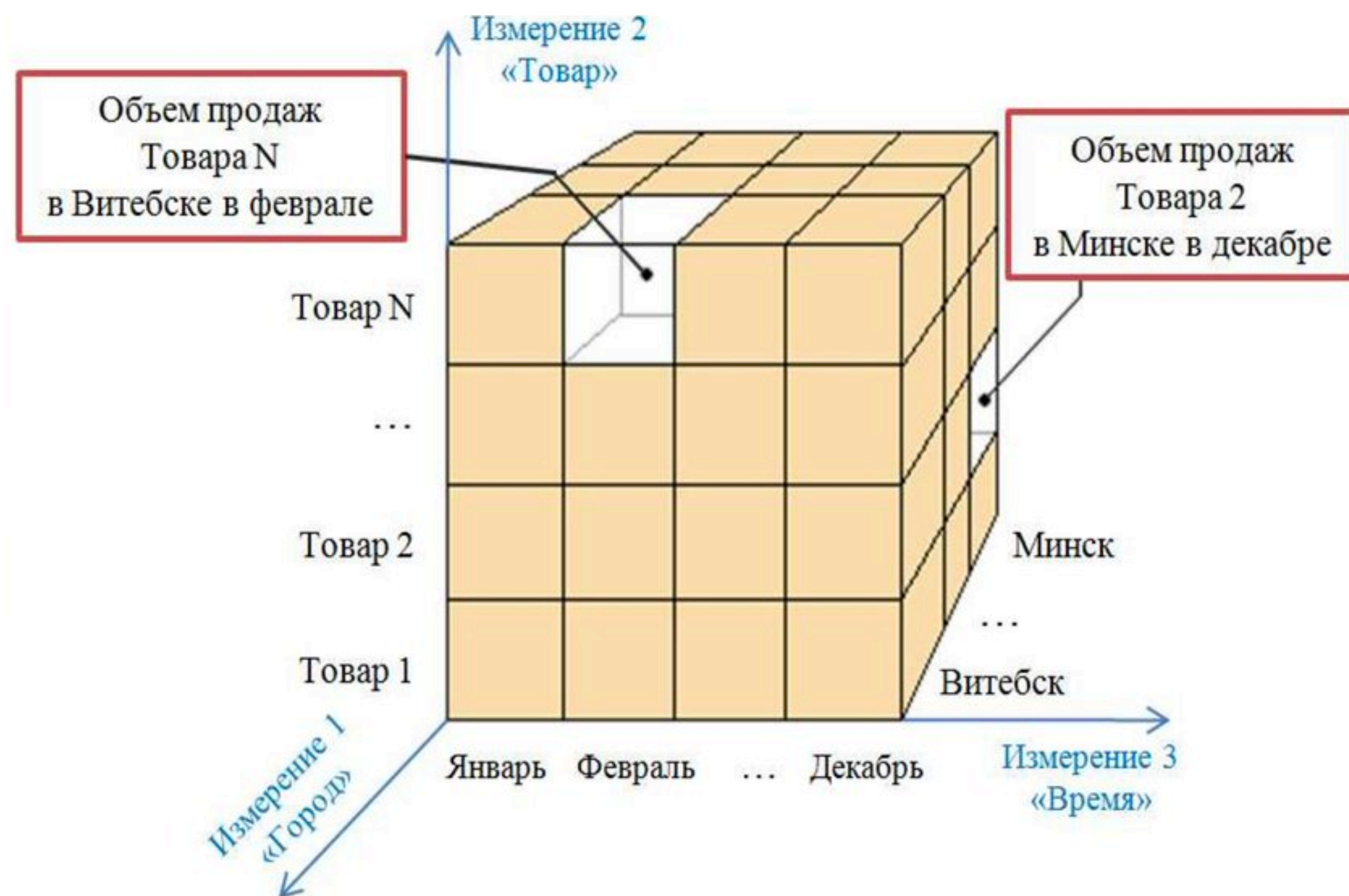
Пространственная модель

Если реляционная модель акцентируется на целостности и эффективности ввода данных, то **размерная модель (Dimensional)** ориентирована в первую очередь на выполнение сложных запросов к БД.

В размерном моделировании для ХД принят стандарт модели, называемый **схемой звезда** (star schema), которая обеспечивает высокую скорость выполнения запроса посредством денормализации и разделения данных.



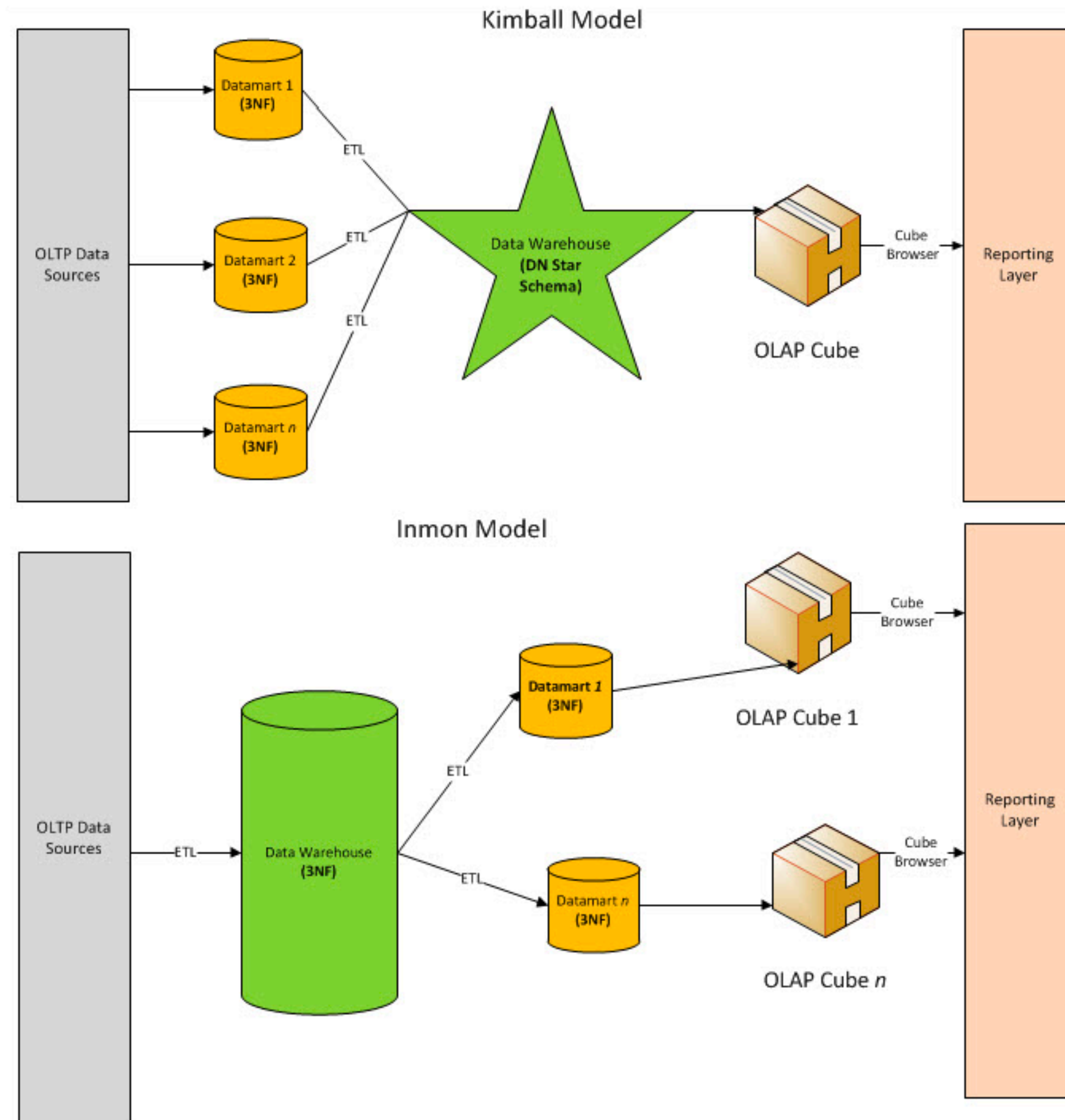
OLAP-куб



Осями многомерной системы координат служат основные атрибуты анализируемого бизнес-процесса.

Например, для продаж это могут быть товар, регион, тип покупателя. На пересечениях осей измерений (Dimensions) находятся данные, количественно характеризующие процесс — меры (Measures).

Kimball vs. Inmon



Отличия двух подходов:

1. Архитектура хранилища
 1. у Кимболла - пространственная организации
 2. у Инмона - нормализованная.
2. Физическая организации хранилища.
 1. у Инмона - это физически целостный реально существующий объект
 2. у Кимболла - скорее "виртуальный" объект. Коллекция витрин данных, которые могут быть пространственно разобщенными.

ETL – аббревиатура от Extract, Transform, Load

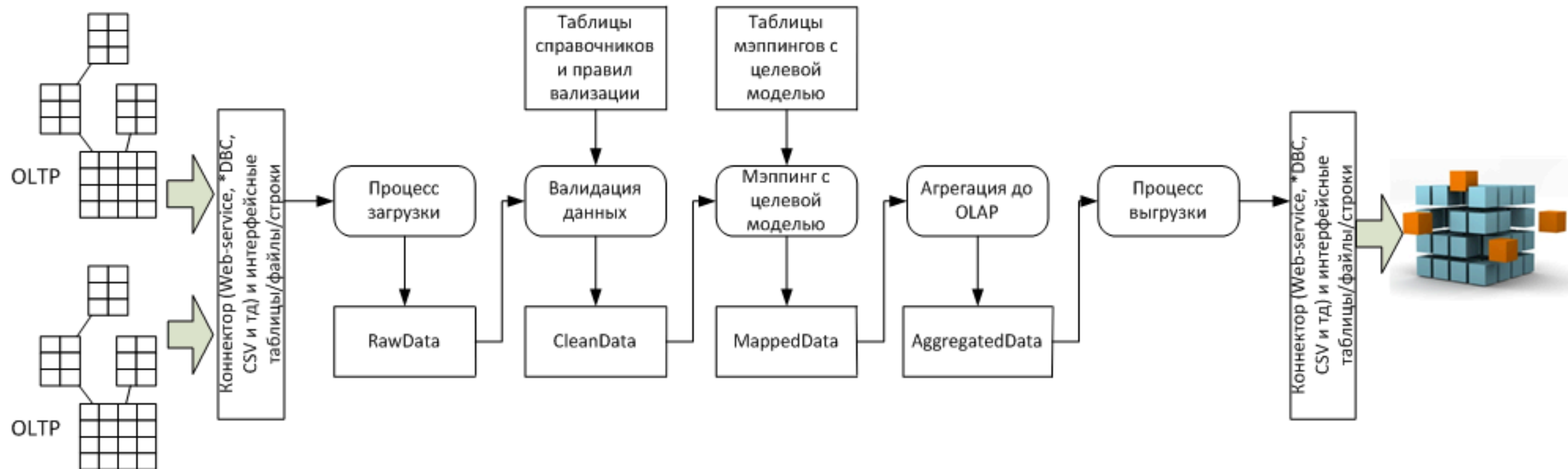
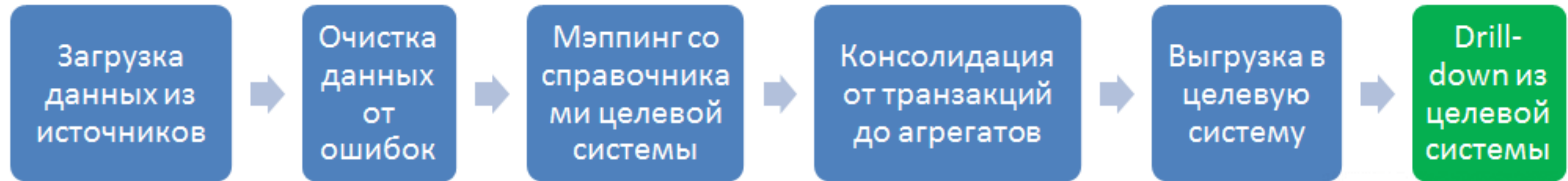
Какие задачи решает:

- Привести все данные к единой системе значений и детализации, попутно обеспечив их качество и надежность;
- Обеспечить аудиторский след при преобразовании (Transform) данных, чтобы после преобразования можно было понять, из каких именно исходных данных и сумм собралась каждая строка преобразованных данных.

Виды ошибок данных:

- Как случайные ошибки, возникшие на уровне ввода, переноса данных, или из-за багов;
- Как различия в справочниках и детализации данных между смежными ИТ- системами.

Как работает ETL система



Процесс загрузки данных

Задача этого этапа - затянуть в ETL данные произвольного качества для дальнейшей обработки. При проектировании процесса загрузки данных необходимо помнить о том что:

- Надо учитывать требования бизнеса по длительности всего процесса;
- Данные могут загружаться набегаящей волной;
- Данные могут перегружаться много раз;
- Данные всегда содержат ошибки;
- Надо учитывать возможность обогащения данных.

Процесс валидации

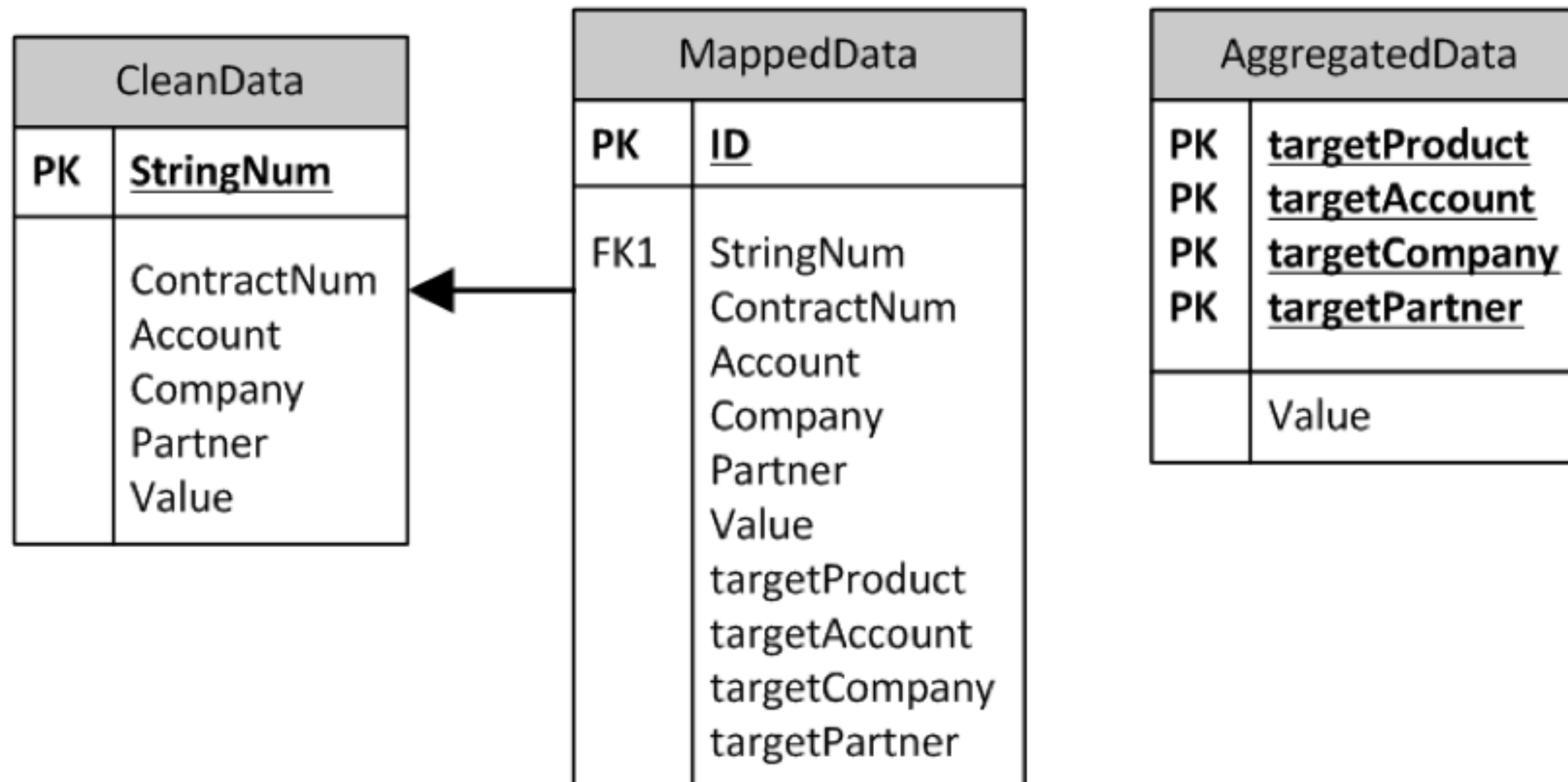
Данный процесс отвечает за выявление ошибок и пробелов в данных, переданных в ETL.

Главный вопрос – как вычислить возможные виды ошибок в данных, и по каким признакам их идентифицировать?

Типы данных	Внутри поля	По отношению к другим полям	Совместимость форматов при передачи между системами
Перечисление и текст	• Не из списка разрешенных значений • Отсутствие обязательных значений • Не соответствие формату (например, все договора должны нумероваться «ДГВ»)	• Не из списка разрешенных значений для связанного элемента • Отсутствие обязательных элементов для связанного элемента • Не соответствие формату для заданного элемента (например, для продукта «АИС» все договора должны нумероваться «АИСxxxxx»)	• Символы допустимые в одном формате, недопустимы в другом • Кодировка • Обратная совместимость • Новые значения (нет в мэппингах) • Устаревшие значения (не из списка разрешенных в целевой системе)
Числа и порядки	• Не число • Не в границах разрешенного интервала значений • Пропущено порядковое	• Не выполняется отношение • Присвоен неправильный порядковый номер • Разницы за счет разных правил округления значений	• Переполнение • Потеря точности и знаков • Несовместимость форматов при конвертации не в число
Даты и периоды		• День недели не соответствует дате • Сумма единиц времени не соответствует из-за разницы рабочие/не рабочие/праздничные/сокращенные дни	• Несовместимость форматов даты при передаче текстом • Ошибка точности отсчета и точности при передаче числом

Процесс мэппинга данных

Таблица замэпленных данных должна включать одновременно два набора полей – старых и новых аналитик, чтобы можно было сделать select по исходным аналитикам и посмотреть, какие целевые аналитики им присвоены, и наоборот:

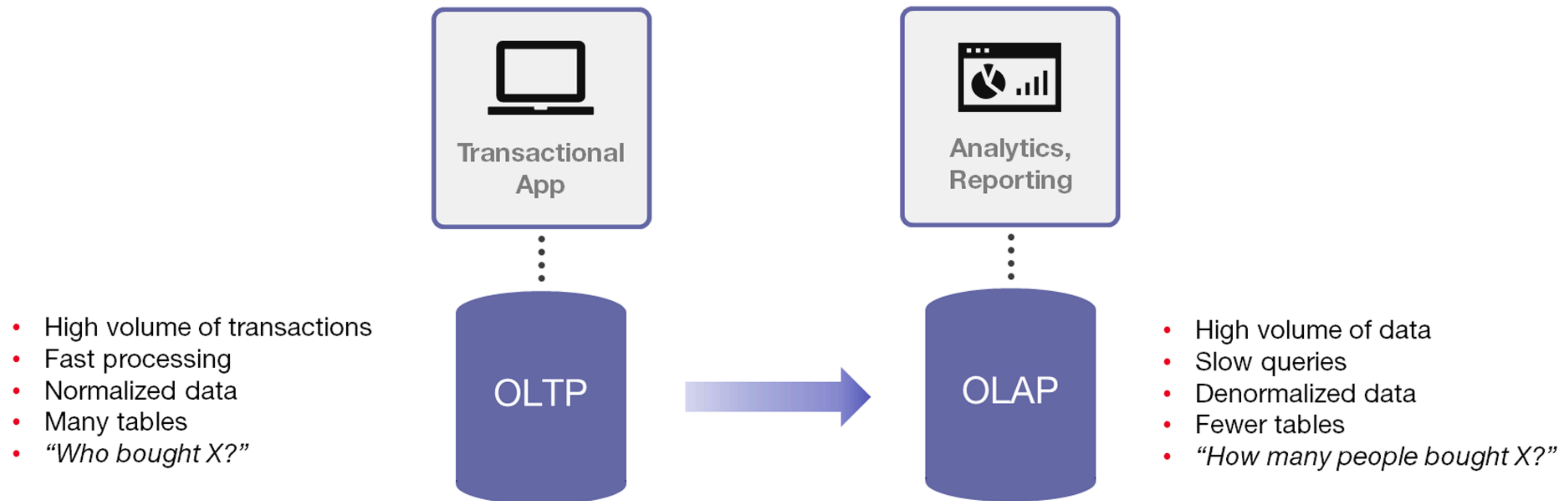


Процесс агрегации данных

Этот процесс нужен из-за разности детализации данных в OLTP и OLAP системах.

OLTP система может содержать несколько сумм для одного и того же набора элементов справочников.

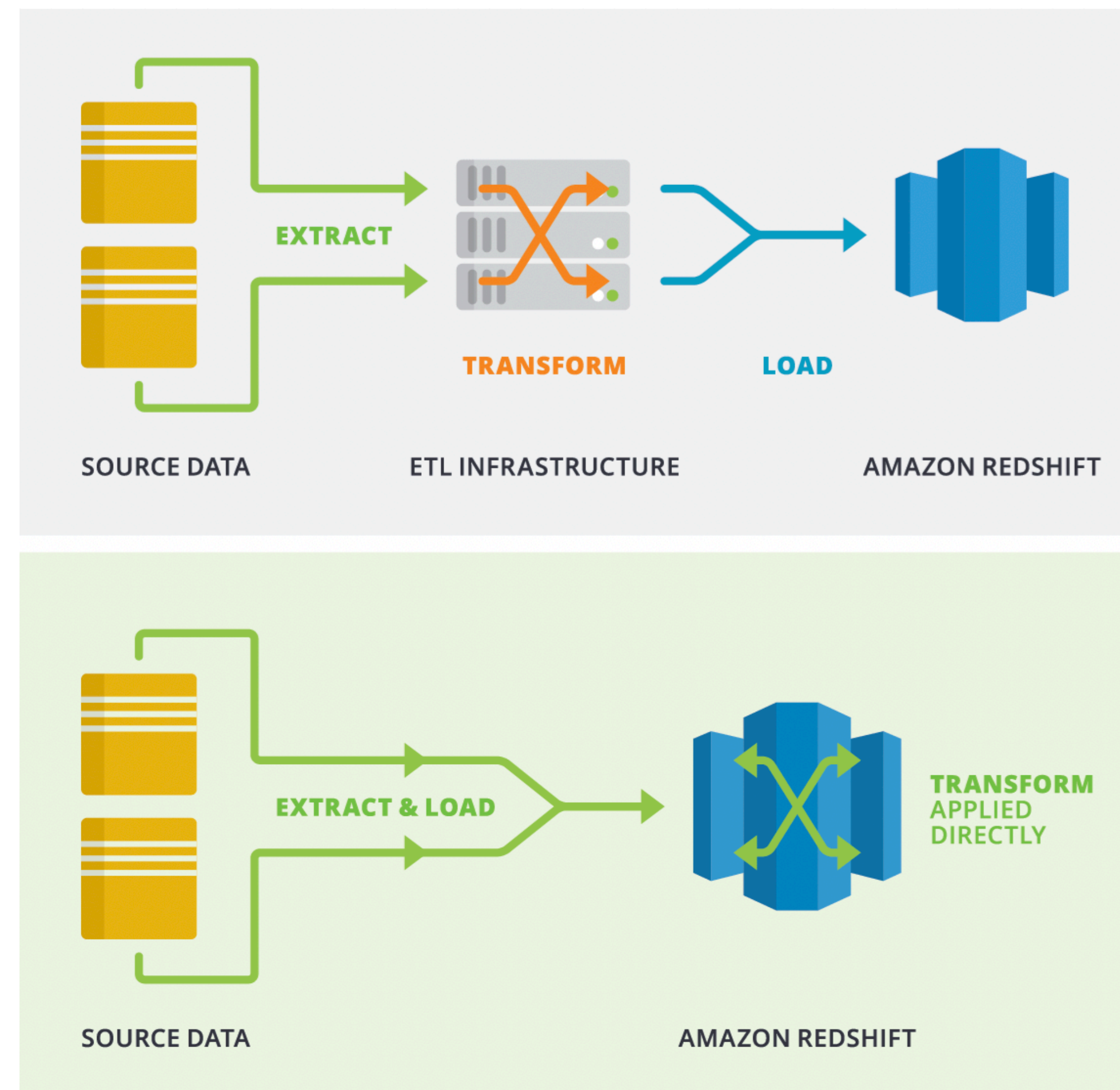
OLAP-системы — это, по сути, полностью денормализованная таблица фактов и окружающие ее таблицы справочников.



ETL vs ELT

В случае **ELT (Extract, Load, Transform)** данные сразу же загружаются после извлечения из исходных пулов данных.

- Промежуточная база данных отсутствует, что означает, что данные немедленно загружаются в единый централизованный репозиторий.
- Данные преобразуются в системе хранилища данных для использования с инструментами бизнес-аналитики и аналитики.



ВІ инструменты

Business Intelligence (BI) – это комплекс методов и компьютерных инструментов для сбора, хранения, обработки и визуализации данных, которые превращают транзакционную информацию в наглядные отчеты.

Главная цель ВІ – выделять ключевые метрики и тренды, помогать моделировать альтернативные сценарии и своевременно реагировать на изменения бизнеса.

ВІ-системы позволяют принимать:

- Оперативные решения
- Стратегические решения
- Объединять внутренние (финансовые, CRM) и внешние (рынок, конкуренты) данные



MOLAP (Multidimensional OLAP)

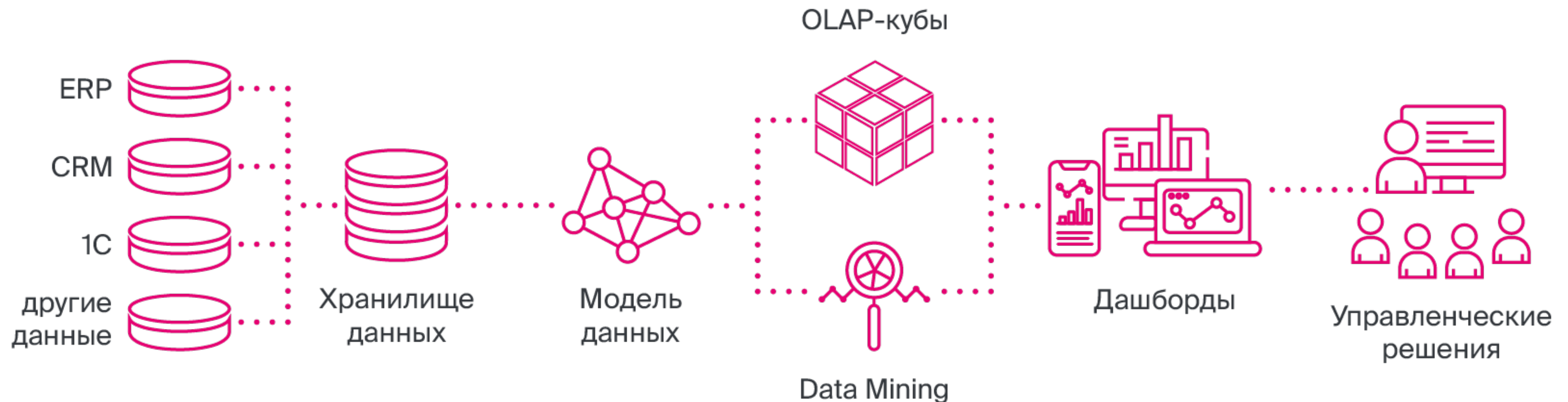
BI инструменты, которые перед построением отчетов собирают данные в особые структуры. Запросы к данным используют именно эти структуры.

Преимущества:

- Нет конкуренции с пользователями за ресурс СУБД
- Высокая скорость запросов

Недостатки:

- Только регламентное обновление данных
- Только заданные примитивы



ROLAP (Relational OLAP)

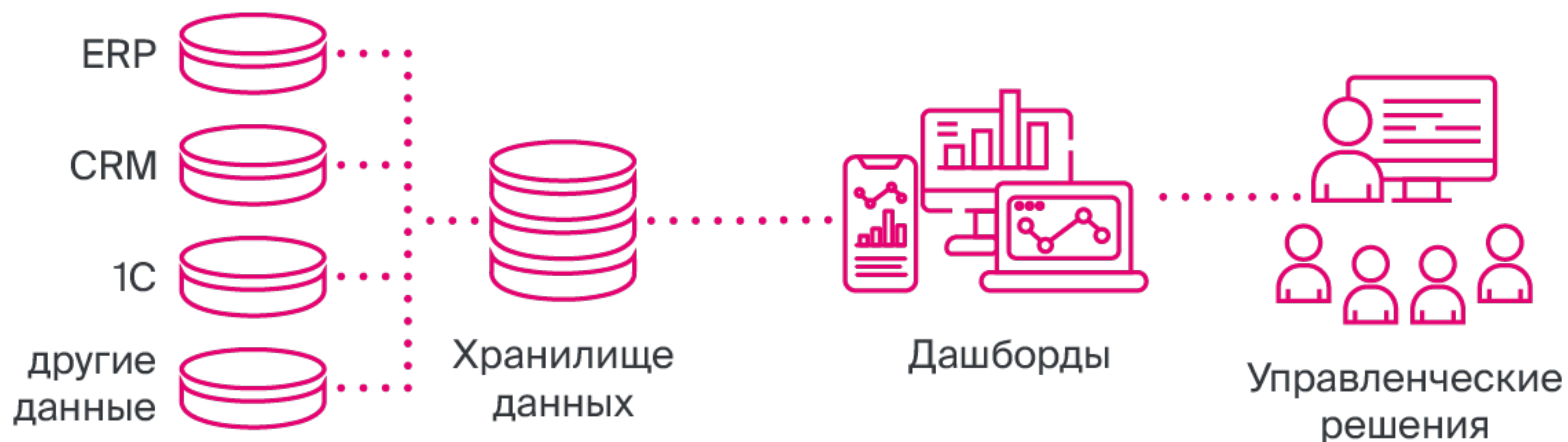
BI инструменты, которые отправляют запросы на получение данных напрямую в СУБД.

Преимущества:

- Построение real-time отчетов
- Реализация графиков любого уровня сложности

Недостатки:

- Конкуренция с пользователями
- При выполнении сложного запроса может нагружать СУБД



Классификация BI-инструментов

MOLAP подходит, если:

- Основные аналитические отчёты сформированы заранее и редко меняются
- Критична максимальная скорость отклика даже на сложные свёртки
- Объёмы (и число измерений) укладываются в возможность куба

Примеры систем:

- PowerBI

ROLAP подходит, если:

- Схема данных часто эволюционирует, добавляются новые поля или источники
- Нужно опираться на уже имеющийся DWH и минимизировать дублирование
- Допустима чуть более медленная, но гибкая аналитика

Примеры систем:

- SuperSet
- Grafana

Гибкие методологии проектирования хранилищ данных

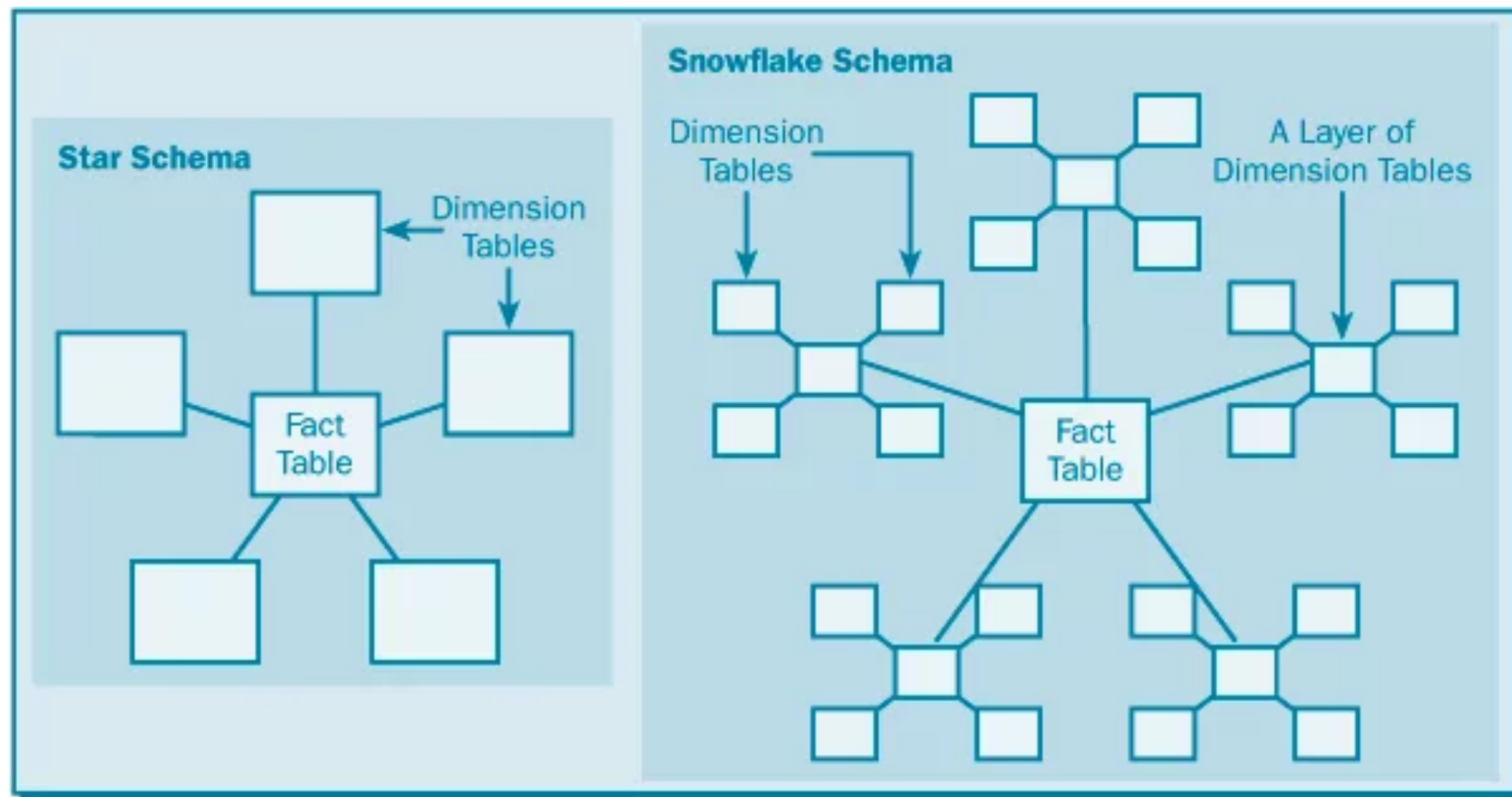
Модель Data Warehouse (DWH)

На текущий момент в качестве модели DWH может быть выбрана одна из следующих трех моделей хранения данных:

- Снежинка
- DataVault 2.0
- Якорная модель



Снежинка / Группа звездочек



Плюсы:

- Настройка проверок консистентности данных
- Структура соотносится с моделью данных

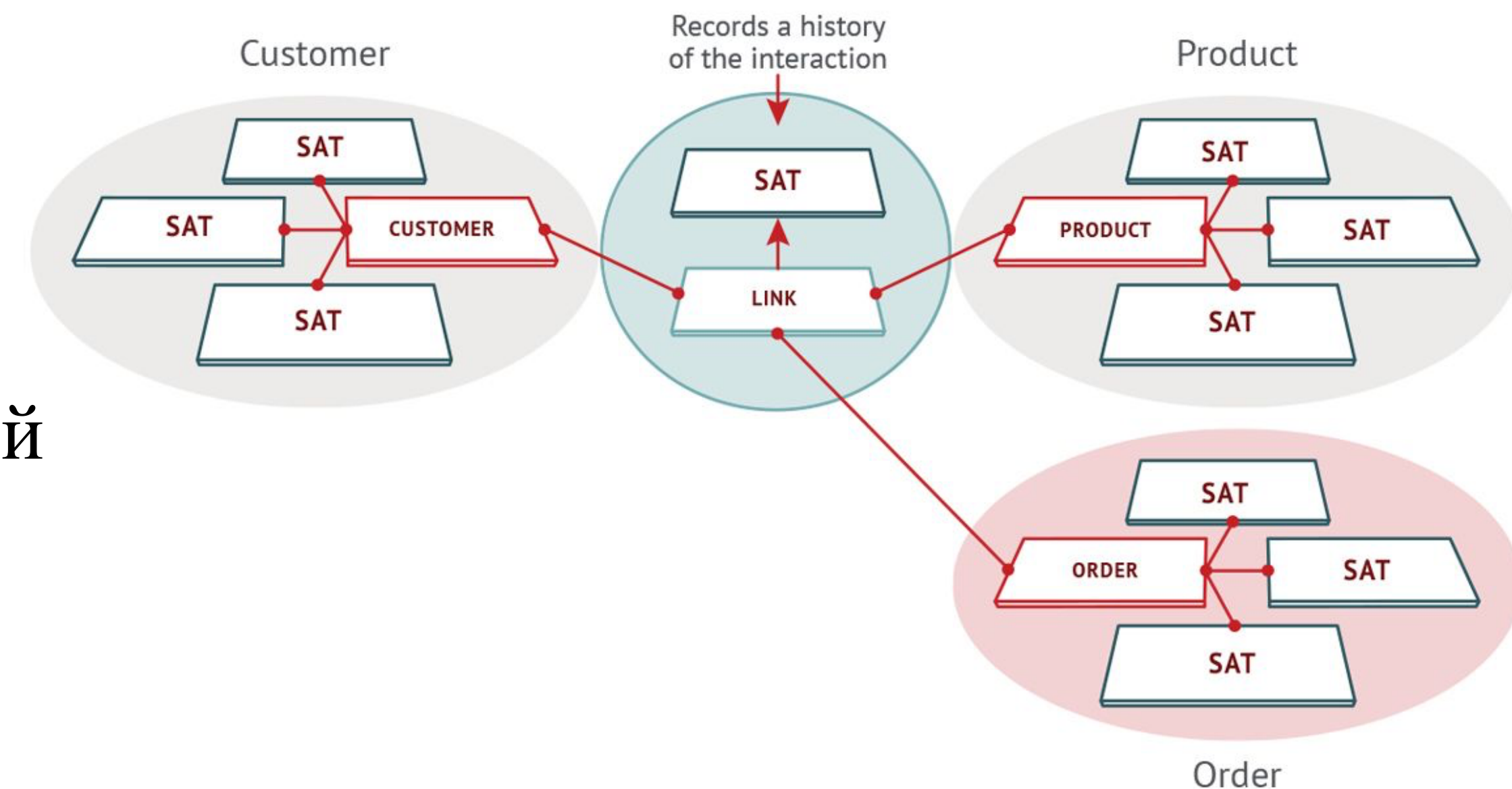
Минусы:

- Модель не подходит для крупных проектов или для проектов с быстро меняющейся структурой данных
- При внесении и добавлении данных в хранилище требуется множество согласований
- Снижается производительность запросов

Data Vault 2.0

Данные в Data Vault представлены в виде трех сущностей:

- **Хаб (hub)** – бизнес-сущность, например клиент, продукт, заказ. Хаб-таблица содержит несколько полей: натуральные ключи сущности, суррогатный ключ, ссылка на источник записи и время добавления записи.
- **Линк (связь, link)** – представление отношений между Хабами. Линки строятся в виде таблиц «многие ко многим» и содержат ссылки на суррогатные ключи связанных Хабов.
- **Сателлит (satellite)** - изменяемые атрибуты сущностей, контекстные данные для хаб-таблицы, связанные с Хабами и Связями по принципу «один ко многим».



Data Vault 2.0

Плюсы Data Vault:

- Универсальная структура, которая позволяет сформировать витрины под любые потребности бизнеса
- Первые верхнеуровневые отчеты доступны уже после загрузки Хабов и Связей
- Можно независимо разрабатывать смежные источники и легко заменять их, так как данные почти полностью отвязаны друг от друга

Минусы Data Vault:

- Большое количество таблиц и join'ов, которые замедляют выполнение запросов относительно денормализованных хранилищ
- Сложность выстраивания проверки данных из-за большого количества сущностей и сложных связей между ними
- Методология требует от специалиста дополнительных навыков и знаний особенности работы специфических функций, например хеширования

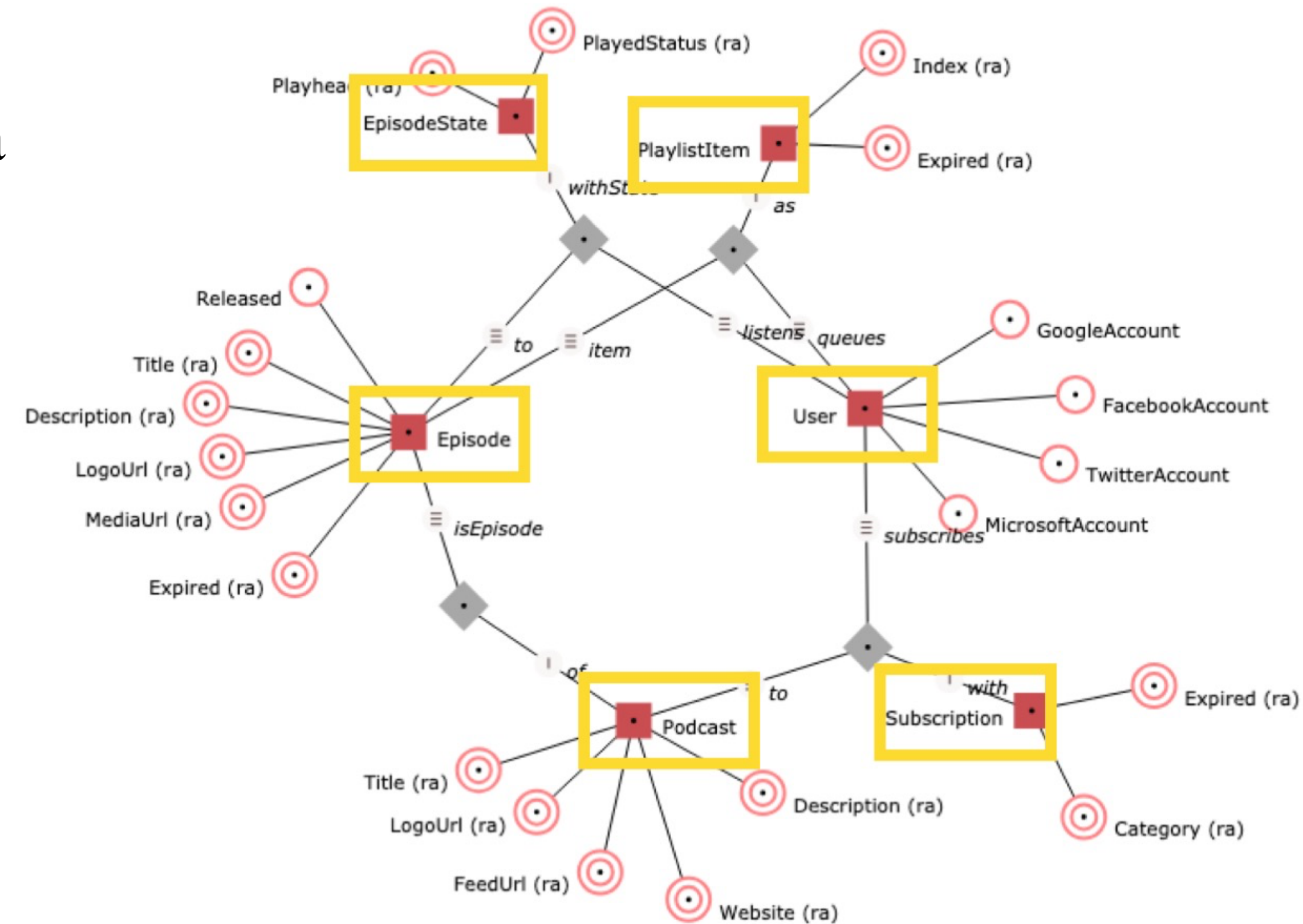
Data Vault 1.0 vs Data Vault 2.0

Аспект	Data Vault 1.0	Data Vault 2.0
Хэш-ключи	Хэш-ключи не были центральной концепцией, ограничивающей целостность и отслеживаемость данных.	Отдает приоритет хэш-ключам, обеспечивая целостность данных и улучшая отслеживаемость для повышения
Процедуры загрузки	Процедуры загрузки в Data Vault 1.0 могут быть сложными и часто включать порядковые номера, что влияет на эффективность.	Упрощает процедуры загрузки, повышает эффективность и устраняет необходимость в сложных порядковых номерах.
Зависимости	Имели значительные зависимости, потенциально замедляющие загрузку данных из-за последовательной обработки.	Уменьшает зависимости, обеспечивая более быструю обработку данных за счет распараллеливания.
Масштабируемость	Столкнулся с проблемами при работе с большими наборами данных из-за ограничений конструкции.	Эффективно обрабатывает большие данные, что делает его пригодным для сложных наборов данных.
Гибкость	Менее адаптируется к изменениям в источниках данных и бизнес-требованиях.	Гибкость и оперативность реагирования на изменения, идеально подходят для динамичных сред.

Якорная модель

Данные в якорной модели представлены в виде трех сущностей:

- **Якорь** – представляет собой сущность или событие, содержит суррогатные ключи, ссылку на источник и время добавления записи
- **Атрибут** – используется для моделирования свойств и характеристик якорей, содержит суррогатный ключ якоря, значение атрибута, ссылку на источник записи и время добавления записи
- **Связь** – моделирует отношения между якорями
- **Узел** – используется для моделирования общих свойств (состояния)



Якорная модель

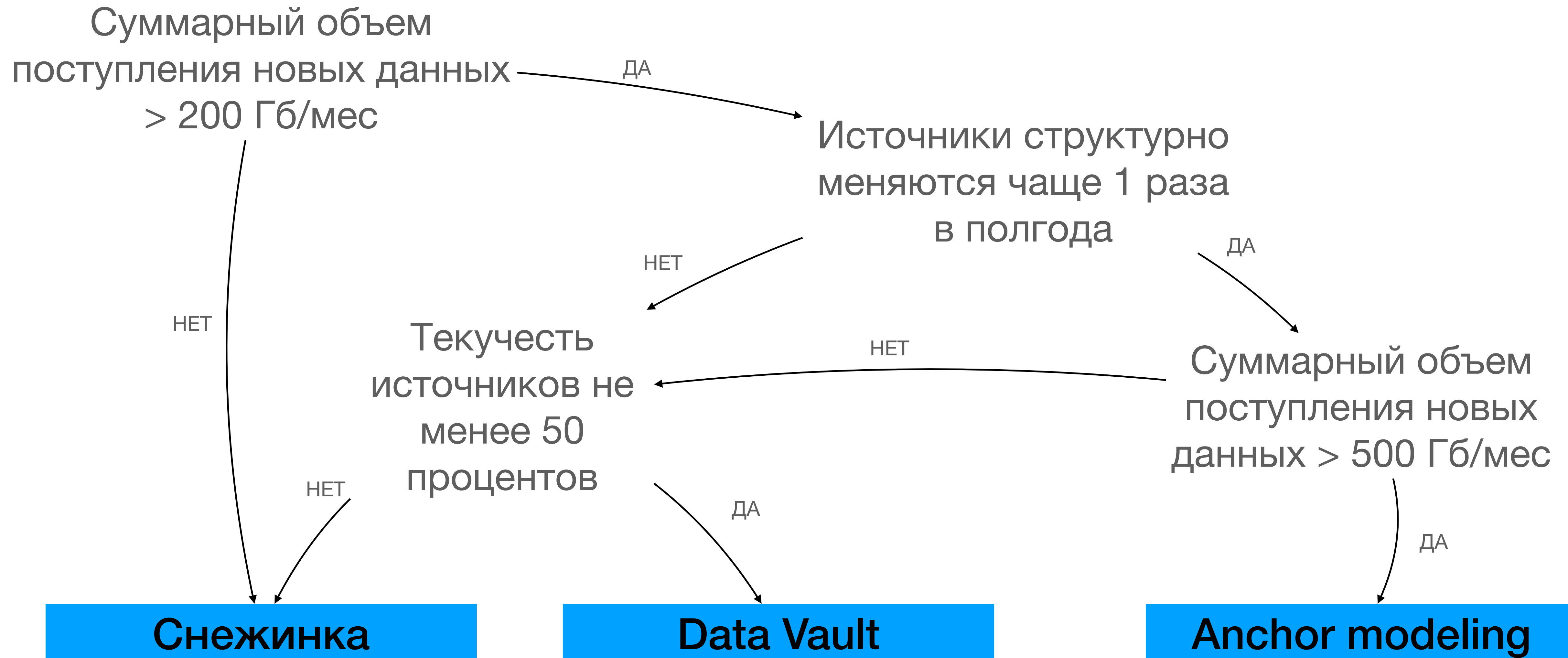
Плюсы Anchor Modeling:

- Нормализованная модель, которая эффективно обрабатывает изменения и позволяет масштабировать хранилища данных без отмены предыдущих действий
- Можно независимо разрабатывать смежные источники, так как данные почти полностью отвязаны друг от друга
- Значительная экономия места в связи с отсутствием нулевых значений (null) и дублирования

Минусы Anchor Modeling:

- Создает высокую нагрузку на базу даже для основных видов запросов
- Сложно настроить проверку данных, так как модель состоит из большого количества сущностей со сложными связями
- Большое количество таблиц и join'ов, повышающих риск ошибок
- Требуется постоянной поддержки и документирования, так как документация уникальна для каждого бизнеса. Специалистам необходимы дополнительные знания для понимания документации.

Как определить, какая модель подойдет лучше всего?



Data Lake

Data Lake — это хранилище данных, которое может хранить большое количество структурированных, полуструктурированных и неструктурированных данных. Это место для хранения всех типов данных в собственном формате без фиксированных ограничений на размер учетной записи или файл. Он предлагает большое количество данных для повышения аналитической производительности и встроенной интеграции.

Data Lake похожа на настоящее озеро и реки. Точно так же, как в озере есть несколько притоков, озеро данных содержит структурированные данные, неструктурированные данные, от машины к машине, журналы, проходящие в режиме реального времени.

Data Lake

СТРУКТУРИРОВАННЫЕ ДАННЫЕ

1. Данные в строках и столбцах
2. Легко сортируемые и обрабатываемые



Входящий поток представляет собой данные из баз данных, электронных таблиц, контента социальных медиа и т. п.



НЕСТРУКТУРИРОВАННЫЕ ДАННЫЕ

1. В «сыром», исходном виде
2. Письма, PDF
3. Изображения
4. Видео и аудио и т.п.

Data Lake

Озеро данных - это набор данных, в котором вы проводите аналитику по всем имеющимся данным в компании

Выходной поток – анализируемые данные



BI ПЛАТФОРМА

Благодаря «ОЗЕРУ» в BI вы можете быстро просеять нужные или ненужные данные и найти ключевой разрез или аналитику

Функции и динамическое программирование

Функции

По поведению:

- Функция является **детерминированной**, если при одном и том же заданном входном значении она всегда возвращает один и тот же результат.
- Функция является **недетерминированной**, если она может возвращать различные значения при одном и том же заданном входном значении.

Категории волатильности функций

Категория волатильности - это “обещание” оптимизатору относительно поведения функции:

- `VOLATILE` - может выполнять любые запросы, включая изменение базы данных.
- `STABLE` - не может изменять базу данных и гарантированно возвращает одни и те же результаты при одинаковых аргументах для всех строк в одном операторе.
- `IMMUTABLE` - не может изменять базу данных и гарантированно возвращает одни и те же результаты при одинаковых аргументах навсегда.

Типы возвращаемых значений

По типу возвращаемого значения:

- Скалярная функция;
- Подставляемая табличная функция;
- Многооператорная табличная функция.

Скалярная функция

Синтаксис:

CREATE [OR REPLACE] FUNCTION [имя-схемы.]
имя-функции ([список-объявлений-параметров])
RETURNS скалярный-тип-данных
AS \$\$
 тело-функции
\$\$ [;]

```
CREATE FUNCTION one()  
RETURNS integer AS  
begin  
    SELECT 1 AS result;  
end  
LANGUAGE SQL;
```

```
CREATE          FUNCTION      add_em(integer,  
integer)  
RETURNS integer AS  
$$  
    SELECT $1 + $2;  
$$  
LANGUAGE SQL;
```

```
CREATE FUNCTION getSalary(int)  
RETURNS int AS  
$$  
    SELECT    max(Salary)      AS    result  
FROM foo  
    WHERE fooid = $1;  
$$  
LANGUAGE SQL;
```

Подставляемая табличная функция

Синтаксис:

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [
  список-объявлений-параметров ] )
RETURNS SETOF <скалярный тип>
AS $$
[ выражение-выборки ]
$$ [ ; ]
```

```
CREATE FUNCTION list_children(text)
RETURNS SETOF text AS
$$
    SELECT name FROM nodes
    WHERE parent = $1
$$
LANGUAGE SQL;
```

```
CREATE FUNCTION get_foo(id int)
RETURNS foo AS
$$
    RETURN QUERY
    SELECT * FROM foo
    WHERE fooid = id ;
$$ LANGUAGE SQL;
```

Многооператорная функция

Синтаксис:

CREATE FUNCTION [имя-схемы.] имя-функции (
 [список-объявлений-параметров])
 RETURNS TABLE (определение-таблицы)
 AS \$\$
 [SQL- запрос]
 \$\$ [;]

```
create table tab (id int, name text);

insert into tab values(1, 'aaa');
insert into tab values(2, 'bbb');
insert into tab values(3, 'ccc');
insert into tab values(4, 'ddd');
insert into tab values(5, 'eee');
insert into tab values(6, 'fff');
```

```
create or replace function
sum_n_product_with_tab(x int, y int)
returns table(sum int, product int)
AS
$$
begin
    create temp table if not exists
test_calc as
    select id, name
    from tab
    where id > $2;

    return query
    select tab.id, $1 * tab.id
    from test_calc tab;

end
$$ language plpgsql;
```


Управляющие структуры

Использование RETURN
(В функциях,
возвращающих set of
rows):

- NEXT выражение;
- QUERY query;
- QUERY EXECUTE
строка-команды;

```
CREATE OR REPLACE FUNCTION get_all_foo()  
RETURNS SETOF foo AS  
$BODY$  
DECLARE  
    r foo%rowtype;  
BEGIN  
    FOR r IN SELECT * FROM foo  
              WHERE fooid > 0  
    LOOP  
        -- здесь возможна обработка данных  
        -- возвращается текущая строка запроса  
        RETURN NEXT r;  
    END LOOP;  
    RETURN;  
END;  
$BODY$  
LANGUAGE plpgsql;  
  
SELECT * FROM get_all_foo();
```

Метаданные (information_schema)

Информационная схема (information_schema) состоит из набора представлений, содержащих информацию об объектах, определённых в текущей базе данных.

Информационная схема описана в стандарте SQL и поэтому можно рассчитывать на её переносимость и стабильность — в отличие от системных каталогов, которые привязаны к PostgreSQL, и моделируются, отталкиваясь от реализации.

- information_schema.schemata
- information_schema.tables
- information_schema.columns
- information_schema.routines
- information_schema.triggers
- information_schema.views
- information_schema.
table_constraints
- information_schema.
check_constraints

Триггеры и курсоры

Триггеры

Триггер - это хранимая процедура особого типа, которая выполняет одну или несколько инструкций в ответ на событие.

По типу события триггеры делятся на два класса:

- DDL-триггеры
- DML-триггеры

DDL-триггеры

```
CREATE EVENT TRIGGER имя
ON событие
[ WHEN переменная_фильтра
  IN (значение_фильтра
    [, ... ] )
  [ AND ... ] ]
EXECUTE PROCEDURE
имя_функции ( )
```

```
CREATE OR REPLACE FUNCTION
abort_any_command()
RETURNS event_trigger
LANGUAGE plpgsql AS
$$
    BEGIN RAISE EXCEPTION 'команда %
      отключена', tg_tag;
END;
$;
```

```
CREATE EVENT TRIGGER abort_ddl
ON ddl_command_start

EXECUTE PROCEDURE abort_any_command();
```

События (event) DDL-триггера

- LOGIN
- DDL_COMMAND_START и DDL_COMMAND_END
- CREATE
- ALTER
- DROP
- COMMENT
- GRANT
- REINDEX
- REFRESH MATERIALIZED VIEW
- REVOKE
- SQL_DROP
- TABLE_REWRITE

```
CREATE EVENT TRIGGER init_session
ON login
EXECUTE FUNCTION init_session();
```

```
CREATE OR REPLACE FUNCTION init_session()
RETURNS event_trigger
LANGUAGE plpgsql AS
$$
DECLARE
    hour integer = EXTRACT('hour' FROM
                                current_time at
                                time zone 'utc');
BEGIN
    IF hour BETWEEN 2 AND 4 THEN
        RAISE EXCEPTION 'Login forbidden';
    END IF;
END;
$$
```


DML-триггеры

Для поддержания согласованности и точности данных используются декларативные и процедурные методы.

Триггеры применяются в следующих случаях:

- если использование методов декларативной целостности данных не отвечает функциональным потребностям приложения;
- если необходимо каскадное изменение через связанные таблицы в базе данных;
- если база данных денормализована и требуется способ автоматизированного обновления избыточных данных в нескольких таблицах;
- если необходимо сверить значение в одной таблице с неидентичным значением в другой таблице;
- если требуется вывод пользовательских сообщений и сложная обработка ошибок.

Классы DML-триггеров

Существуют два класса триггеров:

- **INSTEAD OF**. Триггеры этого класса выполняются в обход действий, вызывавших их срабатывание, заменяя эти действия. Например, обновление таблицы, в которой есть триггер **INSTEAD OF**, вызовет срабатывание этого триггера. В результате вместо оператора обновления выполняется код триггера.
- **AFTER/BEFORE**. Триггеры этого класса исполняются после или до

DML-триггеры

Синтаксис:

```
CREATE [ OR REPLACE ] CONSTRAINT ]
TRIGGER name { BEFORE | AFTER | INSTEAD OF } { event [ OR ... ] }
ON table_name [ FROM referenced_table_name ]
[ NOT DEFERRABLE |
[ DEFERRABLE ]
[ INITIALLY IMMEDIATE | INITIALLY DEFERRED ]
[ REFERRENCING { { OLD | NEW } TABLE [ AS ] transition_relation_name } [
... ] ] [
FOR [ EACH ] { ROW | STATEMENT } ]
[ WHEN ( condition ) ]
EXECUTE { FUNCTION | PROCEDURE } function_name ( arguments )
```


DML-триггеры

```
CREATE TRIGGER check_upd
BEFORE UPDATE
ON accounts
FOR EACH ROW
EXECUTE FUNCTION check_account_update();
```

```
CREATE OR REPLACE TRIGGER check_upd
BEFORE UPDATE OF balance
ON accounts
FOR EACH ROW
EXECUTE FUNCTION check_account_update();
```

```
CREATE TRIGGER log_update
AFTER UPDATE
ON accounts
FOR EACH ROW
WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION log_account_update();
```

```
CREATE OR REPLACE FUNCTION
employee_insert_trigger_fnc()
RETURNS trigger AS
$$
    INSERT INTO "Employee_Audit" (
        "EmployeeId",
        "LastName",
        "FirstName",
        "UserName",
        "EmpAdditionTime")
    VALUES (NEW."EmployeeId",
        NEW."LastName",
        NEW."FirstName",
        current_user,
        current_date);
    RETURN NEW;
$$ LANGUAGE 'plpgsql';
```

```
CREATE TRIGGER employee_insert_trigger
AFTER INSERT ON "Employee"
FOR EACH ROW
EXECUTE PROCEDURE
employee_insert_trigger_fnc();
```

Alter для триггера

```
ALTER EVENT TRIGGER name DISABLE
ALTER EVENT TRIGGER name ENABLE [ REPLICA | ALWAYS ]
ALTER EVENT TRIGGER name OWNER TO
{ new_owner |
  CURRENT_ROLE |
  CURRENT_USER |
  SESSION_USER
}
ALTER EVENT TRIGGER name RENAME TO new_name
```

Курсоры

Операции в реляционной базе данных выполняются над множеством строк. Набор строк, возвращаемый инструкцией `SELECT`, содержит все строки, которые удовлетворяют условиям, указанным в предложении `WHERE`.

Курсоры можно классифицировать:

- По области видимости;
- По типу;
- По способу перемещения по курсору;
- По способу распараллеливания курсора

По области видимости

По области видимости имени курсора различают:

- **Локальные курсоры (LOCAL).** Область курсора локальна по отношению к пакету, хранимой процедуре или триггеру, в которых этот курсор был создан. Курсор неявно освобождается после завершения выполнения пакета, хранимой процедуры или триггера, за исключением случая, когда курсор был передан параметру OUTPUT.
- **Глобальные курсоры (GLOBAL).** Область курсора является глобальной по отношению к соединению.

По типу

По типу курсора различают:

- **Статические курсоры (STATIC).** Создается временная копия данных для использования курсором. Все запросы к курсору обращаются к указанной временной таблице, сам курсор не позволяет производить изменения.
- **Динамические курсоры (DYNAMIC).** Отображают все изменения данных, сделанные в строках результирующего набора при просмотре этого курсора. Значения данных, порядок, а также членство строк в каждой выборке могут меняться.
- **Курсоры, управляемые набором ключей (KEYSET).** Членство или порядок строк в курсоре не изменяются после его открытия. Набор ключей, однозначно определяющих строки, встроен в таблицу в базе данных.
- **Быстрые последовательные курсоры (FAST_FORWARD)**

По способу перемещения по курсору

- Последовательные курсоры (FORWARD_ONLY). Курсор может просматриваться только от первой строки к последней. Поддерживается только параметр выборки FETCH NEXT.
- Курсоры прокрутки (SCROLL). Перемещение осуществляется по группе записей как вперед, так и назад. Параметр SCROLL не может указываться вместе с параметром для FAST_FORWARD.^[L]_[SEP]

По способу распараллеливания курсора

По способу распараллеливания курсоров различают:

- **READ_ONLY**. Содержимое курсора можно только считывать.
- **SCROLL_LOCKS**. При редактировании данной записи вами никто другой вносить в нее изменения не может. Такую блокировку прокрутки иногда еще называют «пессимистической» блокировкой.
- **OPTIMISTIC**. Означает отсутствие каких бы то ни было блокировок. «Оптимистическая» блокировка предполагает, что даже во время выполнения вами редактирования данных, другие пользователи смогут к ним обращаться.

Объявление и открытие курсора

Синтаксис:

DECLARE

name [[NO] SCROLL]
CURSOR [(arguments)]
FOR query;

OPEN unbound_cursorvar [[NO] SCROLL]
FOR query;

DECLARE

cur1 refcursor;

cur2 **CURSOR FOR** SELECT *
FROM tenk1;

cur3 **CURSOR** (key integer) **FOR**
SELECT *
FROM tenk1
WHERE unique1 = key;

OPEN cur1 **FOR EXECUTE**

format('SELECT * FROM %I
WHERE col1 = \$1', tabname)

USING keyvalue;

Чтение данных из курсора

FETCH [direction { FROM | IN }] cursor
INTO target;

FETCH извлекает следующую строку (в указанном направлении) из курсора в целевую переменную.

Если подходящей строки нет, в целевую переменную записывается значение NULL(s). Как и в случае с SELECT INTO, можно проверить специальную переменную FOUND, чтобы узнать, была ли получена строка.

Если строка не получена, курсор перемещается в положение после последней строки или перед первой строкой, в зависимости от направления движения.

```
FETCH curs1 INTO rowvar;
FETCH curs2 INTO foo, bar, baz;
FETCH LAST FROM curs3 INTO x, y;
FETCH RELATIVE -2 FROM curs4 INTO x;

MOVE curs1;
MOVE LAST FROM curs3;
MOVE RELATIVE -2 FROM curs4;
MOVE FORWARD 2 FROM curs4;

UPDATE foo
SET dataval = myval
WHERE CURRENT OF curs1;

CLOSE curs1;
```


Направление обработки данных курсора

Предложение `direction` может быть любым из вариантов, допустимых в команде `SQL FETCH`, за исключением тех, которые позволяют получить более одной строки.

- `NEXT`
- `PRIOR`
- `FIRST`
- `LAST`
- `ABSOLUTE count`
- `RELATIVE count`
- `FORWARD`
- `BACKWARD`.

Отсутствие `direction` эквивалентно указанию `NEXT`.

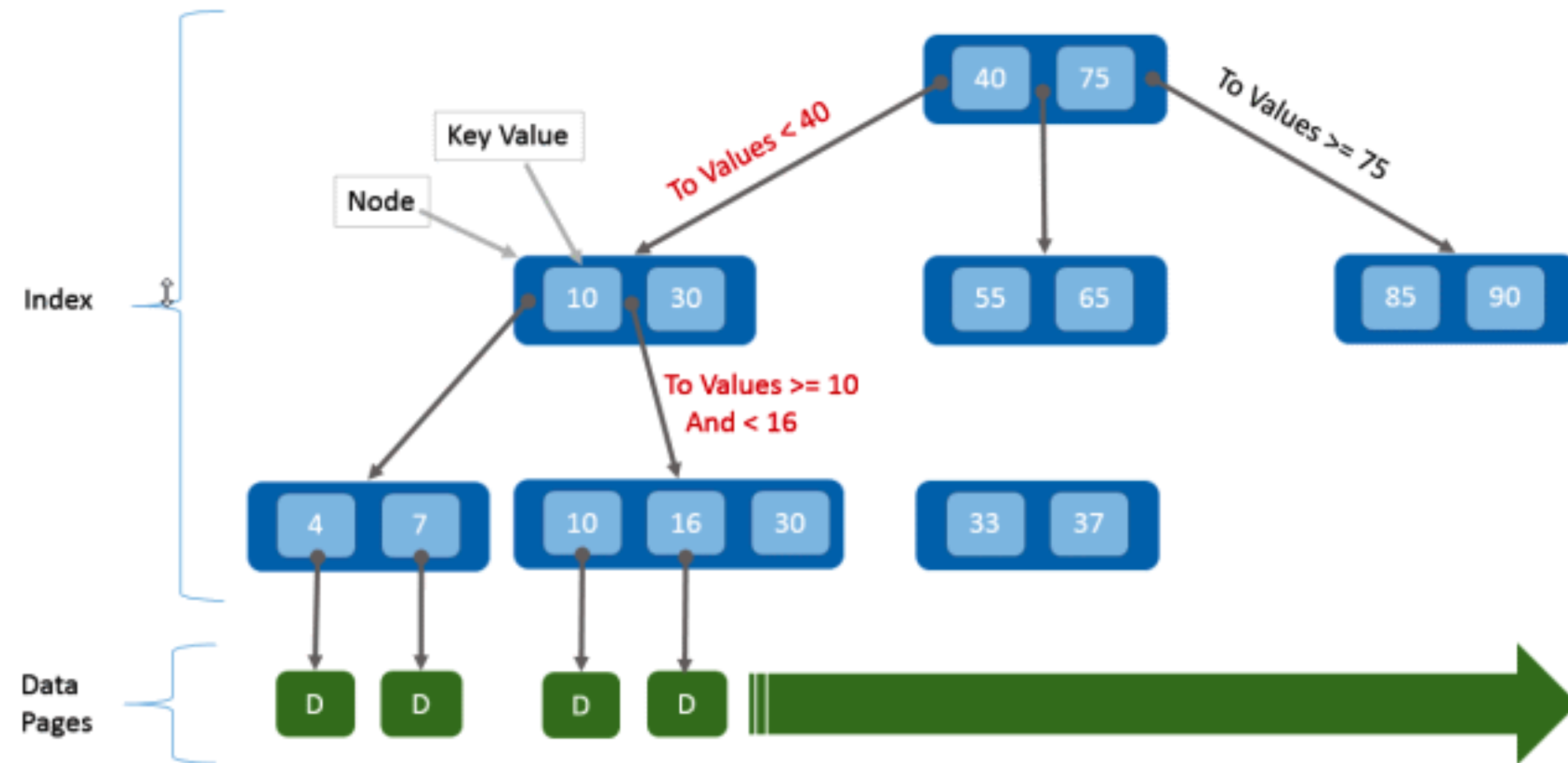
```
CREATE FUNCTION myfunc(refcursor, refcursor)
RETURNS SETOF refcursor AS $$
BEGIN
    OPEN $1 FOR SELECT * FROM table_1;
    RETURN NEXT $1;
    OPEN $2 FOR SELECT * FROM table_2;
    RETURN NEXT $2;
END;
$$ LANGUAGE plpgsql;

BEGIN;
    SELECT * FROM myfunc('a', 'b');
    FETCH ALL FROM a;
    FETCH ALL FROM b;
COMMIT;
```

**Индексы, партиционирование
и сегментирование.**

Индексы

B-Tree Layout



Индекс – это объект базы данных, обеспечивающий дополнительные способы быстрого поиска и извлечения данных.

Плюсы:

- При наличии индекса время поиска нужных строк можно существенно уменьшить.

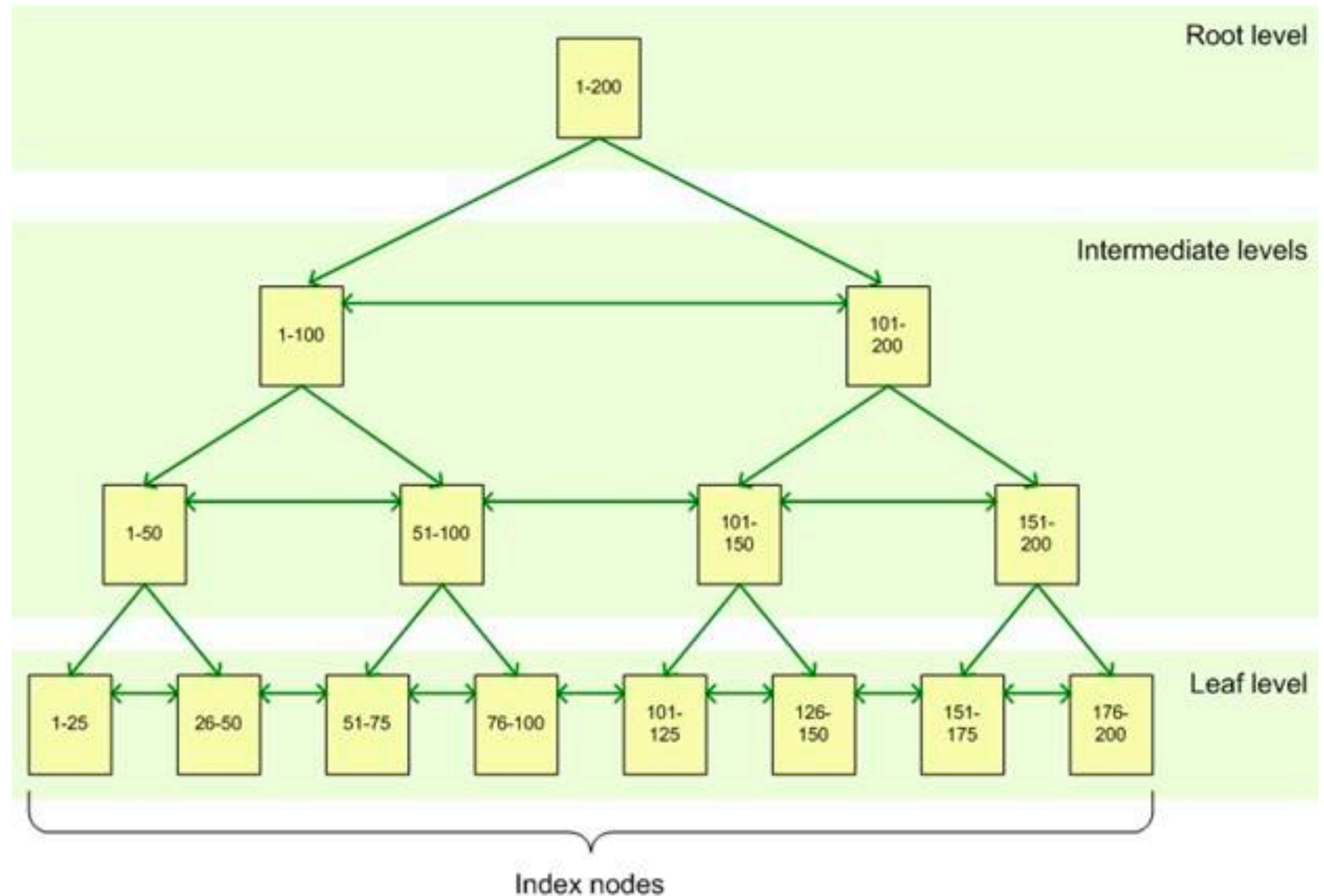
Минусы:

- Дополнительное место на диске и в оперативной памяти,
- Замедляются операции вставки, обновления и удаления записей.

Классификация индексов

Существует два типа индексов:

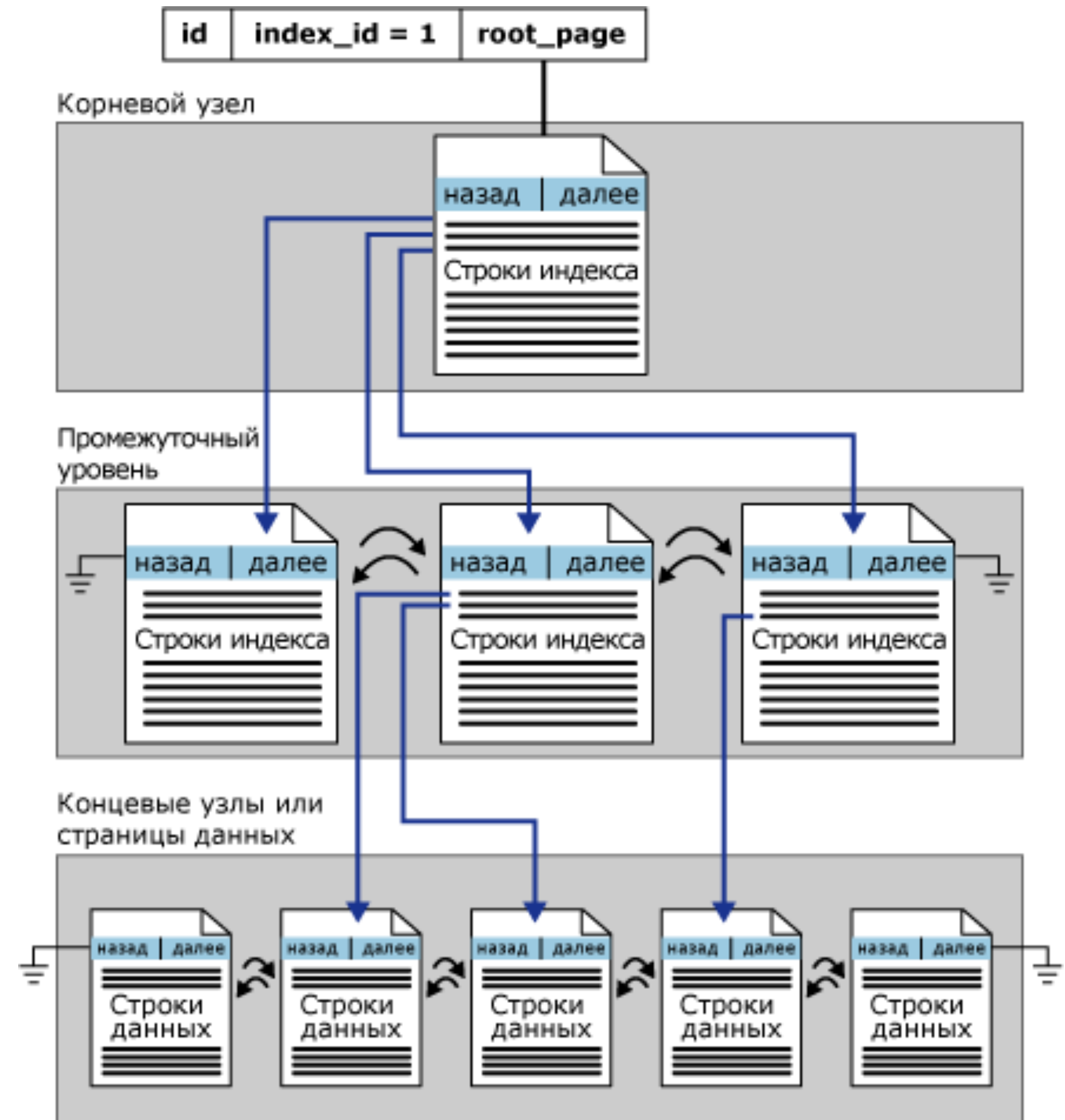
- Кластерные индексы;
- Некластерные индексы, которые включают:
 - на основе кучи;
 - на основе кластерных индексов.



Кластерные индексы

В кластерном индексе таблица представляет собой часть индекса, или индекс представляет собой часть таблицы.

- Листовой узел кластерного индекса – это страница таблицы с данными.
- Поскольку сами данные таблицы являются частью индекса, то очевидно, что для таблицы может быть создан только один кластерный индекс.
- Кластерный индекс является уникальным индексом по определению. Это означает, что все ключи записей должны быть уникальные.



Пример создания индекса на основе В-дерева

Подведем итог:

- В-tree (сбалансированное дерево) — это самый распространенный тип индекса в PostgreSQL.
- Он поддерживает все стандартные операции сравнения ($>$, $<$, $>=$, $<=$, $=$, $<>$) и может использоваться с большинством типов данных.
- В-tree индексы могут быть использованы для сортировки, ограничений уникальности и поиска по диапазону значений.

Описание инструкции:

```
CREATE INDEX ИМЯ  
ON таблица  
[USING тип индекса] (столбец);
```

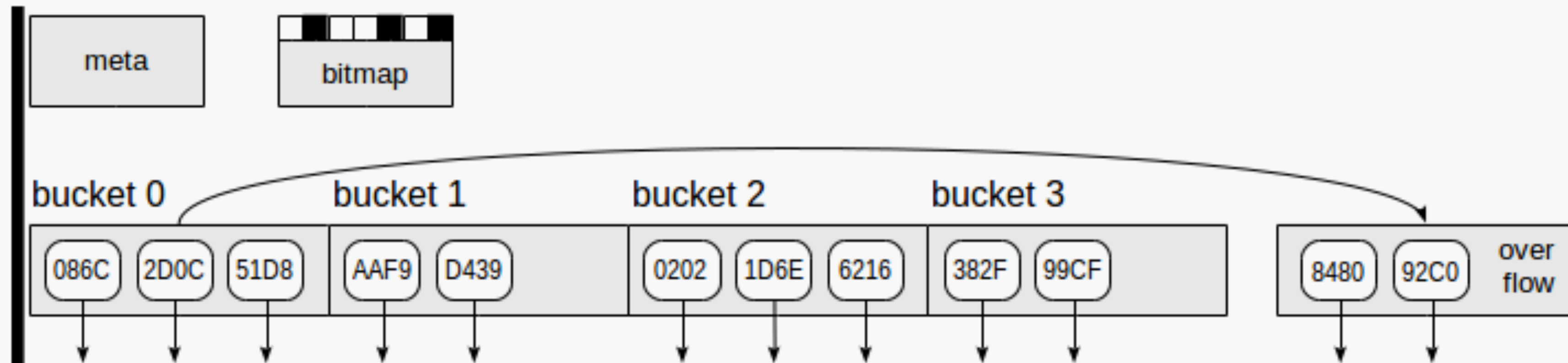
Пример создания индекса:

```
CREATE INDEX ix_example_btree  
ON example_table (column_name);
```


Какие еще бывают индексы

- **Хеш-индексы** хранят 32-битный хеш-код, полученный из значения индексируемого столбца, поэтому хеш-индексы работают только с простыми условиями равенства. Планировщик запросов может применить хеш-индекс, только если индексируемый столбец участвует в сравнении с оператором.
- **GIN-индексы** представляют собой «инвертированные индексы», в которых могут содержаться значения с несколькими ключами, например массивы. Инвертированный индекс содержит отдельный элемент для значения каждого компонента, и может эффективно работать в запросах, проверяющих присутствие определённых значений компонентов.
- **GiST-индексы** представляют собой не просто разновидность индексов, а инфраструктуру, позволяющую реализовать много разных стратегий индексирования. Как следствие, GiST-индексы могут применяться с разными операторами, в зависимости от стратегии индексирования (*класса операторов*).
- **BRIN-индексы** (сокращение от Block Range INdexas, Индексы зон блоков) хранят обобщённые сведения о значениях, находящихся в физически последовательно расположенных блоках таблицы. Поэтому такие индексы наиболее эффективны для столбцов, значения в которых хорошо коррелируют с физическим порядком столбцов таблицы.

Хеш-индексы



Хеш-индекс использует четыре типа страниц (серые прямоугольники):

- Meta page — это нулевая страница, содержащая информацию о том, что находится внутри индекса.
- Bucket pages — это основные страницы индекса, на которых хранятся данные в виде пар «хэш-код — TID».
- Overflow pages — структурированы так же, как страницы корзины, и используются, когда одной страницы корзины недостаточно.
- Bitmap pages — это страницы с переполнением, которые в данный момент свободны и могут быть повторно использованы для других сегментов.

Стрелки вниз, начинающиеся у элементов индексной страницы, обозначают TID, то есть ссылки на строки таблицы.

GIN-индексы

Основная область применения метода GIN — ускорение полнотекстового поиска, поэтому его целесообразно использовать в качестве примера при более подробном обсуждении этого индекса.

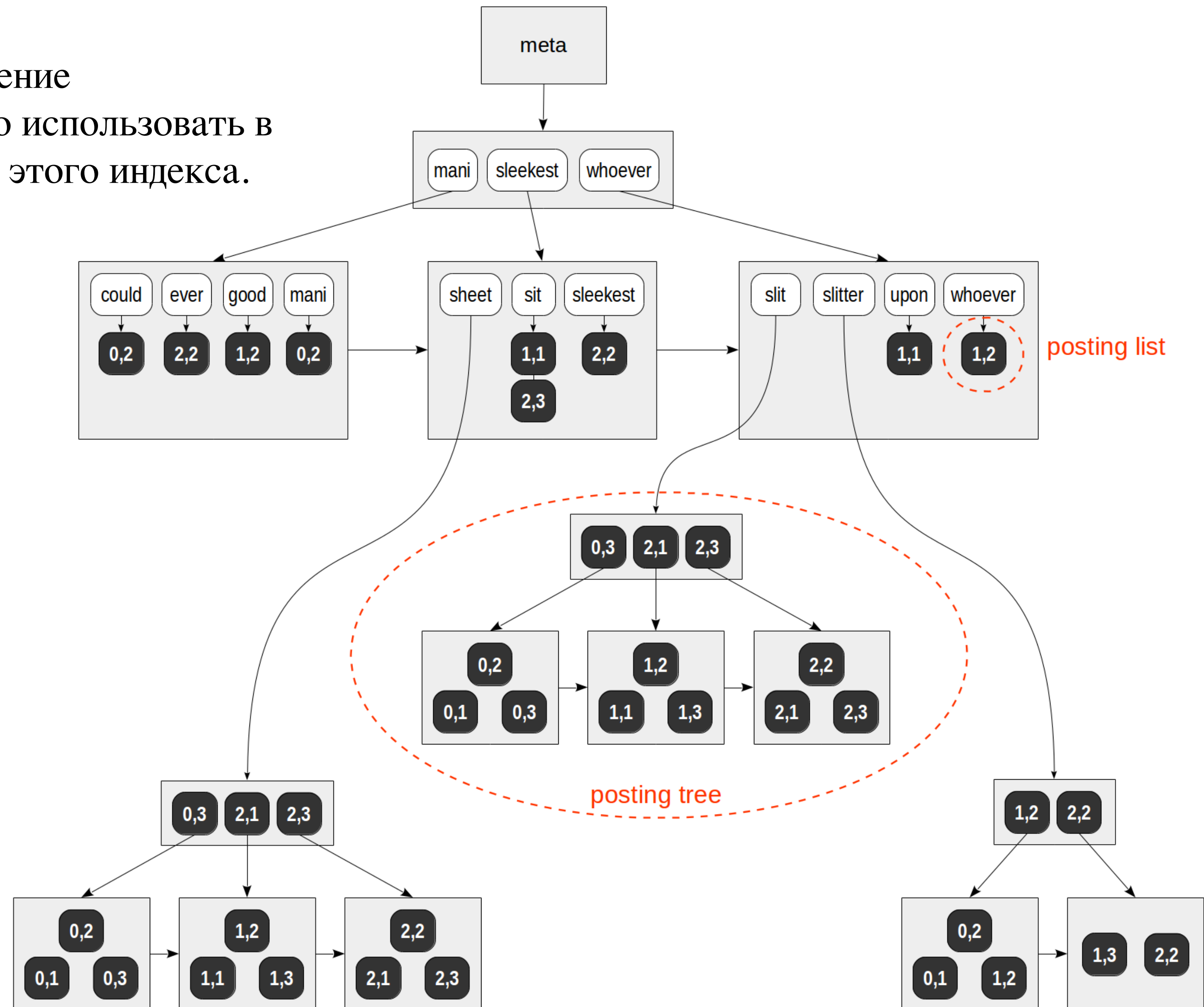
```
create table ts(doc text, doc_tsv tsvector);
```

```
insert into ts(doc) values
```

```
('Can a sheet slitter slit sheets?'),  
( 'How many sheets could a sheet slitter slit?'),  
( 'I slit a sheet, a sheet I slit.'),  
( 'Upon a slitted sheet I sit.'),  
( 'Whoever slit the sheets is a good sheet slitter.'),  
( 'I am a sheet slitter.'),  
( 'I slit sheets.'),  
( 'I am the sleekest sheet slitter that ever slit sheets.'),  
( 'She slits the sheet she sits on.');
```

```
update ts set doc_tsv = to_tsvector(doc);
```

```
create index test_index on ts using gin(doc_tsv);
```

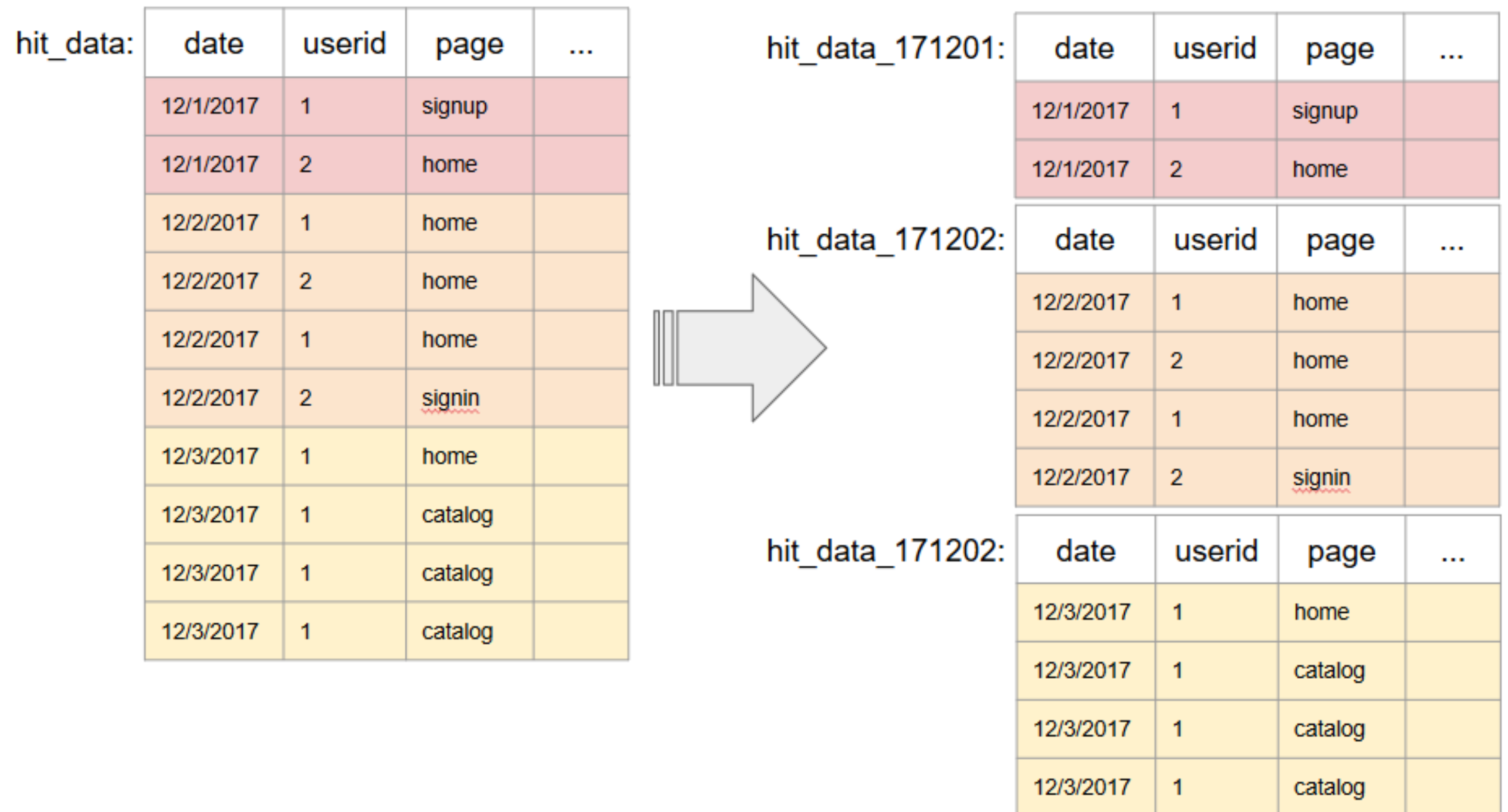


Партиционирование

Партиционирование – это метод разделения больших (исходя из количества записей, а не столбцов) таблиц на много маленьких. И желательно, чтобы это происходило прозрачным для приложения способом.

```
CREATE TABLE measurement (  
  city_id      int not null,  
  logdate     date not null,  
  peaktemp    int,  
  unitsales    int  
) PARTITION BY RANGE (logdate);
```

Partitioning Tables By Date



Партиции в диапазоне

```
CREATE TABLE sales (  
    id int,  
    date date,  
    amt decimal(10,2)  
)  
PARTITION BY RANGE (date)  
( START (date '2022-01-01') INCLUSIVE  
  END (date '2023-01-01') EXCLUSIVE  
  EVERY (INTERVAL '1 day'));
```

```
CREATE TABLE nrank (  
    id int,  
    rank int,  
    year int,  
    color char(1),  
    count int  
)  
PARTITION BY RANGE (year)  
( START (2012) END (2022) EVERY (1),  
  DEFAULT PARTITION extra);
```

```
CREATE TABLE sales (  
    id int,  
    date date,  
    amt decimal(10,2)  
)  
PARTITION BY RANGE (date)  
( PARTITION Jan22 START (date '2022-01-01') INCLUSIVE ,  
  PARTITION Feb22 START (date '2022-02-01') INCLUSIVE ,  
  PARTITION Mar22 START (date '2022-03-01') INCLUSIVE ,  
  PARTITION Apr22 START (date '2022-04-01') INCLUSIVE ,  
  PARTITION May22 START (date '2022-05-01') INCLUSIVE ,  
  PARTITION Jun22 START (date '2022-06-01') INCLUSIVE ,  
  PARTITION Jul22 START (date '2022-07-01') INCLUSIVE ,  
  PARTITION Aug22 START (date '2022-08-01') INCLUSIVE ,  
  PARTITION Sep22 START (date '2022-09-01') INCLUSIVE ,  
  PARTITION Oct22 START (date '2022-10-01') INCLUSIVE ,  
  PARTITION Nov22 START (date '2022-11-01') INCLUSIVE ,  
  PARTITION Dec22 START (date '2022-12-01') INCLUSIVE  
    END (date '2023-01-01') EXCLUSIVE);
```

Партиции по списку

```
CREATE TABLE rank (  
    id int,  
    rank int,  
    year int,  
    color char(1),  
    count int  
)  
PARTITION BY LIST (color)  
( PARTITION red VALUES ('r'),  
  PARTITION blue VALUES ('b'),  
  PARTITION green VALUES ('g'),  
  DEFAULT PARTITION other );
```

Таблица, партиционированная по списку, может использовать столбец любого типа данных, который допускает сравнения на равенство.

При классическом синтаксисе партиция по списку может иметь составной ключ раздела.

- Для разделов списка необходимо объявить раздел для каждого значения списка;
- При отсутствии дефолтной партиции невозможно добавление в таблицу данных, для которых не подходит ни одна партиция.

Многоуровневые партиции

В примере таблица sales партиционируется по year, затем по month, затем по region.

Условия SUBPARTITION TEMPLATE гарантируют, что каждый годовой раздел будет иметь одинаковую структуру подразделов.

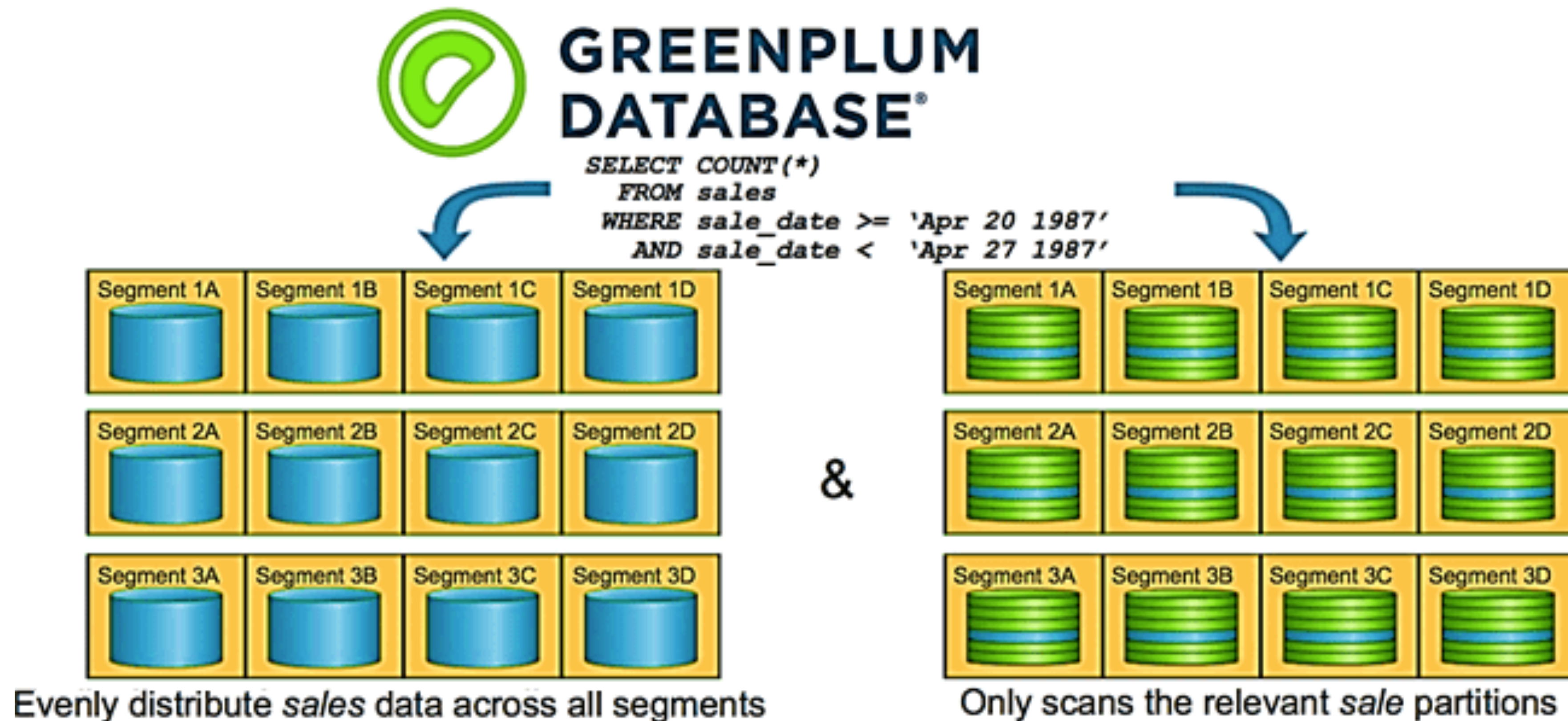
В примере на каждом уровне иерархии обязательно объявляется раздел DEFAULT

При создании многоуровневых разделов на основе диапазонов легко создать большое количество подразделов, некоторые из которых будут содержать мало данных или не будут содержать их вовсе. Это может привести к увеличению количества записей в системных таблицах, что, в свою очередь, увеличит время и объём памяти, необходимые для оптимизации и выполнения запросов.

```
CREATE TABLE p3_sales (  
    id int,  
    year int,  
    month int,  
    day int,  
    region text  
)  
PARTITION BY RANGE (year)  
    SUBPARTITION BY RANGE (month)  
        SUBPARTITION TEMPLATE (  
            START (1) END (13) EVERY (1),  
            DEFAULT SUBPARTITION other_months )  
        SUBPARTITION BY LIST (region)  
            SUBPARTITION TEMPLATE (  
                SUBPARTITION usa VALUES ('usa'),  
                SUBPARTITION europe VALUES ('europe'),  
                SUBPARTITION asia VALUES ('asia'),  
                DEFAULT SUBPARTITION other_regions )  
( START (2002) END (2012) EVERY (1),  
    DEFAULT PARTITION outlying_years );
```

Сегментация (дистрибуция)

Для распределения данных по кластерам используется их сегментация (Segmentation). Задача разработчика — подобрать такой список полей и/или такую функцию (например, хэш-функцию), благодаря которым данные равномерно распределятся по нодам кластера.



Виды сегментации

DISTRIBUTED BY

Чтобы корректно распределить данные по первичным сегментам, для выбранных полей формируется хэш-значения, на основе которых и будет проходить распределение.

```
Create table t1 (  
    col1 integer,  
    col2 varchar(100)  
) with (appendonly=false)  
distributed by (col1);
```

DISTRIBUTED REPLICATED

Зачастую используется для небольших таблиц (к примеру, словарей). Содержимое таблицы дублируется на каждый сегмент.

```
Create table t1 (  
    col1 integer,  
    col2 varchar(100)  
) with (appendonly=false)  
distributed replicated;
```

DISTRIBUTED RANDOMLY

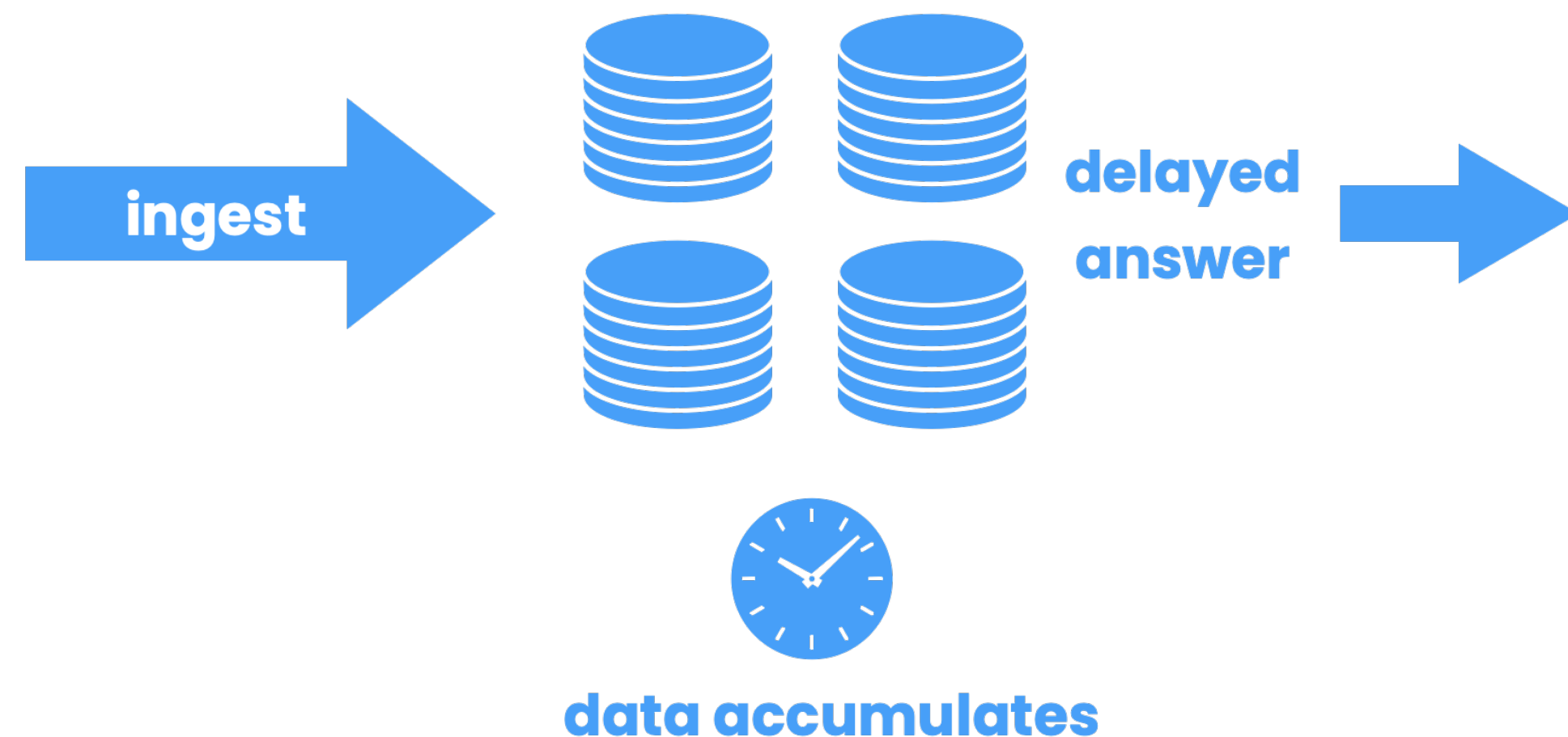
Это – приблизительно равномерное распределение данных между сегментами. Данное распределение используется, если не подходит ни одно из двух предыдущих.

```
Create table t1 (  
    col1 integer,  
    col2 varchar(100)  
) with (appendonly=false)  
distributed randomly;
```

ETL-инструменты

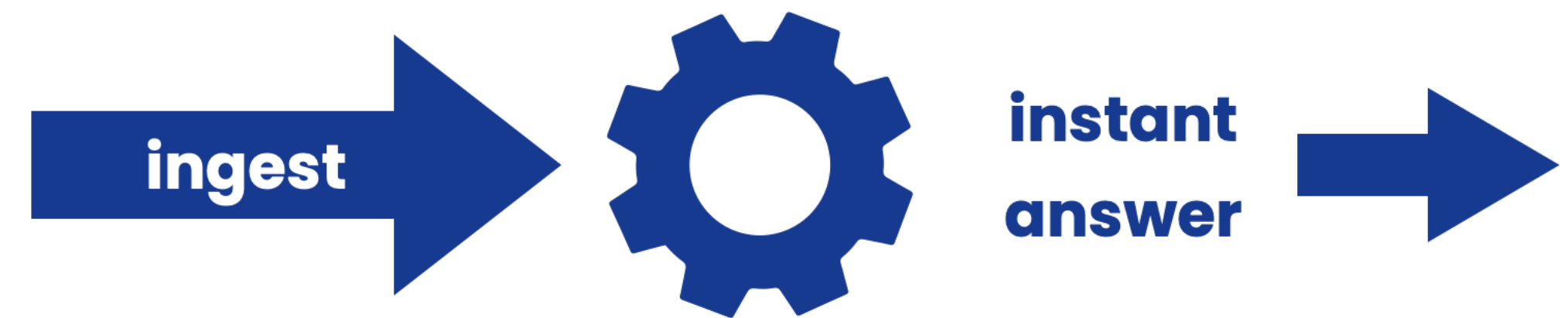
Подходы к обработке данных

Batch Processing



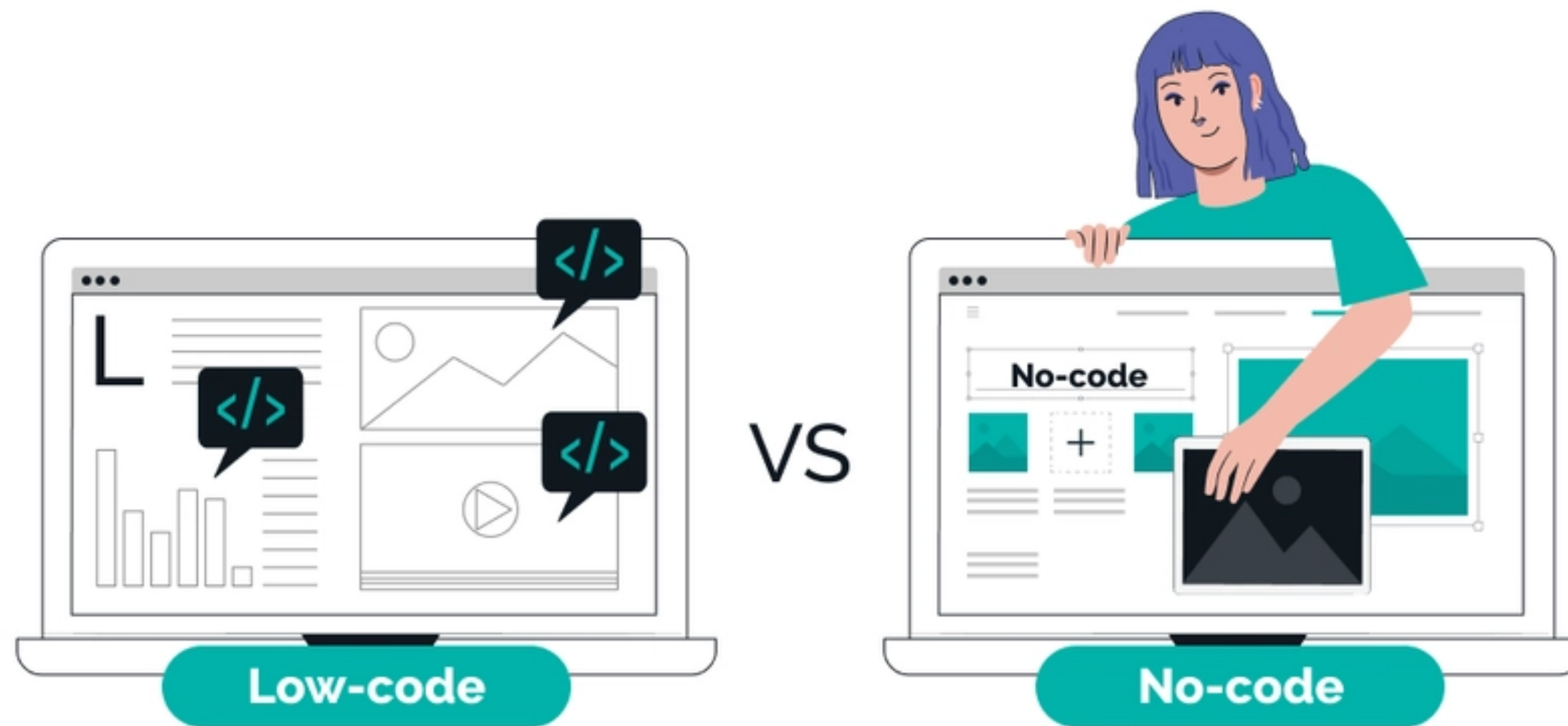
- Большие порции данных
- Загрузка по регламенту, а следовательно и отставание в данных на принимающей стороне

Stream Processing



- Маленькие порции данных (часто в формате изменения)
- Загрузка приближенная к реальному времени

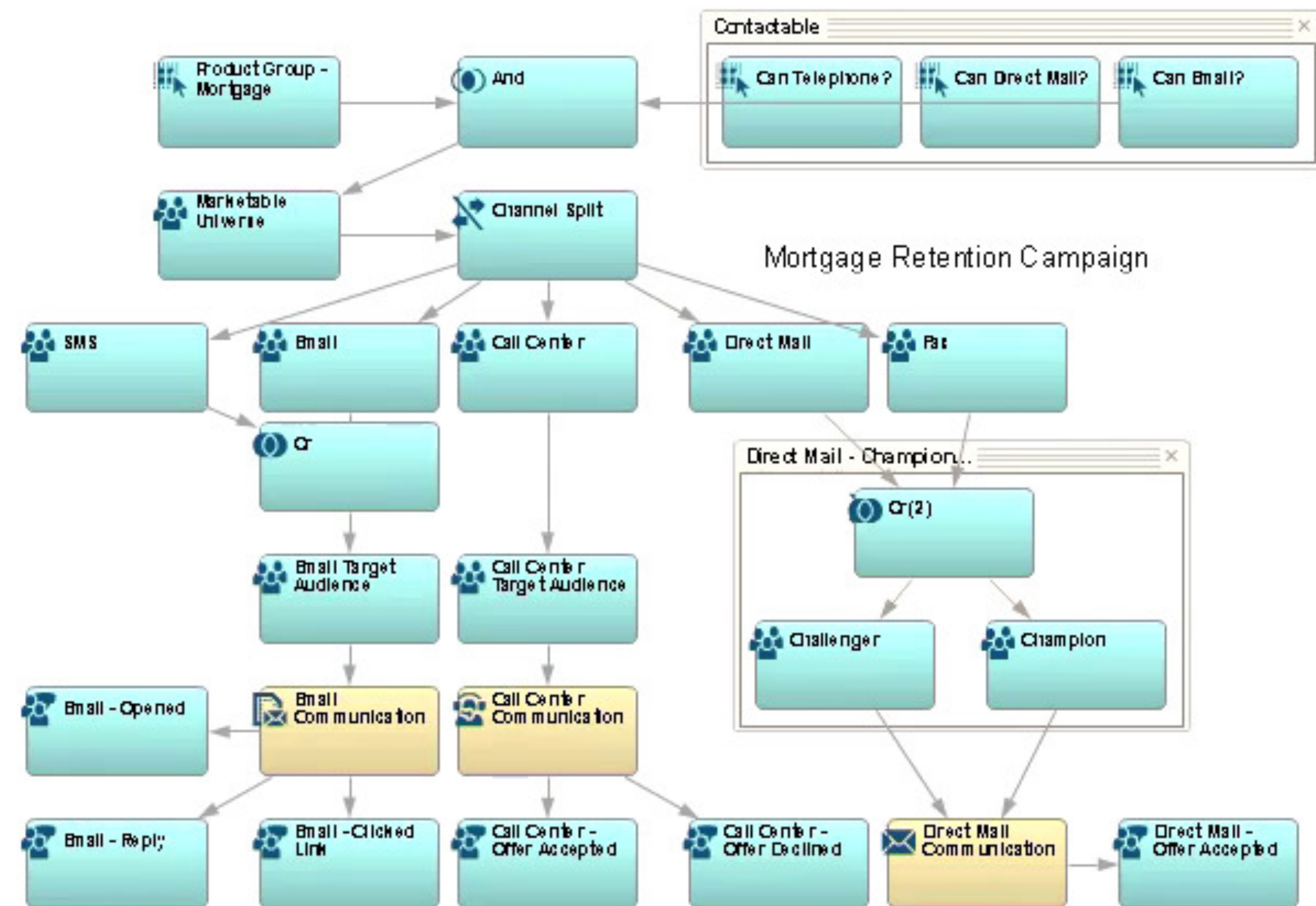
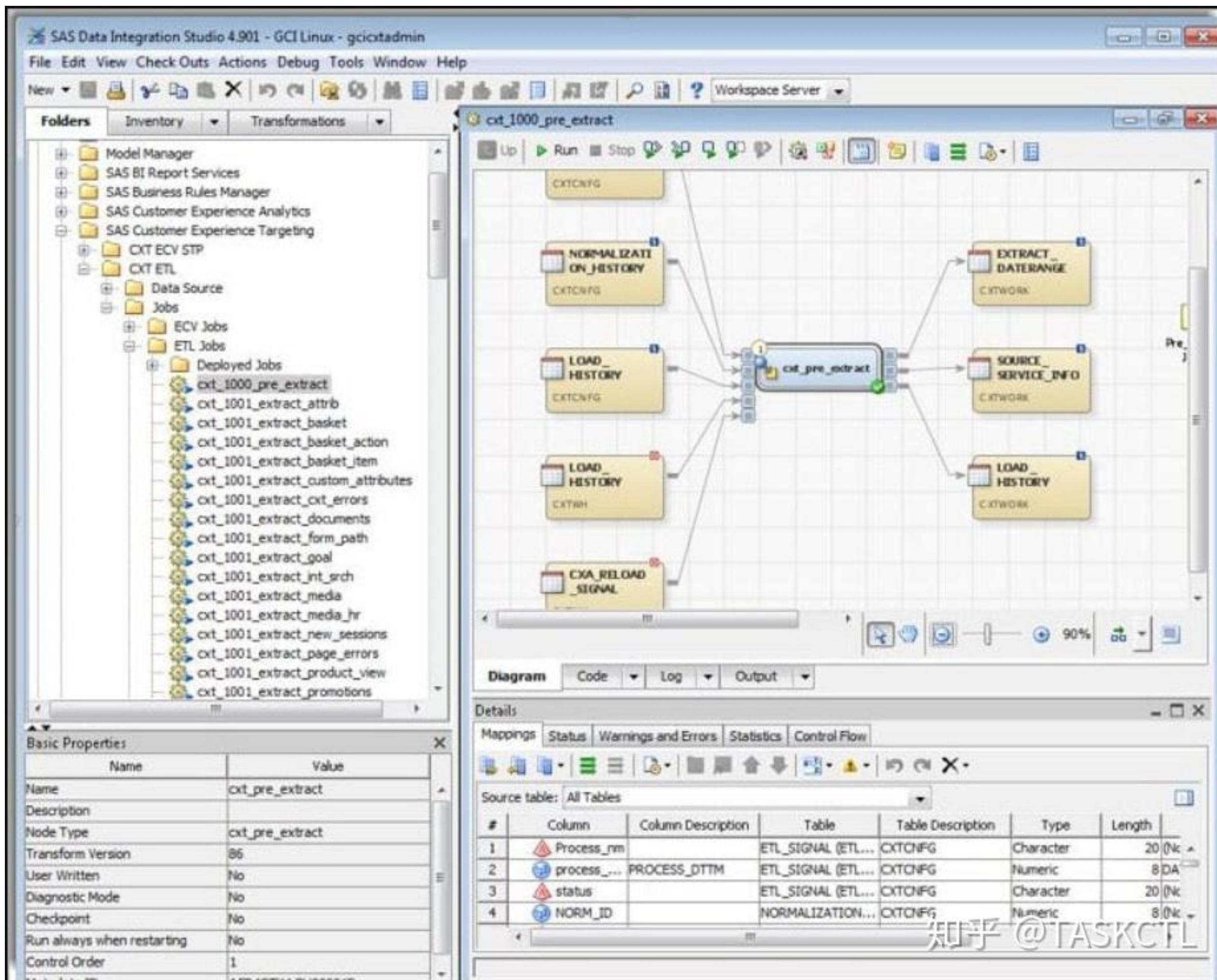
No Code & Low Code



- Используются визуальные инструменты и готовые модули
- Минимальное количество кода
- Для пользователей с минимальными навыками программирования

- Используются визуальные инструменты и готовые модули
- Нет возможности писать код
- Для пользователей без навыков программирования

Low Code приложения для автоматизации загрузки данных



NiFi для потоков обработки сообщений

Apache NiFi предоставляет возможность управления потоками данных из разнообразных источников в режиме реального времени с использованием графического интерфейса.

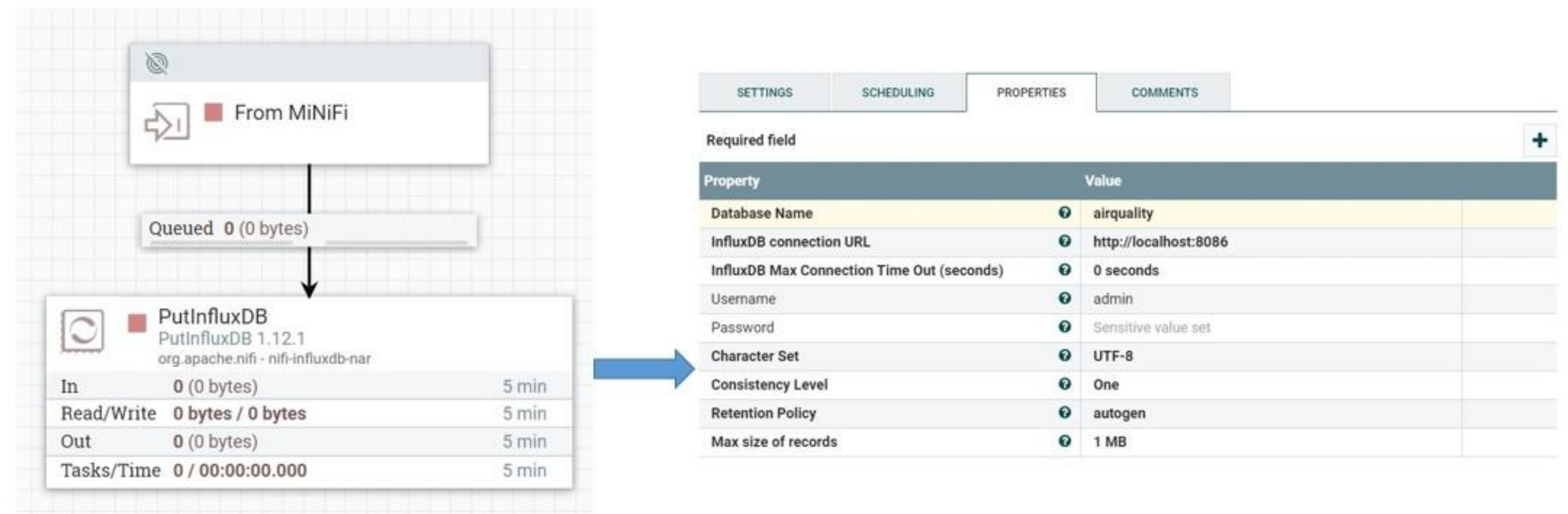
Интересные факты:

- NiFi базируется на технологии NiagaraFiles, ранее разрабатываемой агентством национальной безопасности США
- В ноябре 2014 года исходный код был открыт и передан Apache Software Foundation в рамках программы по передаче технологий NSA Technology Transfer Program.

Компоненты NiFi

NiFi состоит из трех основных компонентов:

- Flow Files – потоки файлов.
- Flow File Processor – своего рода «функции» с входами и выходами параметрами, которые выполняют широкий перечень типовых задач.
- Connections – задают направление движения FlowFiles между процессорами.



Flow Files – потоки файлов

Поточный файл является основным объектом обработки в Apache NiFi.

Он состоит из двух частей:

- Атрибуты, используемые процессорами NiFi для обработки данных, например:
 - UUID – идентификатор потокового файла
 - FileName – имя потокового файла
 - File Size – размер потокового файлы
 - mime.type - MIME-тип потокового файла
- Контейнер с данными (payload), которые обрабатываются процессорами.

FlowFile

DETAILS

ATTRIBUTES

FlowFile Details

UUID
8ebb619b-d1dd-461e-8b34-f0a8de7a8ba9

Filename
New Text Document.txt

File Size
0 bytes

Queue Position
No value set

Queued Duration
00:00:29.433

Lineage Duration
00:00:29.574

Penalized
No

Content Claim

Container
default

Section
1

Identifier
1541399489168-1

Offset
0

Size
0 bytes

 DOWNLOAD

 VIEW

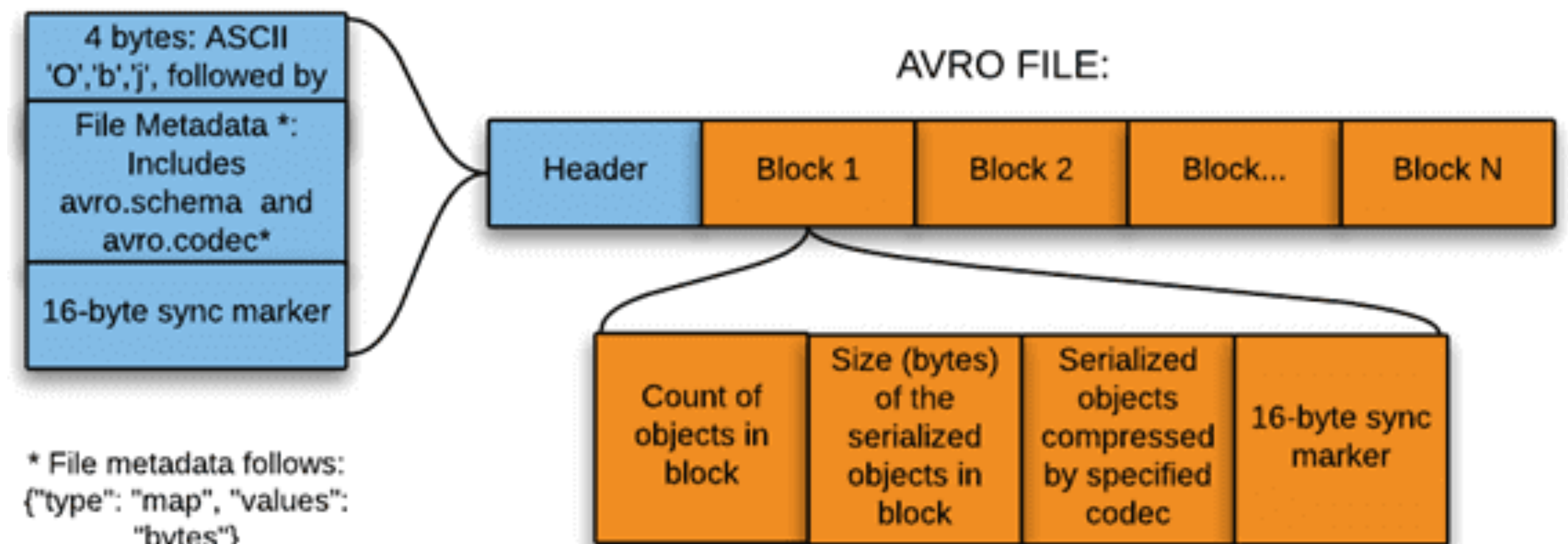
OK

Flow Files. Формат AVRO

AVRO - строчный формат хранения файлов, активно применяемый в экосистеме Apache Hadoop.

Широко используется в качестве платформы сериализации.

Avro сохраняет схему в независимом от реализации текстовом формате JSON, что облегчает ее чтение и интерпретацию.



Flow Files. Структура AVRO

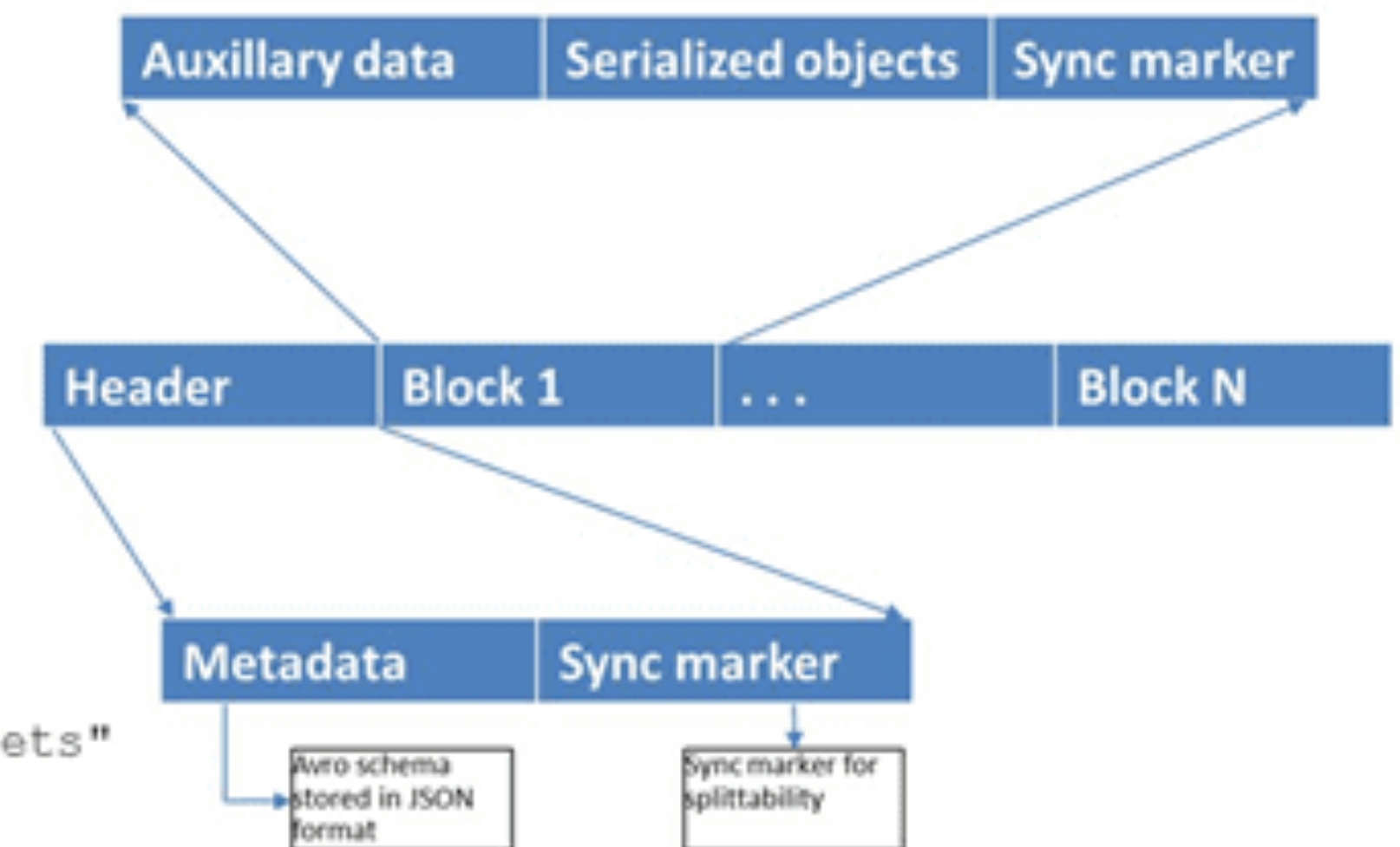
Линейная структура данных формата Avro обеспечивает следующие преимущества:

- высокая скорость записи информации, что особенно важно в потоковых Big Data системах
- формат отлично подходит для ETL-хранилищ и витрин данных, где требуется чтение всех полей записи, а не избирательно по столбцам, как в колоночно-ориентированных форматах.

Sample AVRO schema in JSON format

```
{
  "type" : "record",
  "name" : "tweets",
  "fields" : [ {
    "name" : "username",
    "type" : "string",
  }, {
    "name" : "tweet",
    "type" : "string",
  }, {
    "name" : "timestamp",
    "type" : "long",
  } ],
  "doc:" : "schema for storing tweets"
}
```

Avro file structure

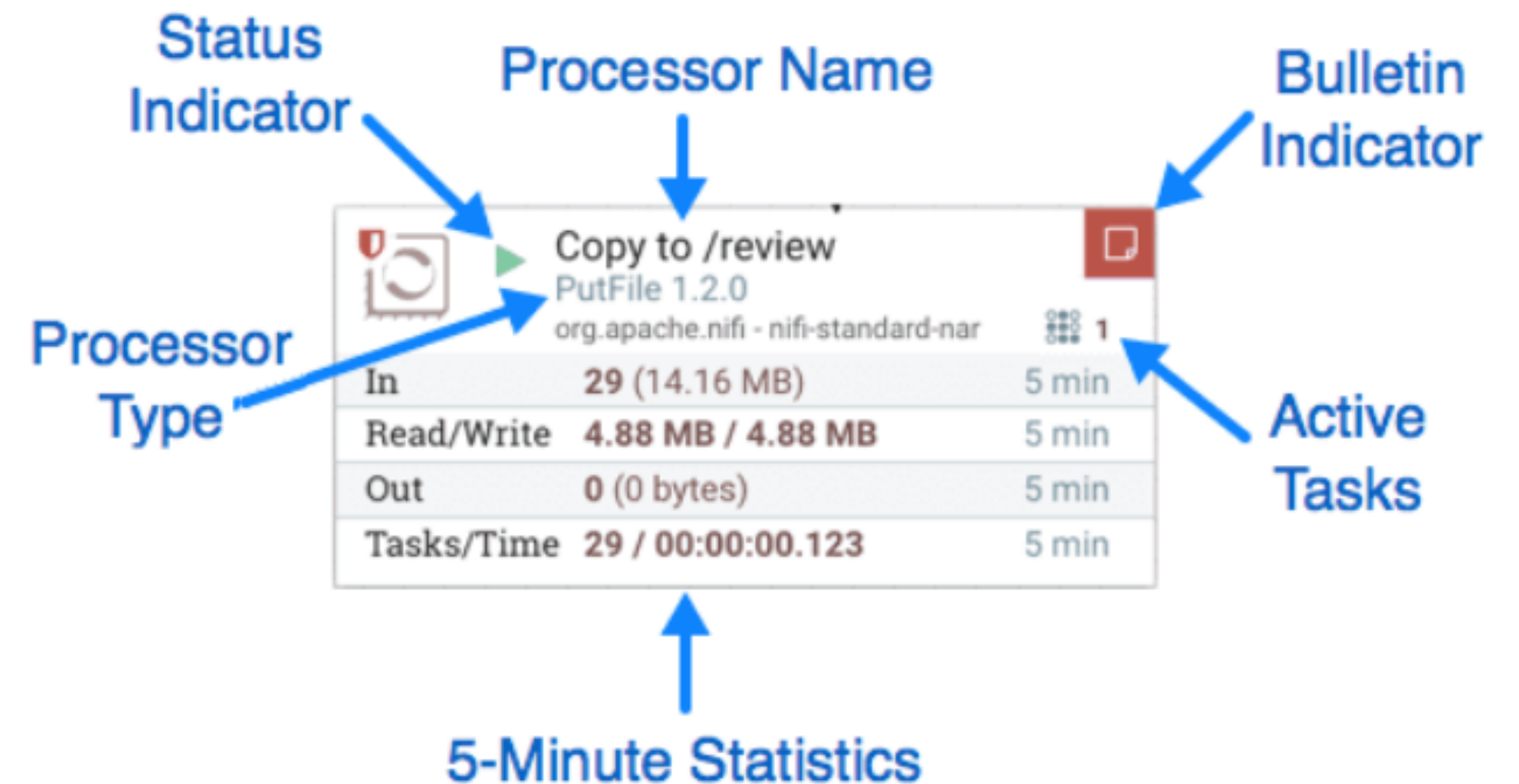


Flow File Processor – «функции»

Процессоры Apache NiFi являются основными блоками создания потока данных. Каждый процессор имеет разные функциональные возможности, что способствует созданию различных выходных потоковых файлов.

Отображаемые в GUI характеристики процессора:

- Processor Name – Пользовательское имя процессора
- Processor Type – Тип используемого процессора
- Active Tasks – Количество активных потоков
- Status Indicator – Статус процессора (включен/выключен)



Виды Flow File Processor-ов

Название	Примеры
Процессоры загрузки данных	GetFile, GetHTTP, GetFTP, GetKafka, ConsumeKafka
Процессоры маршрутизации и посредничества	RouteOnAttribute, RouteOnContent, ControlRate, RouteText
Процессоры доступа к базам данных	ExecuteSQL, PutSQL, PutDatabaseRecord, ListDatabaseTables
Процессоры извлечения атрибутов	UpdateAttribute, EvaluateJSONPath, ExtractText, AttributesToJSON
Процессоры системного взаимодействия	ExecuteScript, ExecuteProcess, ExecuteGroovyScript, ExecuteStreamCommand
Процессоры преобразования данных	ReplaceText, JoltTransformJSON
Отправка процессоров данных	PutEmail, PutKafka, PutSFTP, PutFile, PutFTP
Процессоры расщепления и агрегации	SplitText, SplitJson, SplitXml, MergeContent, SplitContent
HTTP-процессоры	InvokeHTTP, PostHTTP, ListenHTTP

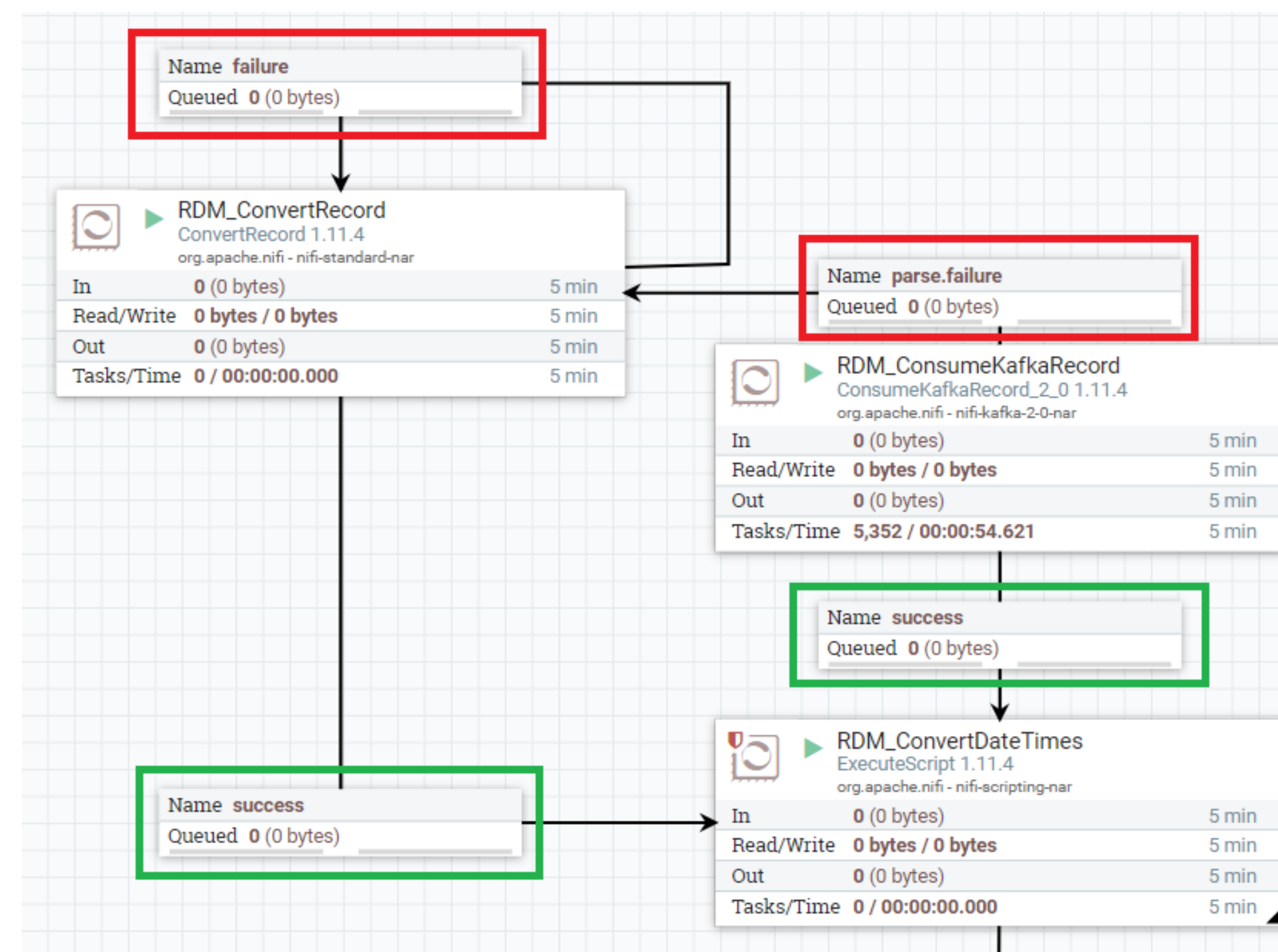
Connections – направление движения FlowFiles

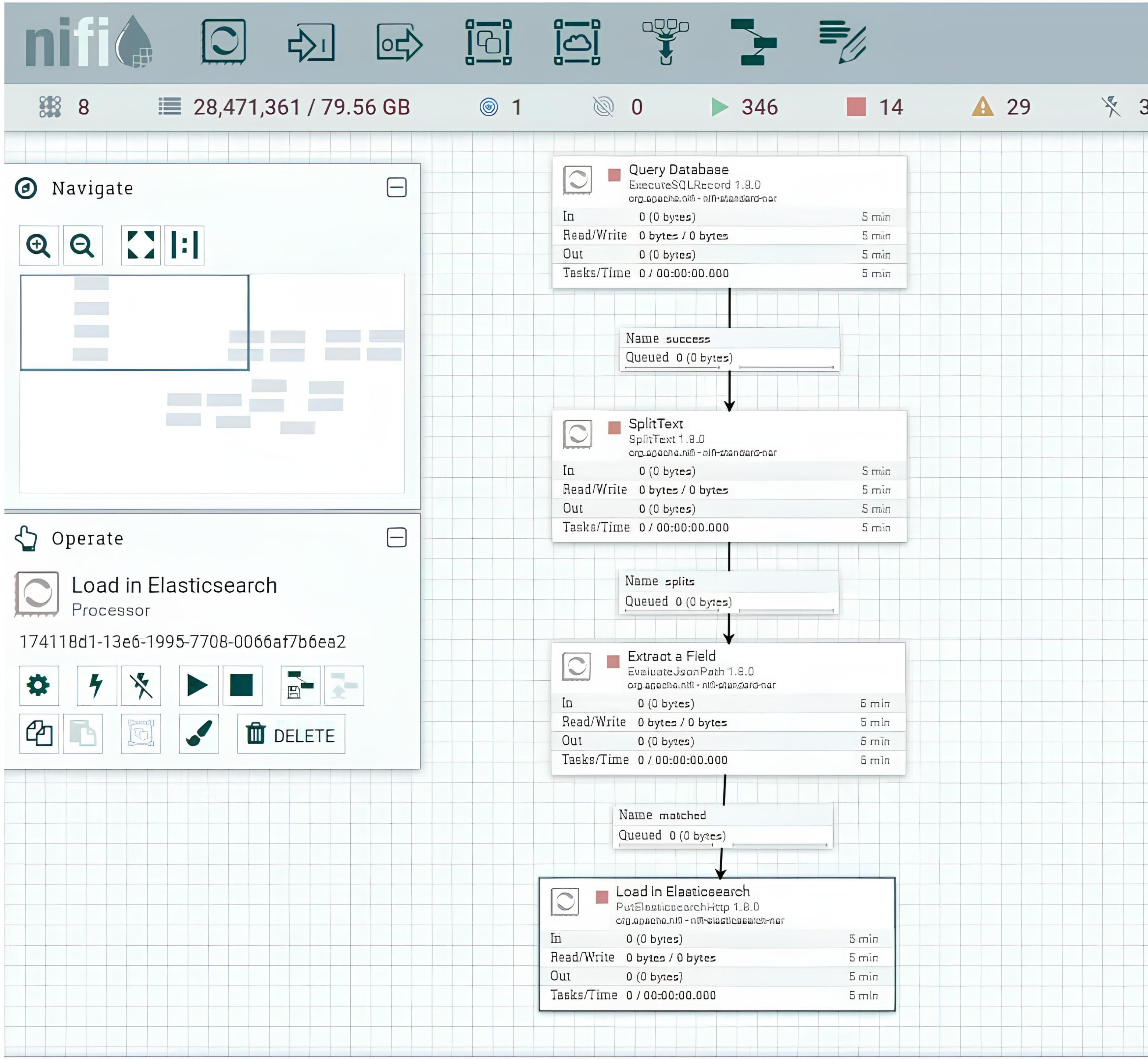
В потоке данных Apache NiFi потоковые файлы перемещаются от одного процессора к другому через соединение.

Каждый раз, когда создается соединение, разработчик выбирает одно или несколько отношений между этими процессорами.

Самые частые соединения:

- Success
- Failure





Пример NiFi-Flow

- Получили данные из БД с помощью запроса
- Разделили полученные записи на отдельные Flow-файлы
- Выделили из каждой записи атрибут
- Загрузили в Elasticsearch (распределённая система для полнотекстового поиска)

NiFi Registry

NiFi Registry - система версионирования NiFi Flow. Как гит, только для ETL.

NiFi Registry состоит из двух основных компонентов:

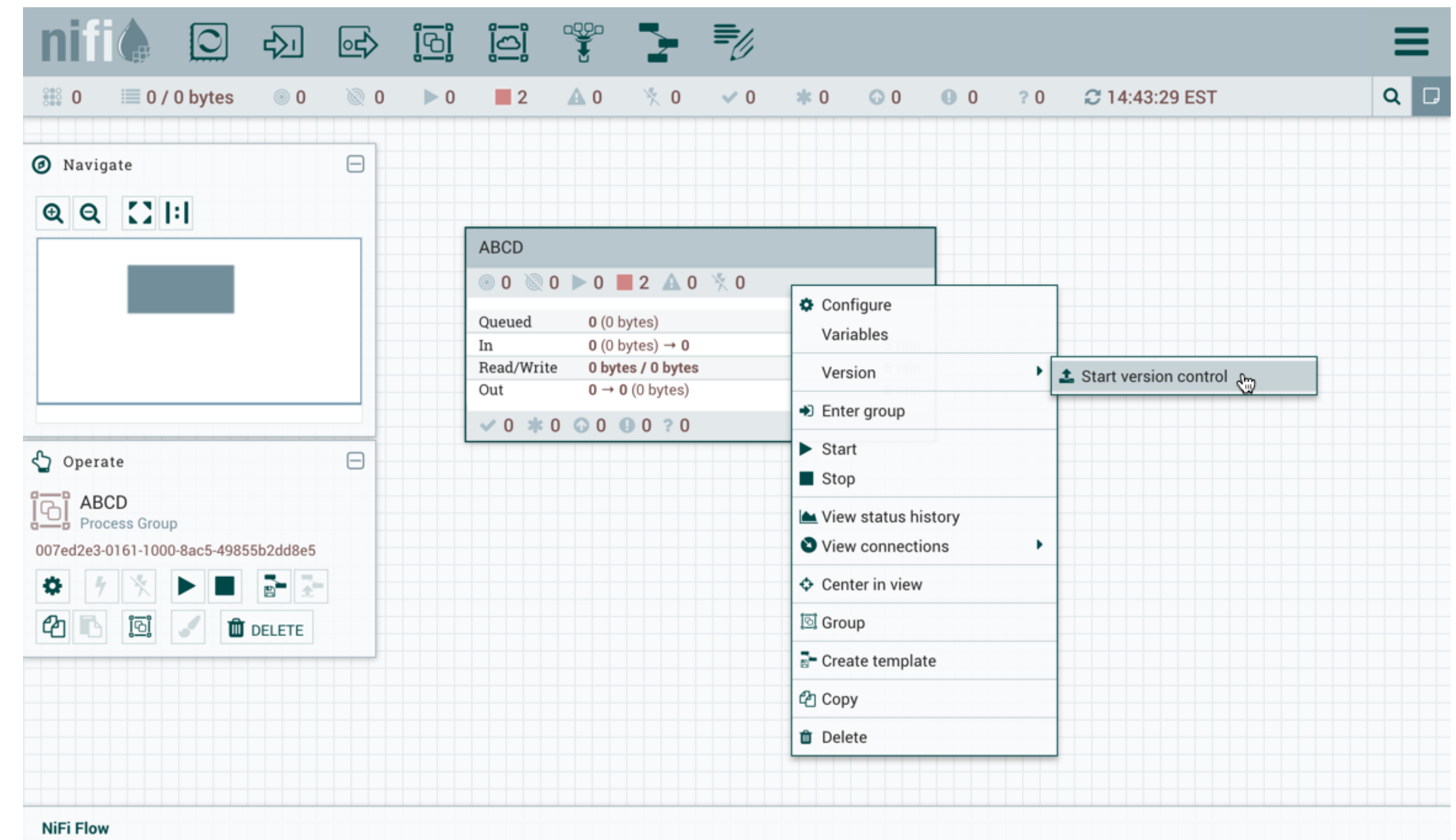
- Flow - поток данных NiFi на уровне группы процессов, который был помещен под контроль версий и сохранен в реестре.
- Bucket - контейнер, который хранит и организует потоки.



Шаблон взаимодействия с NiFi Registry

Типовые действия, которые можно выполнять в NiFi Registry:

- Создать пакет
- Добавить процесс под версионный контроль (работает только над группами)
- Закоммитить NiFi Flow
- Импортировать или экспортировать NiFi Flow



In-Memory DataBase (IMDB)

In-Memory DataBase

Как это обычно бывает, у термина In-Memory DataBase нет устойчивого русского перевода, это что-то вроде «базы данных в оперативной памяти».

Базы данных in-memory хранятся в оперативной памяти сервера. Очевидно, что это обеспечивает большие преимущества в скорости, поскольку операции с данными в памяти выполняются меньшим числом инструкций процессора

In-Memory DataBase

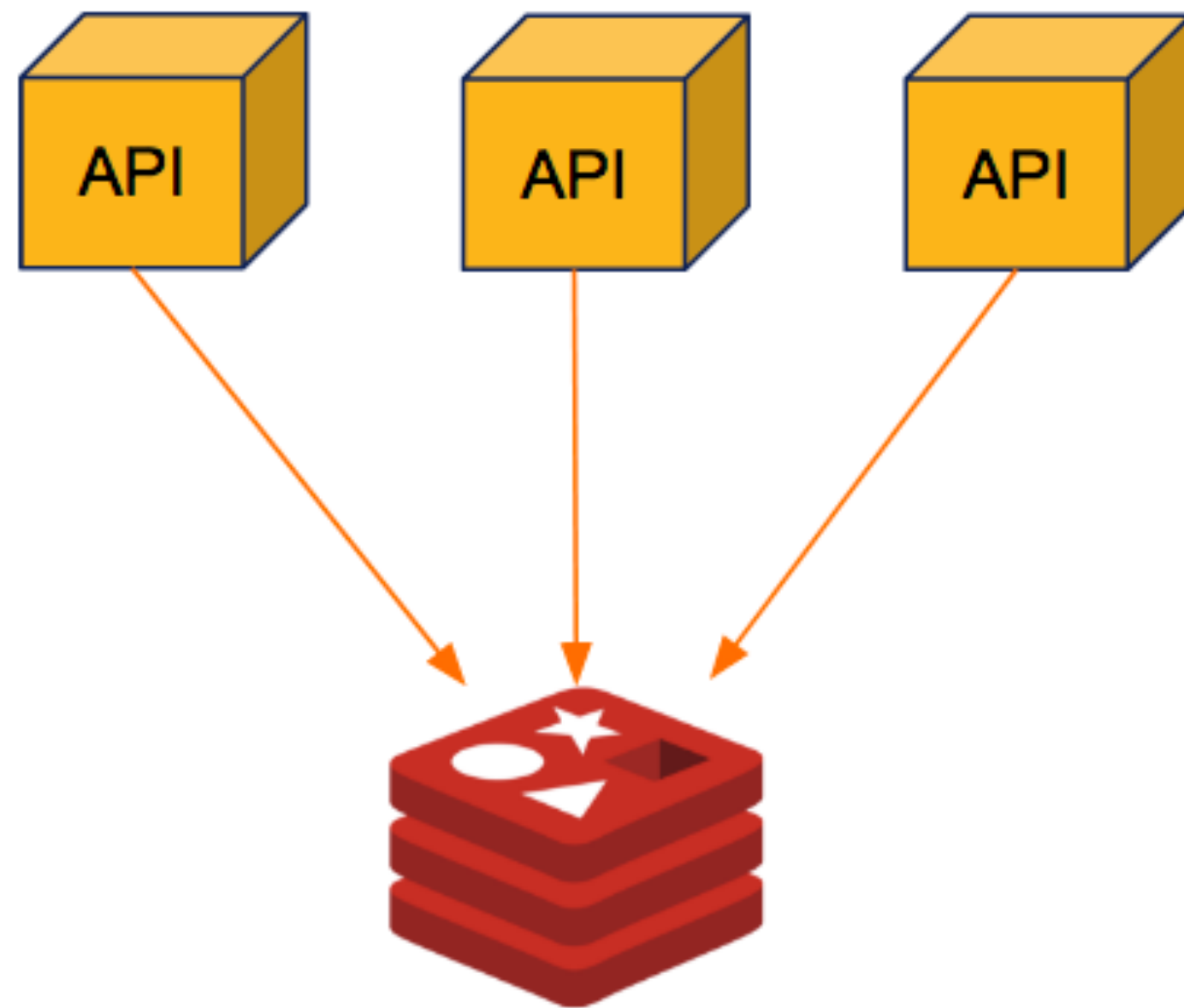
В терминах ACID базы данных in-memory конечно жертвуют надежностью, ведь при обесточивании или перезагрузке в них пропадают все данные.

Существуют ряд методов, которые позволяют снизить этот риск:

- Снэпшоты
- Логи транзакций
- Использование NVRAM или NVDIMM

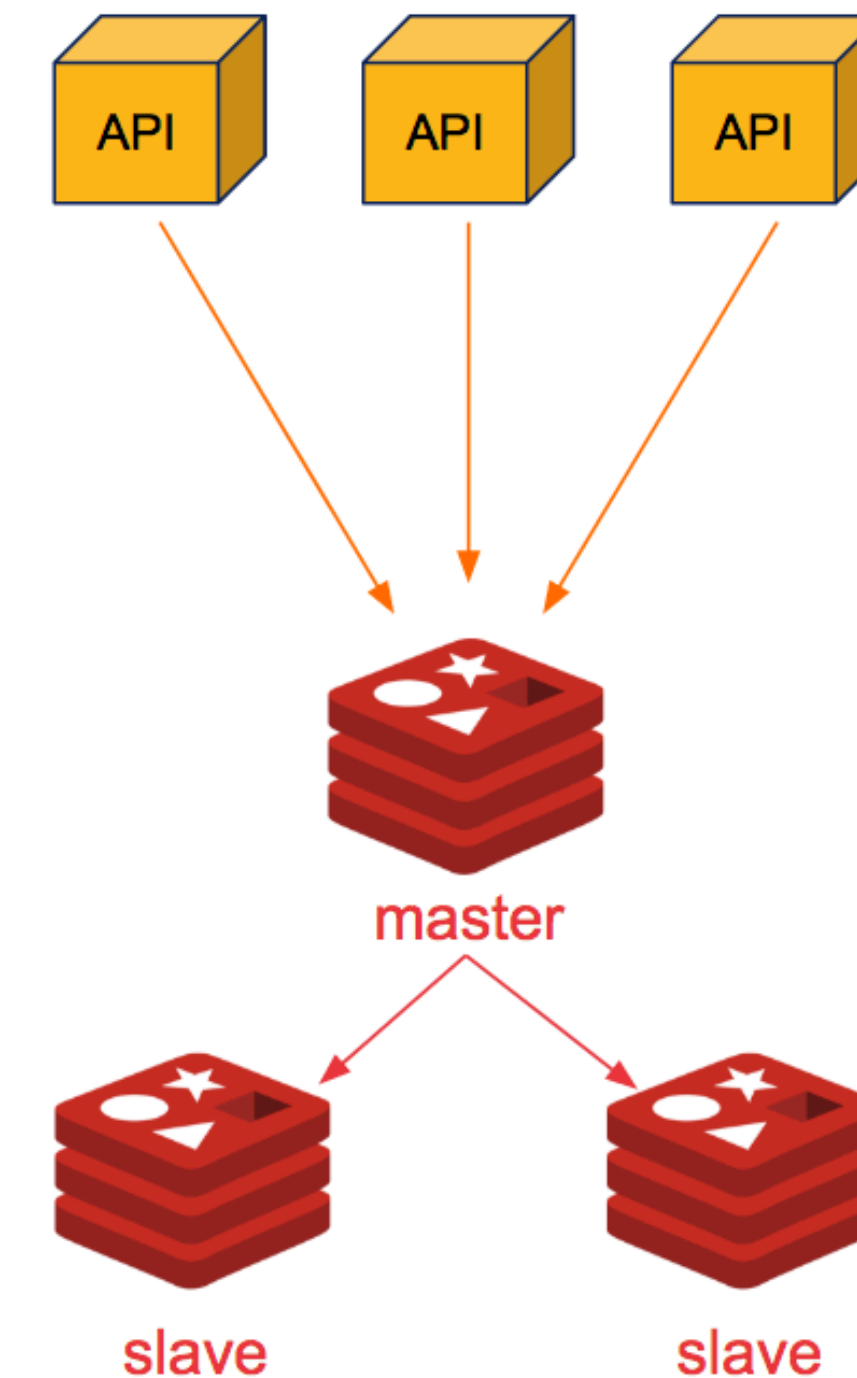
Топология Redis

Один Redis-инстанс



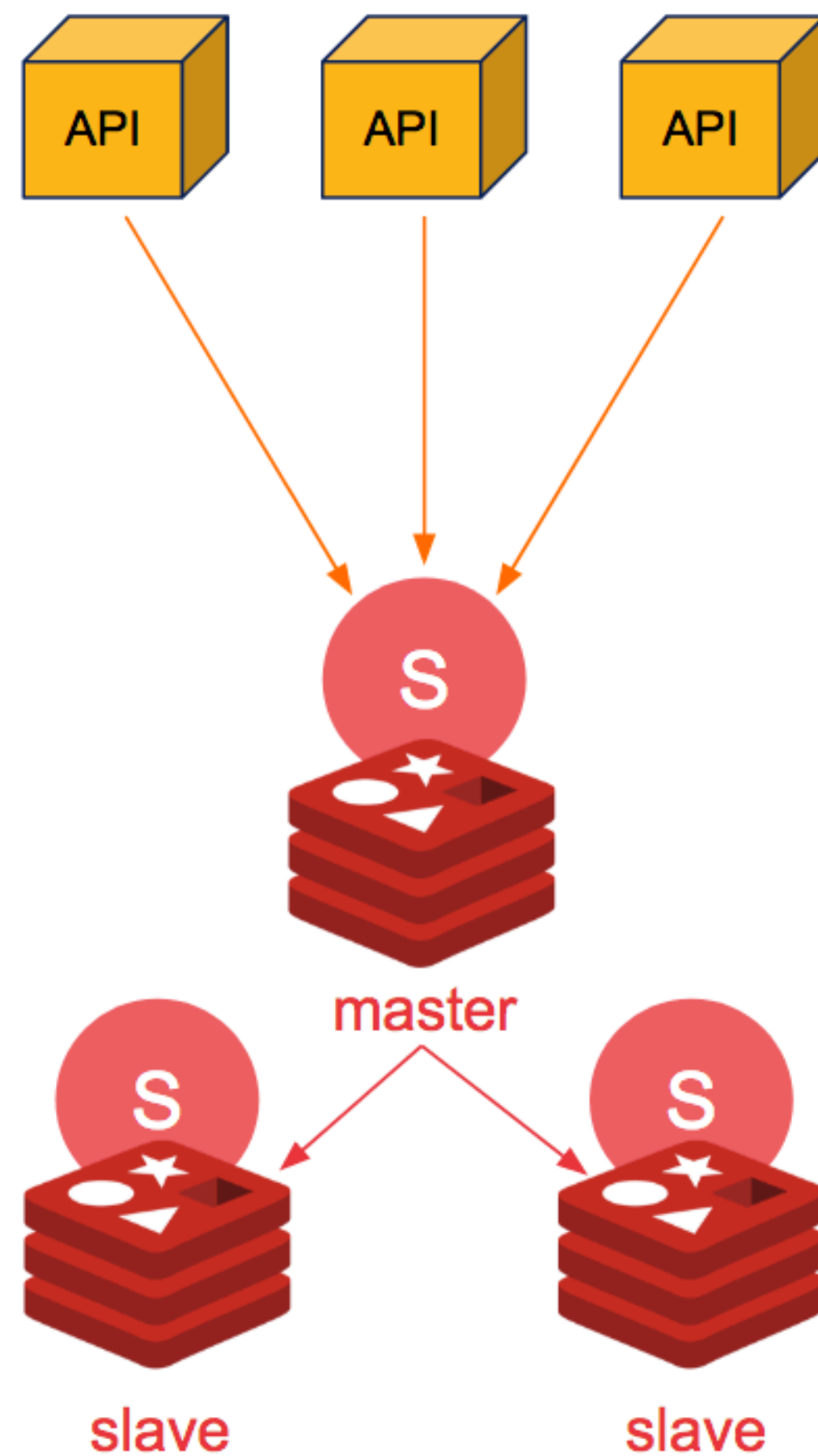
Имеет право на существование, но исключительно для сред разработки и тестирования, поскольку не является отказоустойчивой.

Master-slave репликация



В такой топологии между мастером и репликой происходит постоянная асинхронная репликация

Топология Redis Sentinel

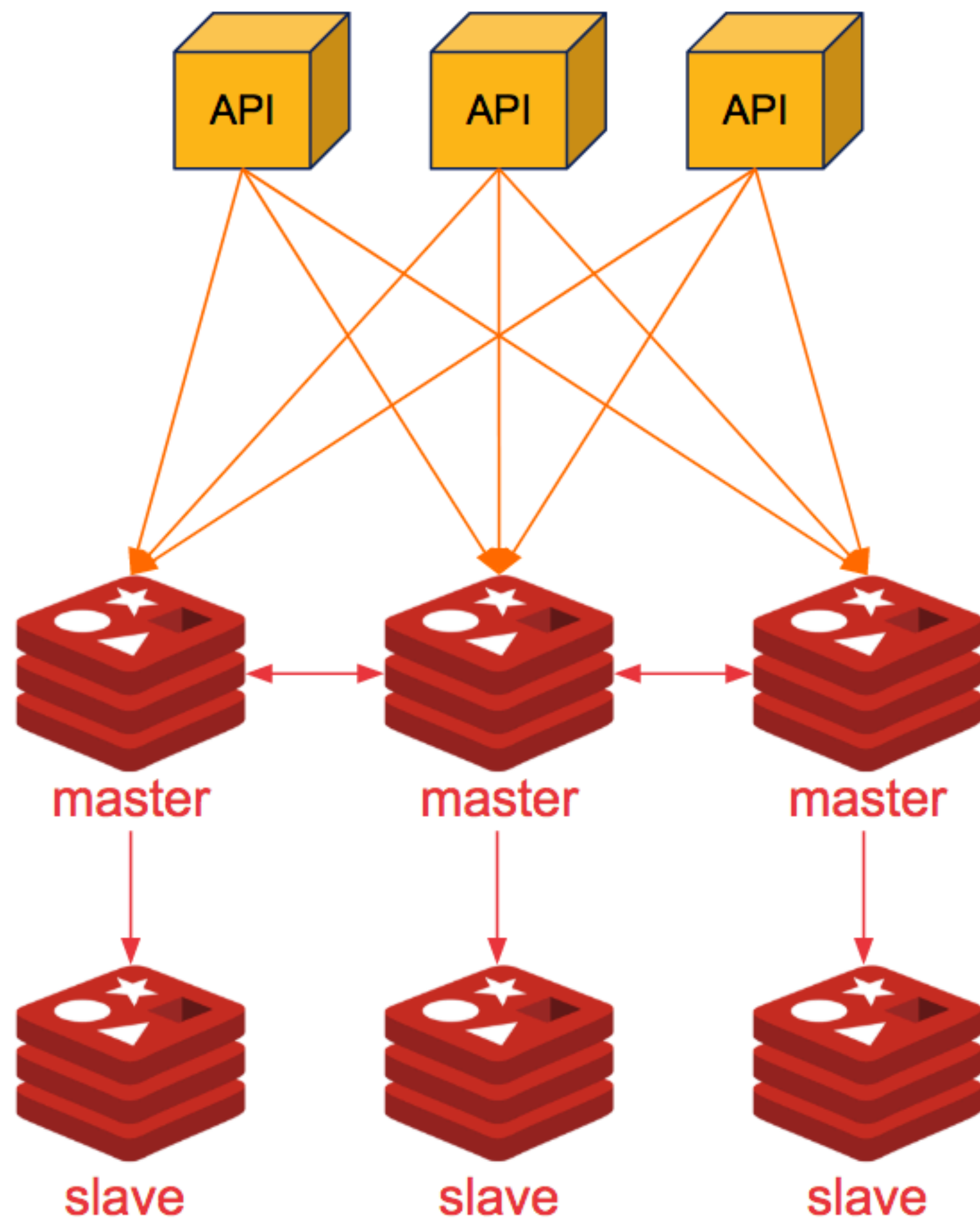


Эта топология развёртывания широко применялась на ранних версиях Redis, до поддержки полноценного кластера.

Она состоит из отдельных специальных узлов, которые мониторят работу основных узлов Redis, отслеживают сбои Master и запускают процесс восстановления.

Работает поверх топологии Master-Replica.

Топология Redis Cluster



Для репликации используется протокол Gossip: каждый узел передает информацию известным ему «соседям». Клиенты могут работать как с мастером, так и с репликой, но с реплики идет только чтение.

Данные в кластере шардированы. Кластер использует хеш-слоты. Всего кластер имеет 16384 слота. Каждый узел Redis отвечает за конкретное подмножество хеш-слотов.

Например:

- Узел А содержит хеш-слоты от 0 до 5500
- Узел В содержит хеш-слоты от 5501 до 11001
- Узел С содержит хеш-слоты от 11001 до 16383

Redis. key-value архитектура

Redis — быстрое хранилище в памяти с открытым исходным кодом для структур данных «ключ-значение». Redis поставляется с набором разнообразных структур данных в памяти, что упрощает создание различных специальных приложений.

Несколько фактов о ключах:

- Плохая идея - хранить слишком длинные ключи.
- Хорошая идея — придерживаться схемы при построении ключей: «object-type:id:field».

Типы данных

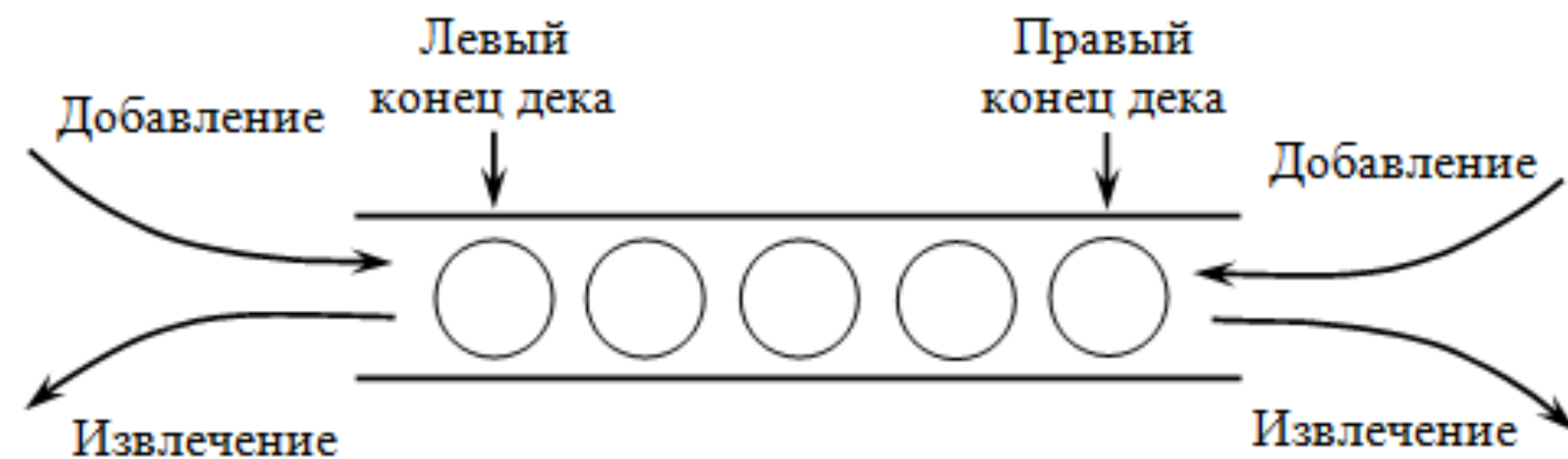
- **Строки** (strings) - базовый тип, ограничен размером 512 Мб.
- **Списки** (lists) - максимальное количество элементов - $2^{32} - 1$
- **Множества** (sets) - максимальное количество элементов - $2^{32} - 1$
- **Хэш-таблицы** (hashes) - максимальное количество элементов - $2^{32} - 1$
- **Упорядоченные множества** (sorted sets) - элементы упорядочены по параметру «score»

Базовые команды для Redis

- **SET** - Команда используется для установки ключа и его значения, с дополнительными необязательными параметрами для указания срока действия записи значения ключа
- **TTL** - Когда ключ установлен с истечением срока действия
- **GET** - Команда используется для получения значения, связанного с ключом
- **EXISTS** - Команда проверяет, существует ли что то с данным ключом. Она возвращает 1 если объект существует или 0 если нет
- **FLUSHALL** - Команда полностью удаляет все данные в текущем сеансе
- **DEL** - Команда удаляет ключ и соответствующее значение
- **KEYS** - Возвращает все ключи из базы по указанному шаблону
- **INCR / DECR** - Инкремент / декримент

Список

Список это последовательность значений упорядоченных по порядку их создания. Список в Redis - аналог двух-стороннего стека и применяется обычно для создания очередей.



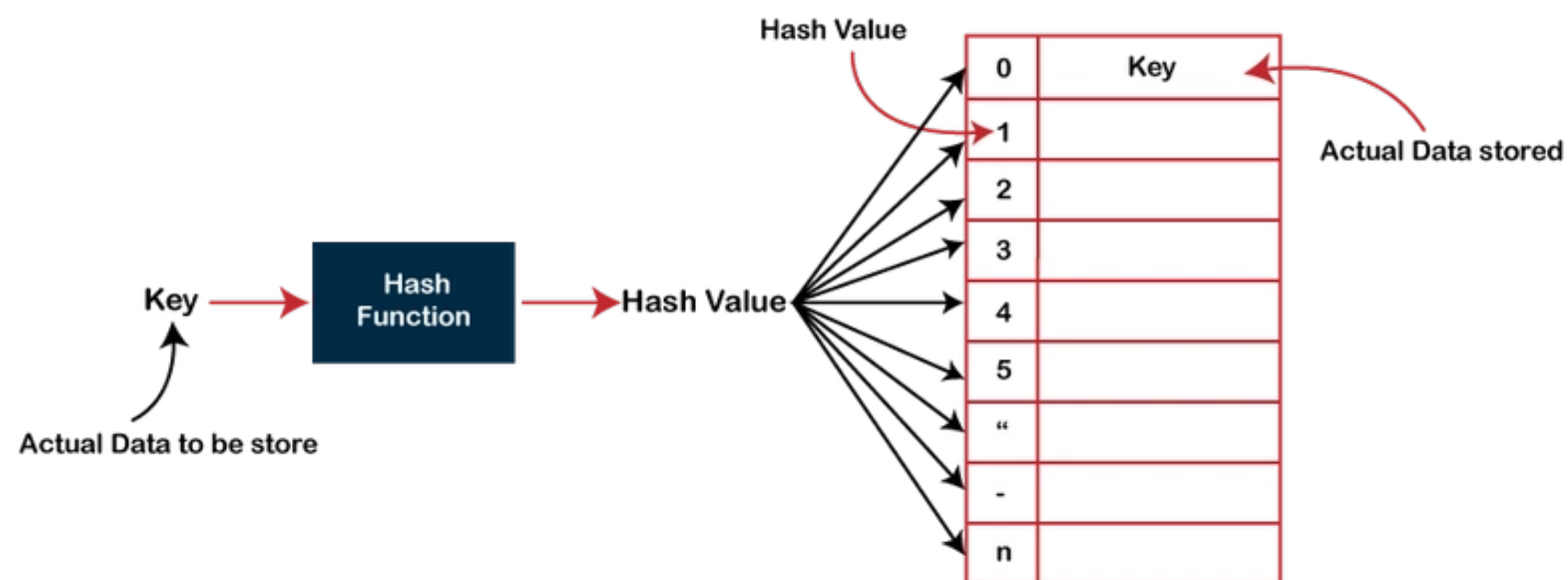
Базовые команды:

- LPUSH — Команда добавления элемента в список
- LRANGE — возвращает срез списка с левой стороны
- LPOP — вернуть одно значение с левой стороны и удалить его из списка

Команды RPUSH, RRANGE, RPOP аналогичны, но только для правой стороны.

Хэш-таблица

Redis позволяет в качестве значения так же использовать ключ: значение. Что по сути будет почти аналогией объектов из JavaScript или словари в Python.



Базовые команды:

- Для записи объекта используется команда HSET в следующем формате HSET имя_ключа имя_атрибута значение
- Для чтения объекта используется команда HGET в формате HGET имя_ключа имя_атрибута
- Команда HGETALL используется для получения всей хэш-таблицы

Механизм подписок PUS-SUB

Одно из основных преимуществ Redis от других key-value хранилищ заключается в том, что в Redis есть механизм подписок.

Redis можно использовать как сервер сообщений.

Одни клиенты подписываются на определенные каналы используя команду SUBSCRIBE имя_канала:

```
SUBSCRIBE channel
```

Другие клиенты могут отправлять сообщения в этот канал используя команду PUBLISH имя_канала значения:

```
PUBLISH channel "hello world"
```


Python и Redis

Redis очень широко применяется в современной разработке ПО.

Библиотеки поддержки есть для любого языка программирования.

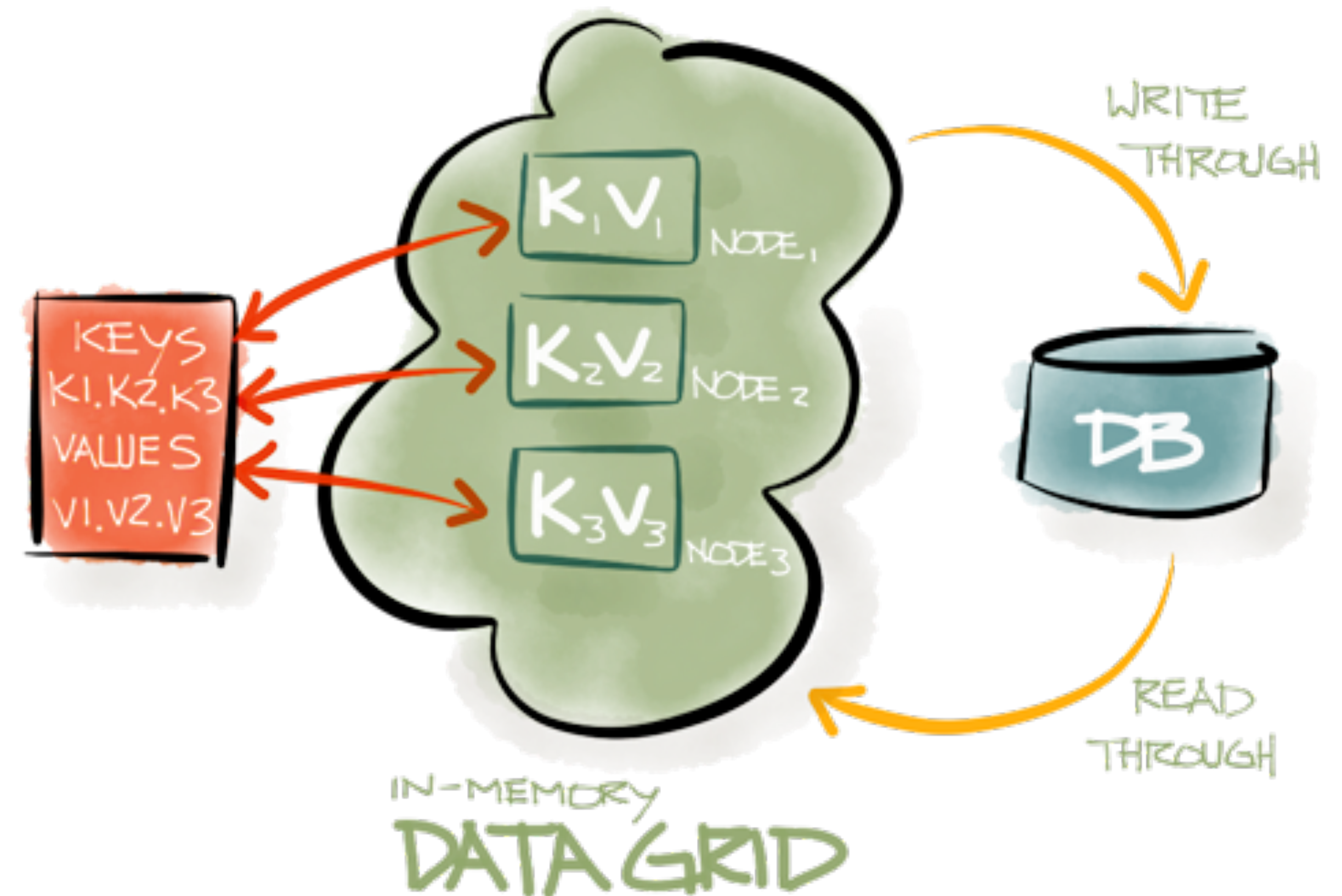
Рассмотрим python библиотеку Redis.

```
from time import sleep
import redis
```

```
with redis.Redis(host='127.0.0.1') as client:
    client.set('Language', 'Python', ex=5)
    print(client.get('Language'))
    sleep(3)
    print(client.ttl('Language'))
    sleep(3)
    print(client.ttl('Language'))
```

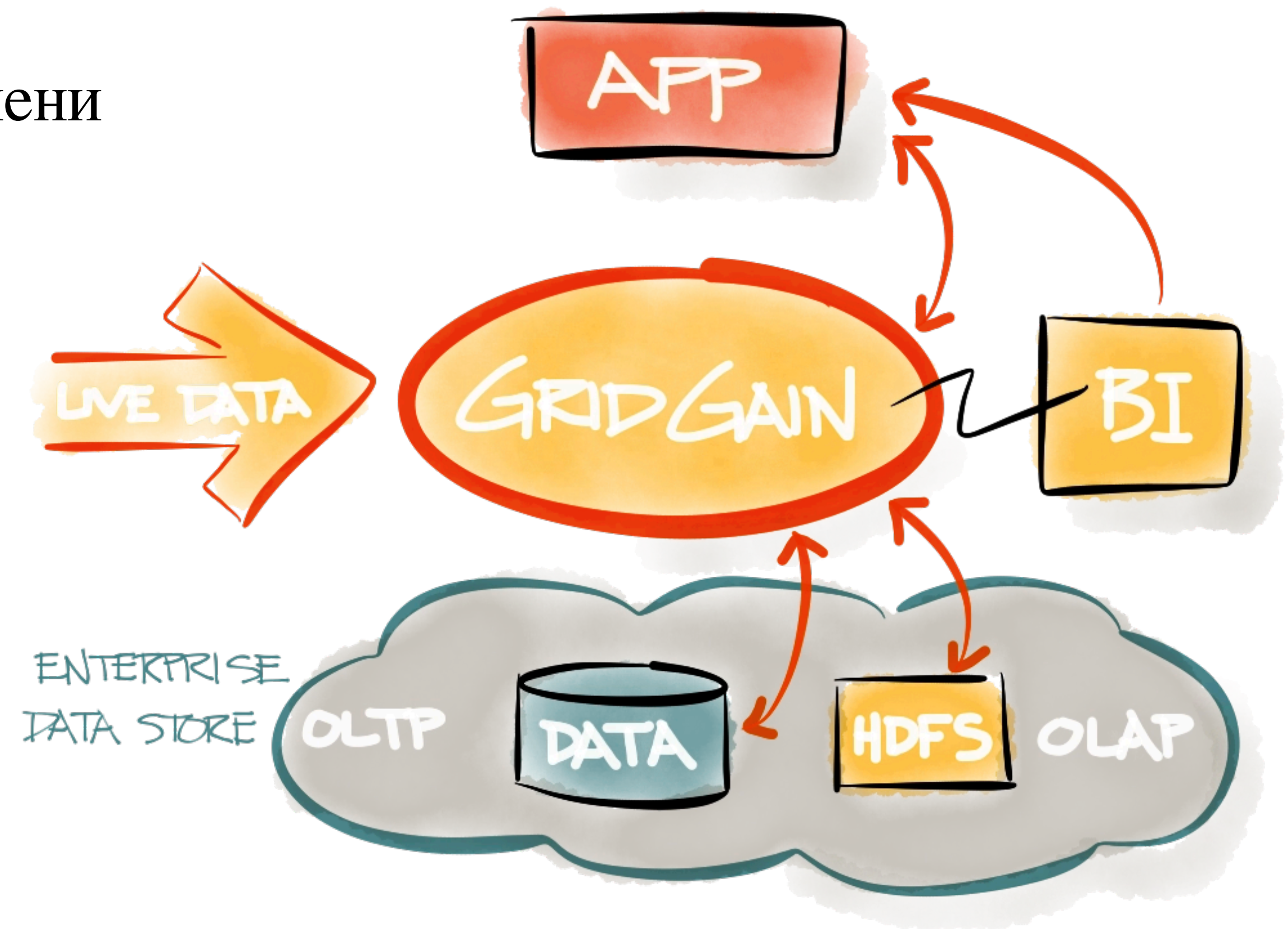

Преимущества In-Memory DataBase

- Высокая производительность
- Структуры данных в памяти
- Универсальность и простота использования
- Репликация и сохранность
- Поддержка удобного языка разработки



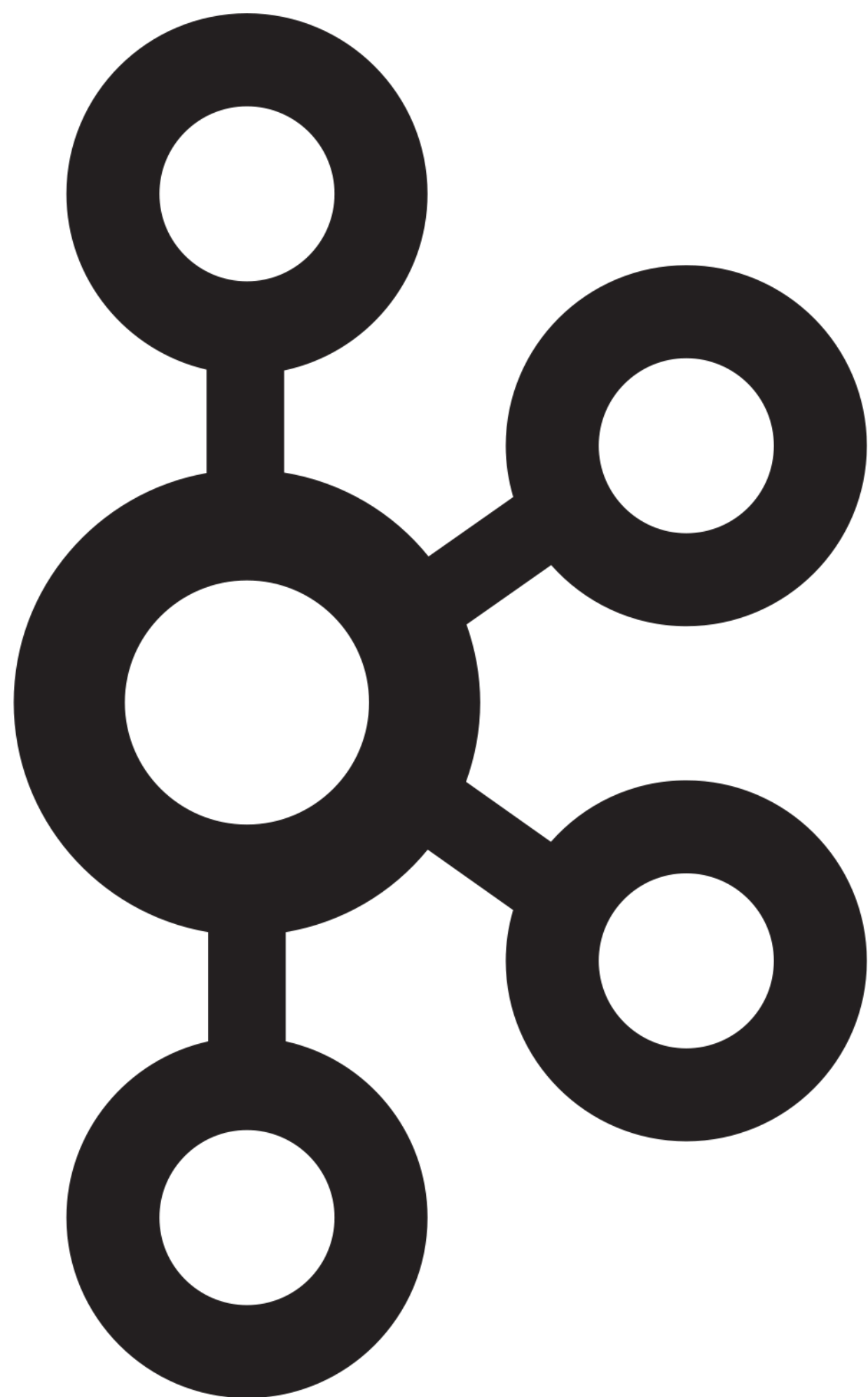
Примеры использования In-Memory DataBase

- Кэширование
- Управление сессиями
- Таблицы лидеров в режиме реального времени
- Ограничение интенсивности
- Очереди
- Чат и обмен сообщениями



Kafka Cluster

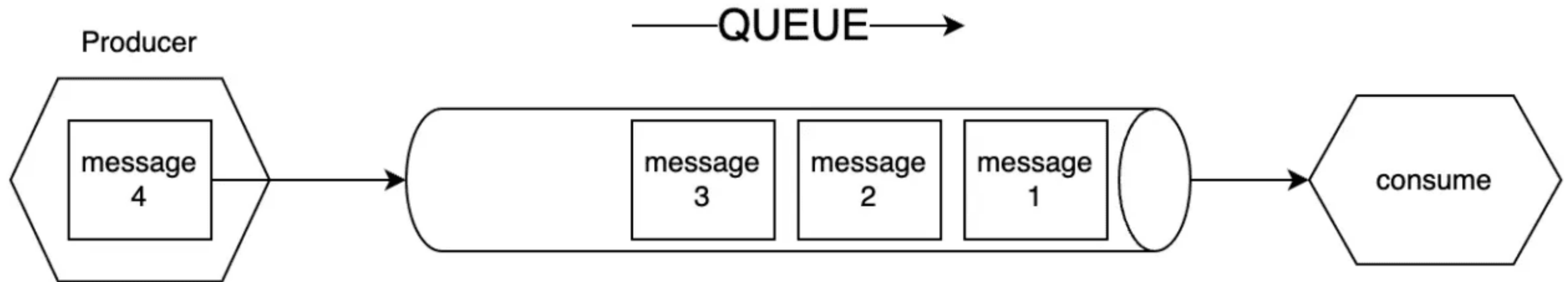
Очередь сообщений



Apache Kafka — распределённый программный брокер сообщений с открытым исходным кодом, разрабатываемый в рамках фонда Apache на языках Java и Scala.

Цель проекта — создание горизонтально масштабируемой платформы для обработки потоковых данных в реальном времени с высокой пропускной способностью и низкой задержкой.

Очередь сообщений

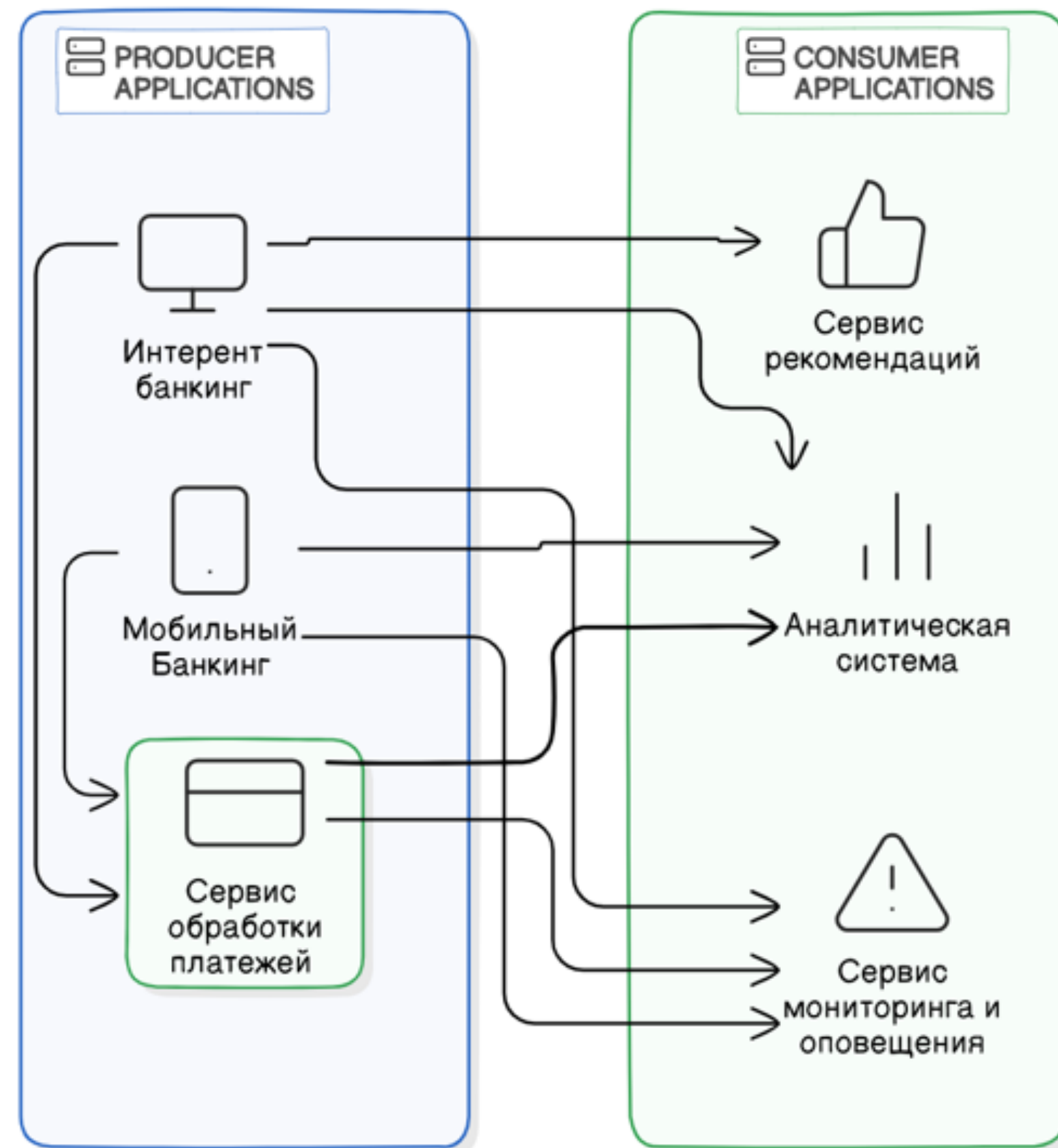


В процессе стриминга данных используются основные три участника обработки данных:

- Producer - тот, кто создает данные и направляет данные в очередь
- Queue - очередь сообщений
- Consumer - тот, кто принимает сообщения из очереди

Средство взаимодействия микросервисов

Без централизованной
очереди сообщений



С централизованной
очередью сообщений

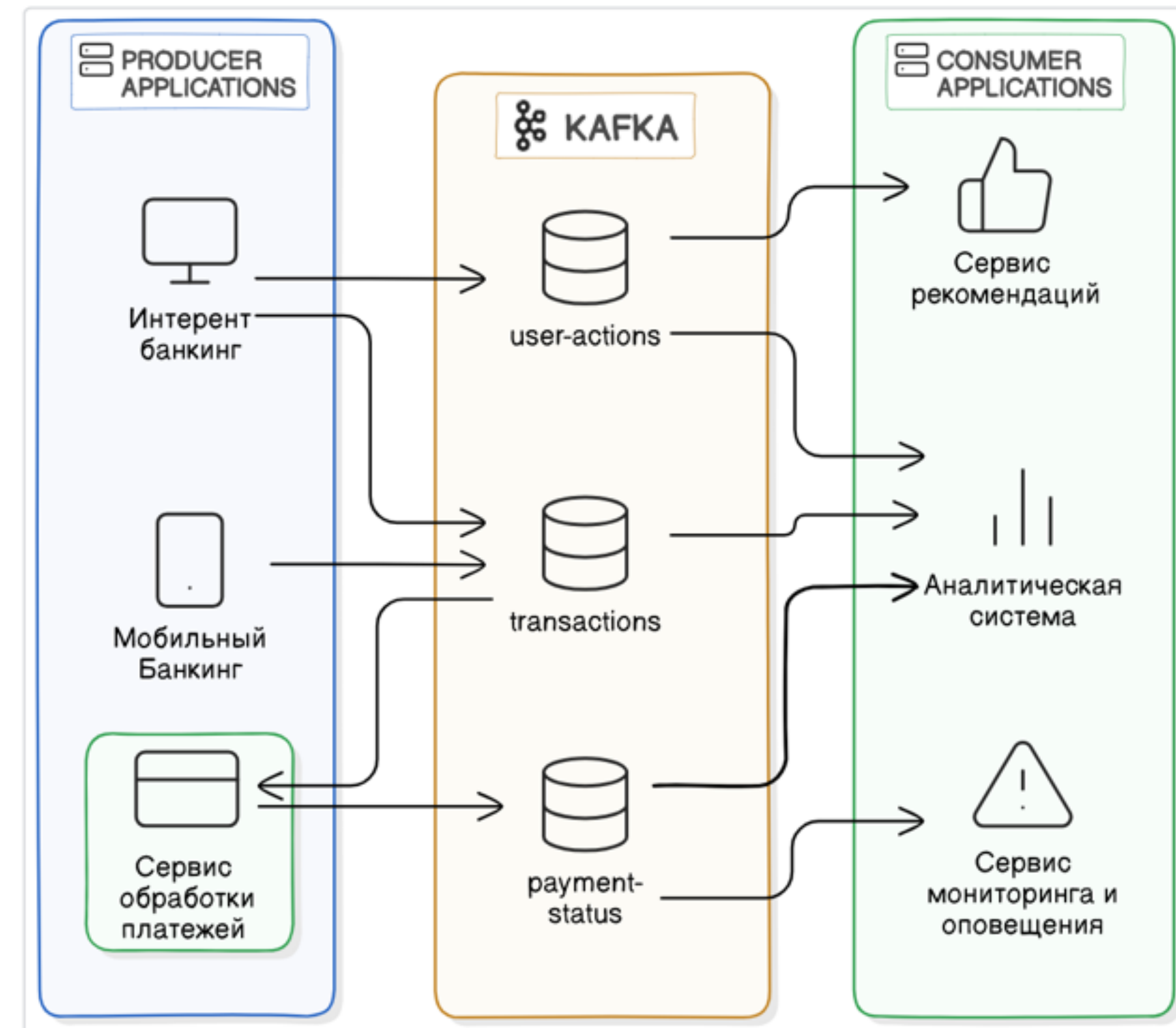
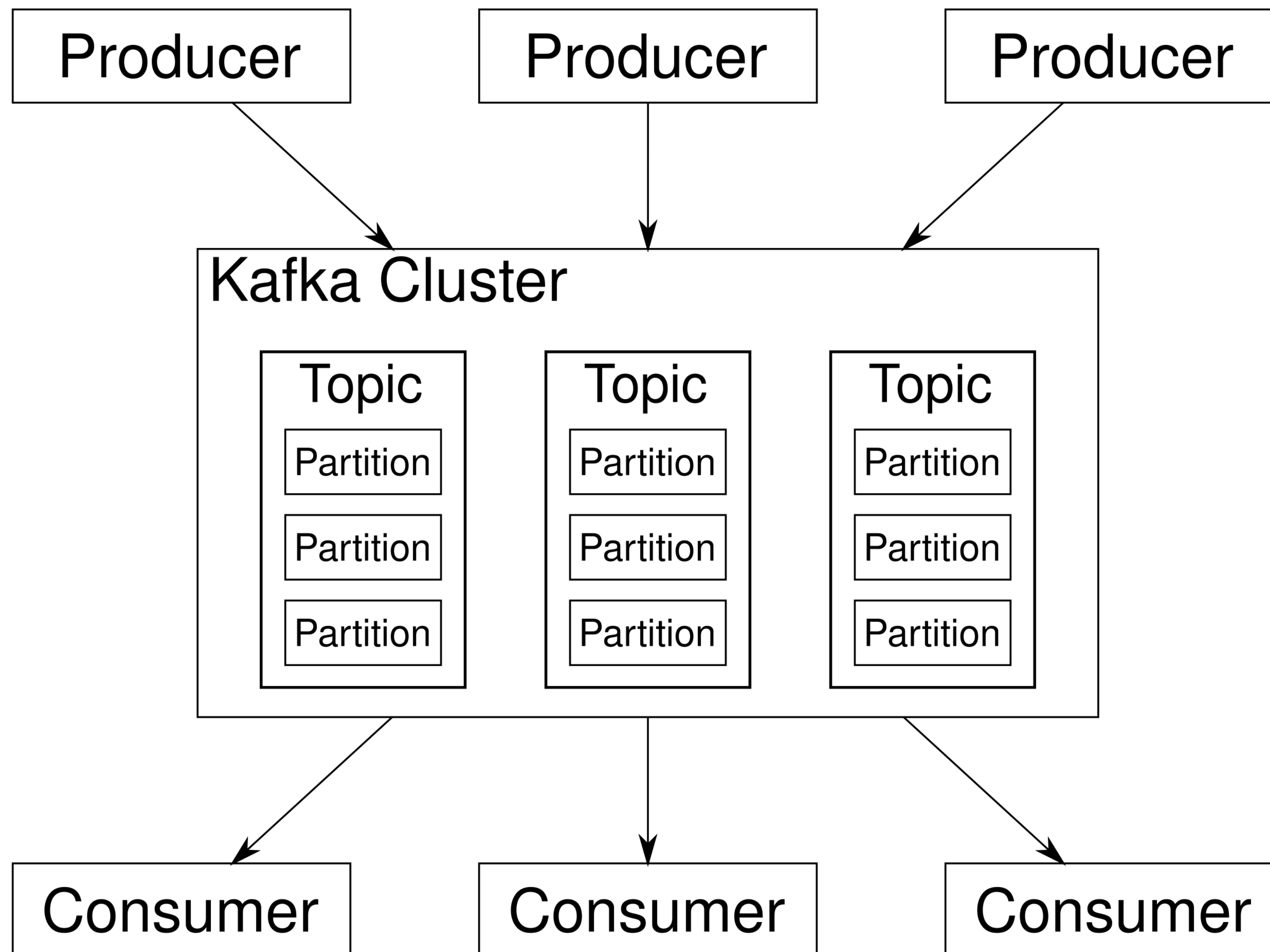


Схема Kafka Cluster

ОСНОВНЫЕ ПОНЯТИЯ
Kafka Cluster:

- Topic
- Partition
- Offset

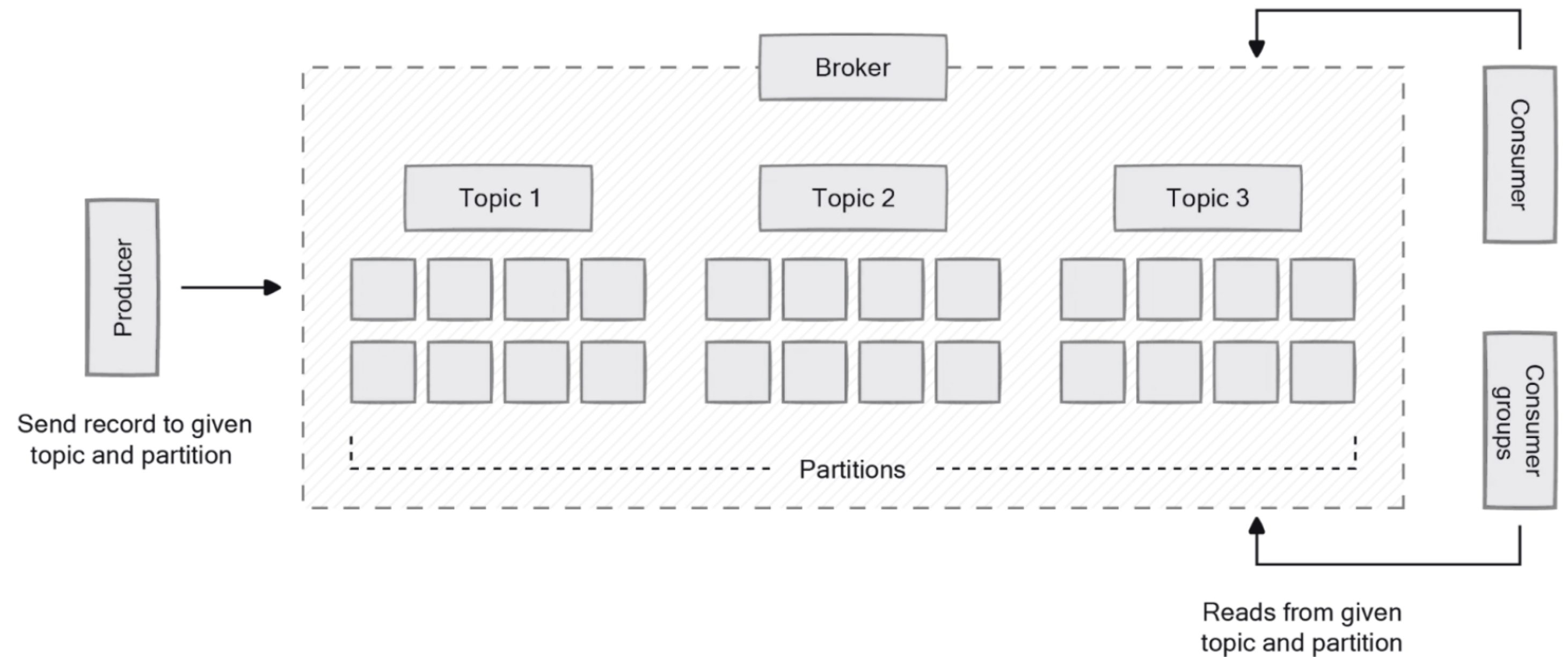


Kafka. Топик

Топик в Kafka — это логическая единица хранения сообщений.

Каждый топик представляет собой категорию или поток данных:

- производители (producers) отправляют в топик свои сообщения
- потребители (consumers, консьюмеры) читают сообщения из топика.



Kafka. Партиция

Партиция — это физическая единица хранения сообщений внутри топика. Партиция разбивает топик на несколько логически независимых частей, где каждая представляет собой упорядоченный, неизменяемый набор записей. Сообщения добавляются только в конец партиции.

Для чего нужны партиции:

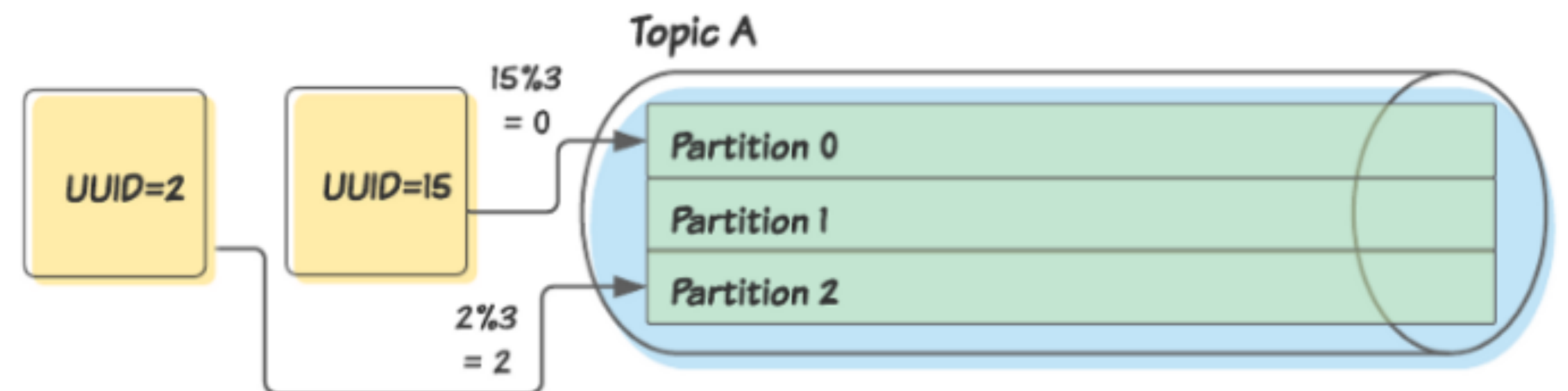
- Параллелизм
- Масштабируемость
- Отказоустойчивость



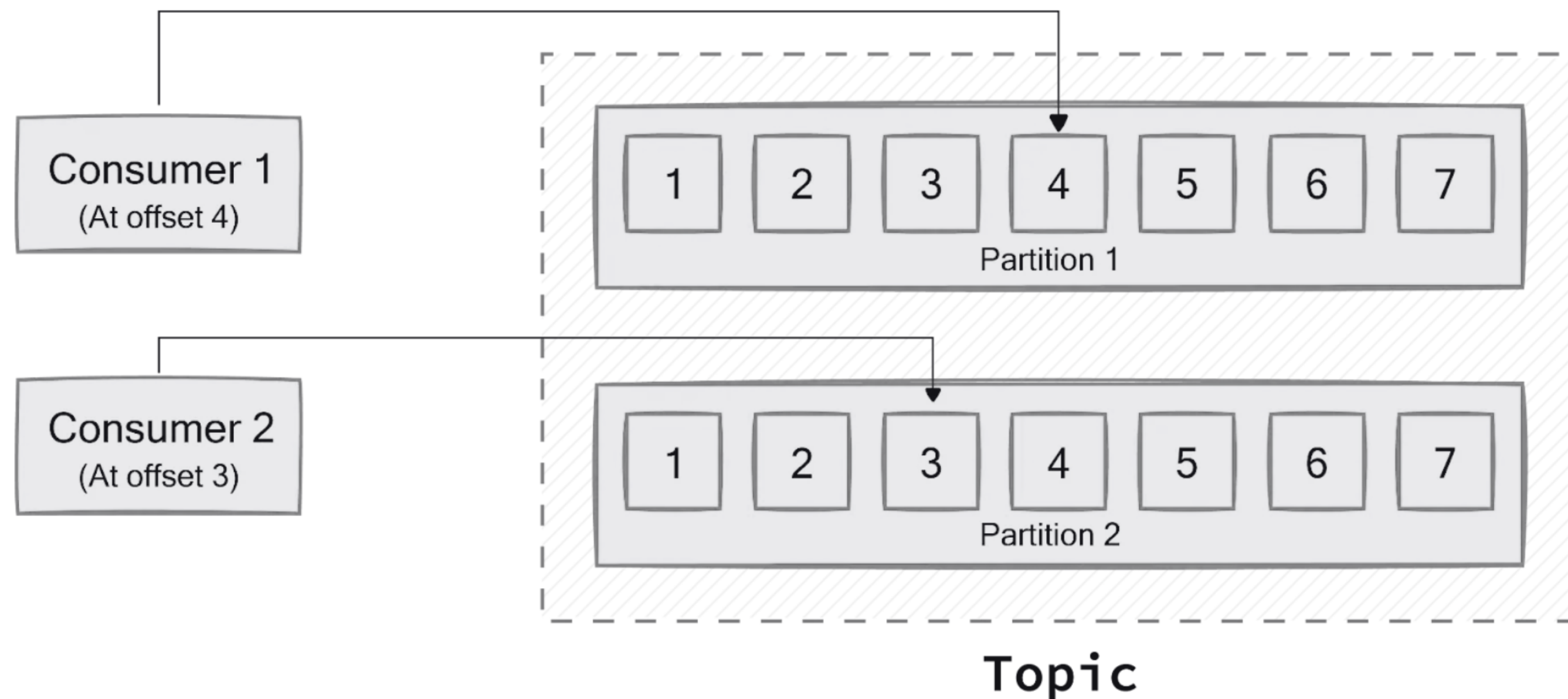
Kafka. Наполнение данными патриции

Kafka позволяет детерминированно назначать сообщения в партии при помощи **ключа партии** (*partition key*). Ключ партии — это часть данных (обычно некоторый атрибут самого сообщения, например, идентификатор), к которой применяется алгоритм для определения партии.

- Назначим ключом партии поле UUID сообщений.
- Продюсер применяет алгоритм (например, как GP модифицирует каждое значение UUID по количеству партий)
- Каждое сообщение помещается в соответствующую партию



Kafka. Смещение



Offset (смещение) — это номер сообщения внутри **partition**.

Kafka не забывает события после чтения. Сообщения не удаляются в течении времени которое указано в настройках кафки. Каждый consumer сам помнит, до какого offset он дочитал.

Проверка подключения и мета информации

Подключение к кластеру с помощью
KafkaCat (kcat)

```
kcat -b  
<внутренний_IP_брокера>:<порт_брокера> \  
-X security.protocol=SASL_PLAINTEXT \  
-X sasl.mechanism=SCRAM-SHA-512 \  
-X sasl.username="<логин_пользователя>" \  
-X sasl.password="<пароль_пользователя>"
```

Проверка метаданных кластера с
помощью KafkaCat (kcat)

```
kcat -b  
<внутренний_IP_брокера>:<порт_брокера> -L \  
-X security.protocol=SASL_PLAINTEXT \  
-X sasl.mechanism=SCRAM-SHA-512 \  
-X sasl.username="<логин_пользователя>" \  
-X sasl.password="<пароль_пользователя>"
```

KafkaCat. Консольная работа с Кафкой

Запись сообщения в Кафку

```
kcat -P \
  -b <внутренний_IP_брокера>:<порт_брокера> \
  -t <имя_топика> \
  -X security.protocol=SASL_PLAINTEXT \
  -X sasl.mechanism=SCRAM-SHA-512 \
  -X sasl.username="<логин_пользователя>" \
  -X sasl.password="<пароль_пользователя>" -K:
```

Далее необходимо внести сообщение в формате **key**:value

1:foo

2:bar

Для завершения ввода необходимо использовать Ctrl + D

Чтение сообщения из Кафки

```
kcat -b <внутренний_IP_брокера>:<порт_брокера> \
  -t <имя_топика> \
  -C -o beginning \
  -X security.protocol=SASL_PLAINTEXT \
  -X sasl.mechanism=SCRAM-SHA-512 \
  -X sasl.username="<логин_пользователя>" \
  -X sasl.password="<пароль_пользователя>"
```

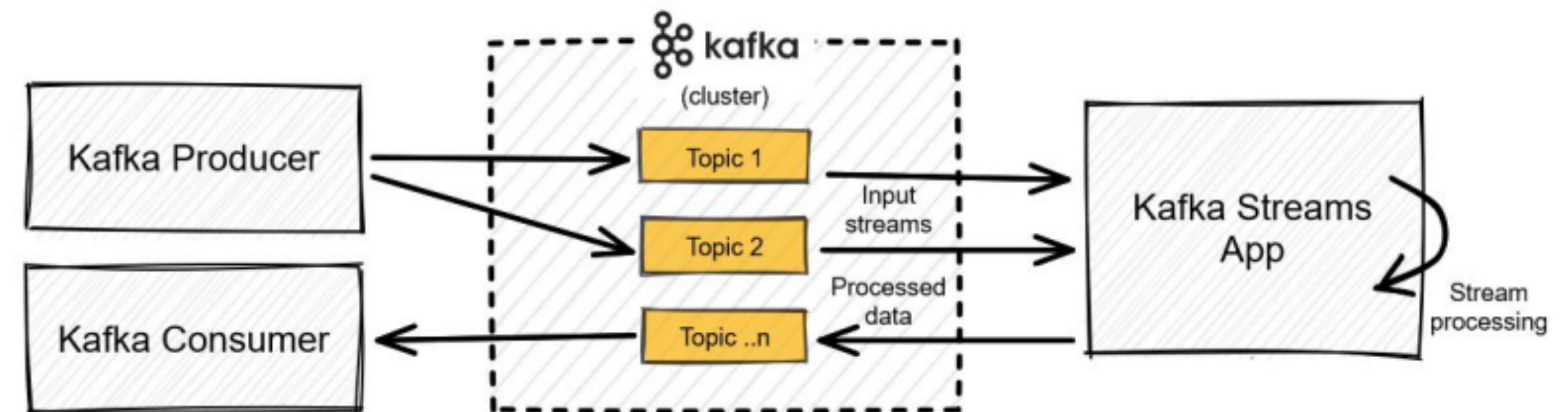
Python. Работа с Кафкой

Запись сообщения в Кафку

```
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='localhost:1234')
for _ in range(100):
    producer.send('foobar', b'some_message_bytes')
```

Чтение сообщения из Кафки

```
from kafka import KafkaConsumer
consumer = KafkaConsumer('my_favorite_topic')
for msg in consumer:
    print(msg)
```



Для чего применять Kafka Cluster

Для каких задач **можно** применять Кафку:

- Обработка событий или логов
- IoT-приложение для сенсоров
- Доставка событий многим потребителям
- Система уведомлений и событий
- Обработка большого потока данных
- Проектирование event-driven системы

Для каких задач **не стоит** применять Кафку:

- Подмена реляционный или не реляционный СУБД
- Чтение подряд больших сообщений данных
- Нужна гибкая кастомизация каждого сообщения
- Нужны очереди, поддерживающие сложные схемы обработки сообщений

Методы оркестрации задач

Apache Airflow

Apache Airflow — это платформа управления обработкой данных с открытым исходным кодом (оркестратор).

Это open source проект, написанный на Python, был создан в 2014 году в компании Airbnb.

В 2016 году Airflow ушел в Apache Software Foundation, прошел через инкубатор и в начале 2019 года перешел в статус top-level проекта Apache.

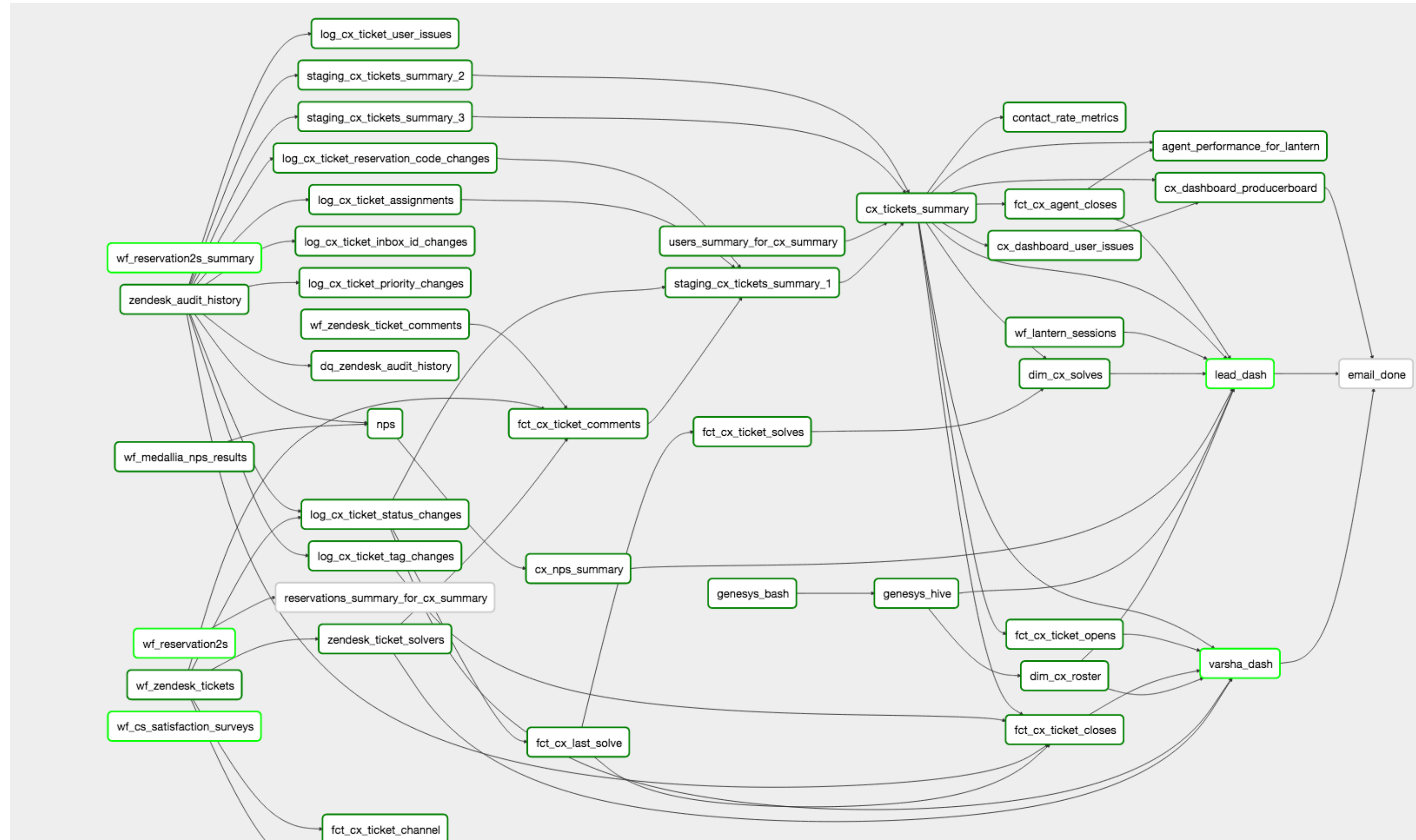


Apache
Airflow

Основные сущности Apache Airflow

DAG (Directed Acyclic Graph) — это некоторое смысловое объединение ваших задач, которые вы хотите выполнить в строго определенной последовательности по определённому расписанию.

DAG состоит из операторов, которые являются единицей работы.



Операторы Apache Airflow

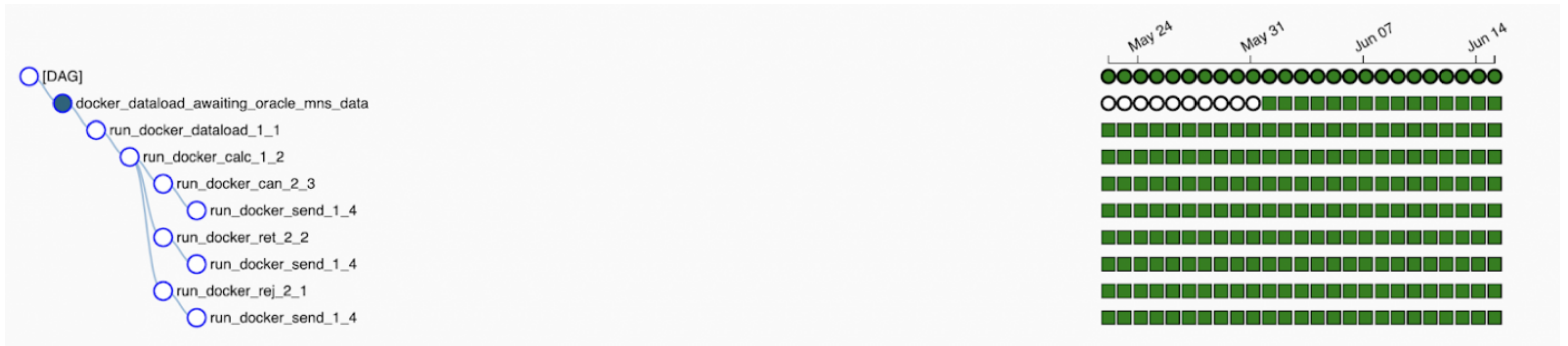
Оператор — это сущность, на основании которой создаются экземпляры заданий, где описывается, что будет происходить во время исполнения экземпляра задания.

Название оператора	Описание
BashOperator	Оператор для выполнения bash-команды
PythonOperator	Оператор для вызова Python-кода
PostgresOperator	Оператор для выполнения SQL-кода в СУБД PostgreSQL
MySqlOperator	Оператор для выполнения SQL-кода в СУБД MySql
SqlOperator	Оператор для выполнения SQL-кода
HTTPOperator	Оператор для работы с http-запросами
Sensor	Оператор ожидания события
EmailOperator	Оператор для отправки email'а
DummyOperator	"Пустой" оператор, используемый для группировки задач (Task)

Идемпотентность данных

Одна из ключевых концепций, лежащих в основе AirFlow — это хранение информации о каждом запуске DAG в соответствии с настроенным расписанием.

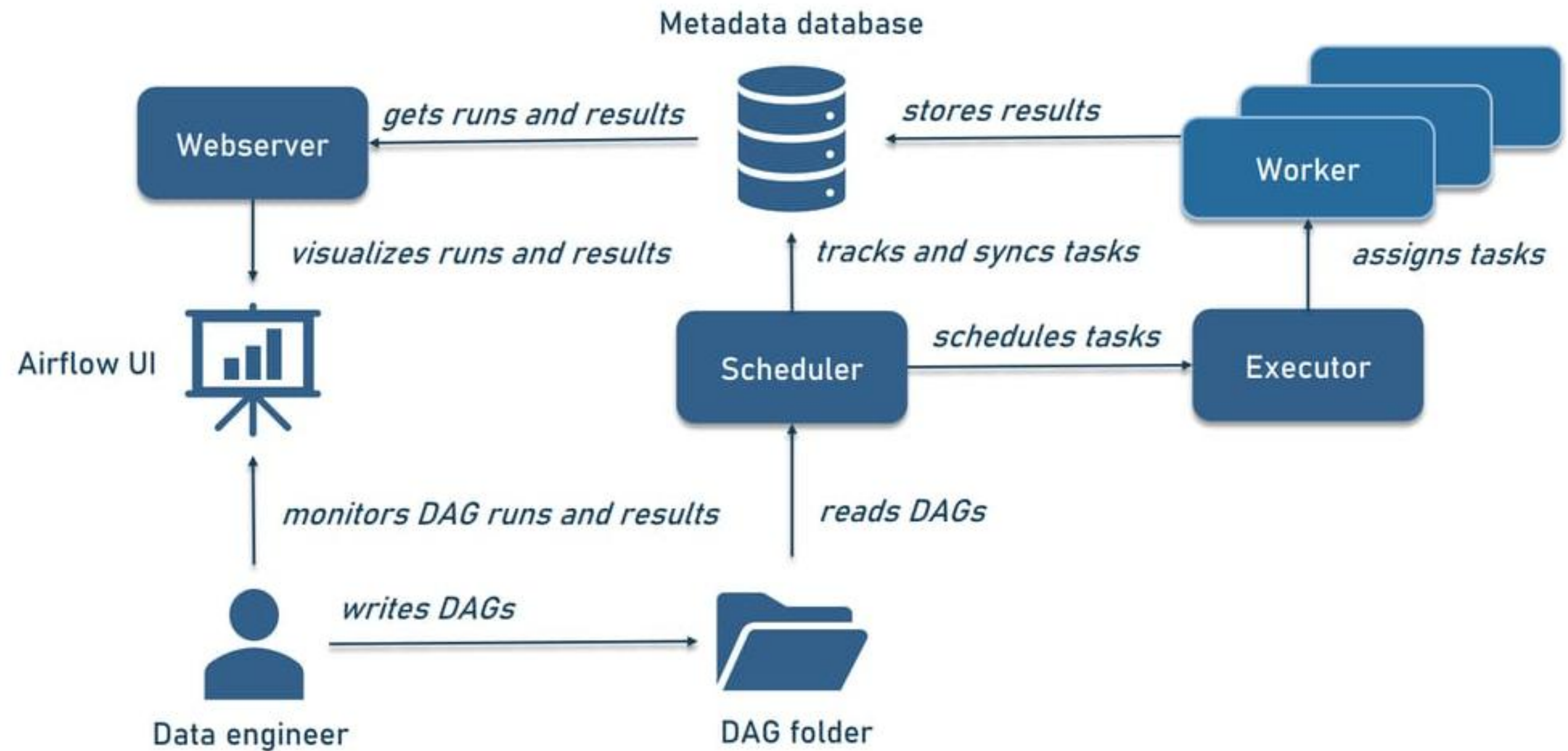
Запуск или перезапуск задачи с одинаковым набором данных для определенной даты в прошлом всегда приводит к одним и тем же результатам. Дополнительно появляется возможность запускать задачи одного DAG для разных временных меток одновременно.



Архитектура Apache Airflow

Apache Airflow
СОСТОИТ ИЗ:

- Web Server
- Metadata Database
- Планировщик (Scheduler)
- Executor
- Worker
- Директория с DAG-файлами



Примеры использования Apache Airflow

Где пригодится Airflow:

- Для проектирования, разработки и обслуживания систем обработки данных
- Для построения несложных витрин данных, отчётов или для подготовки данных для выгрузок, например, для машинного обучения
- Для автоматизации загрузки данных для тестирования приложения, настройки обмена информацией между базами данных или с внешними системами
- Для планирования и мониторинга процессов обработки данных